United States International University-Africa

Education to take you places

NAME: ISHIMWE BENI

ID: 669396

SCHOOL OF SCIENCE AND TECHNOLOGY

DECLARATION

I hereby declare that this project, entitled *"Design and Development of a Cafeteria Ordering System for United States International University – Africa (USIU-A),"* is my original work and has not been submitted to any other university or institution of higher learning for academic credit.

Any work or contributions of other researchers, authors, or practitioners have been duly acknowledged through proper citations and references in accordance with APA guidelines.

This project has been prepared in partial fulfillment of the requirements for the award of the degree in Applied Computer Technology, with a concentration in Cybersecurity, at the United States International University – Africa.

I take full responsibility for the content of this document and affirm that it reflects my personal research, ideas, and efforts in conceptualizing and developing the proposed project.

Student's Name: Ishimwe Beni

Student ID: 669396

Signature:

Date: 23rd January 2026

Supervisor's Name: Prof. Paul Okanda

Signature:

Date:

ACKNOWLEDGEMENT

I wish to express my sincere gratitude to the Almighty God for granting me the strength, wisdom, and resilience to undertake and complete this project successfully. My deepest appreciation goes to my supervisor, Prof. Paul Okanda, whose invaluable guidance, constructive feedback, and encouragement have been instrumental in shaping the direction and quality of this work. Your expertise and patience have provided me with the clarity and motivation needed to bring this project to life. I am equally grateful to the faculty and staff of the School of Science and Technology at United States International University – Africa, for equipping me with the knowledge and skills that have enabled me to carry out this research. Special thanks go to my fellow classmates, colleagues, and friends who have supported me throughout this journey. Your insightful discussions, feedback, and moral encouragement have been of great help.

Finally, I would like to extend my heartfelt appreciation to my family for their unwavering love, prayers, and support. Your encouragement has been my greatest source of strength throughout my academic journey.

ABSTRACT

Managing food orders in a busy university cafeteria presents a significant operational challenge, particularly during peak hours between lectures when hundreds of students compete for limited counter space and preparation time. This project proposes the design and development of a Cafeteria Ordering System for United States International University – Africa (USIU-A). The primary objective of the system is to provide a mobile solution that enables students to browse the cafeteria menu, place orders remotely, and receive real-time status updates - eliminating the need to physically queue at the counter.

The proposed system will be developed as a native Android application using Java and XML, with Firebase Firestore as the cloud-based real-time database, Firebase Authentication for user management, and Firebase Cloud Messaging for push notifications. The application introduces an in-app wallet that allows students to hold a balance and settle orders without cash on every visit. A Smart Pre-order feature enables students to schedule meals in advance around their lecture timetable, including recurring orders by day of the week, with automated wallet deduction enforced by Firebase Cloud Functions at configurable meal cut-off times. An estimated wait time indicator, derived from the current order queue depth, gives students visibility into how long their order will take before committing to it.

A phased methodology will be adopted, encompassing research and requirements gathering, system design, implementation, testing and evaluation, and documentation. The expected outcome is a reliable and efficient ordering platform that reduces peak-hour congestion, improves the dining experience for students, and gives cafeteria staff real-time demand visibility contributing to the broader goal of a smarter, more connected campus at USIU-A.

Table of Contents

DECLARATION **2**

ACKNOWLEDGEMENT **4**

ABSTRACT **5**

1 CHAPTER ONE: INTRODUCTION **9**

1.1 History 9

1.2 Problem Statement 10

1.3 Conclusion 11

2 CHAPTER TWO: LITERATURE REVIEW **13**

2.1 Introduction 13

2.2 Global Perspective 13

2.2.1 UMass Dining App (University of Massachusetts Amherst, USA) 13

2.2.2 NUSmart Dining (National University of Singapore, Singapore) 16

2.3 Regional Perspective 19

2.3.1 UCT Dining App (University of Cape Town, South Africa) 19

2.3.2 Campus Gusto (Nigeria) 22

2.4 Local Perspective 25

2.4.1 University of Nairobi (UoN) Cafeteria, Kenya 25

2.4.2 Strathmore University Cafeteria, Kenya 27

2.5 Strengths and Weaknesses of the Selected Systems 29

2.6 Conclusion 33

3 CHAPTER THREE: AIMS AND OBJECTIVES **35**

3.1 Introduction 35

3.2 General Aims and Objectives 35

3.3 Specific Aims and Objectives 37

3.4 Conclusion 37

4 CHAPTER FOUR: PROPOSED PROJECT **38**

4.1 Introduction 38

4.2 Project Phases 38

4.2.1 Phase 1: Research, Requirements Gathering, and System Design (Weeks 1-3) 38

4.2.2 Phase 2: Development and Implementation (Weeks 4-8) 40

4.2.3 Phase 3: Testing, Evaluation, and Delivery (Weeks 9-10) 42

4.3 Program of work 44
4.3.1 Phase One: Research and Design 44
4.3.2 Phase Two: System Development 46
4.3.3 Phase Three: Testing, Documentation, and Reporting 48
4.4 Gantt Chart 51
4.5 Requirements 2
4.5.1 Hardware Requirements 2
4.5.2 Software Requirements 3
4.5.3 Server Requirements 4
4.5.4 Budget Requirements 5
4.6 Conclusion 6
5 CHAPTER 5: SYSTEM ANALYSIS AND DESIGN 7
5.1 System Models 7
5.1.1 System Architecture Diagram 7
5.1.2 Use Case Diagram 10
5.1.3 Entity Relationship Diagram (ERD) 13
5.1.4 Data Flow Diagram (DFD) 15
5.1.5 Flowchart 3
5.1.6 Activity Diagram 7
5.1.7 Sequence Diagram 9
5.1.8 Class Diagram 13
5.1.9 Wireframes 16
5.2 Functional Requirements 18
5.3 Non-Functional Requirements 20
5.4 Conclusion 22
6 CHAPTER 6: IMPLEMENTATION 24
6.1 Introduction 24
6.2 Frontend Implementation 25
6.3 Backend Implementation 38
6.4 Integration 45
6.5 APIs 51
6.6 Application Deployment 58

6.7 Conclusion 66
7 CHAPTER 7: TESTING AND EVALUATION 68
7.1 Introduction 68
7.2.1 Functional Testing 68
7.2.2 Security Testing 72
7.2.3 Performance Testing 74
7.3: Performance Testing 75
7.3.1 Developer Role Simulation 76
7.3.2 Peer Review Session 77
7.4 Key Findings 78
7.5 Impact Analysis 79
7.6 Conclusion 81
8 CHAPTER 8: CONCLUSION 83
8.1 Introduction 83
8.2 Achievements 84
8.3 Challenges Encountered 85
8.4 Future Work 87
9 REFERENCES 89
10 APPENDIX A: USER MANUAL FOR THE USIU CAFETERIA ORDERING SYSTEM 90
A.1 For Students (Android App) 90
A.2 For Cafeteria Staff (Android App) 92
A.3 For Administrators (Web Panel) 94
11 APPENDIX B: TECHNICAL DEPLOYMENT AND MAINTENANCE GUIDE 95
B.1 Deployment Prerequisites 95
B.2 Environment Variables and Secrets 96
B.3 Deployment Steps 99
B.4 Firestore Composite Indexes 102
B.5 Common Maintenance Tasks 103

1 CHAPTER ONE: INTRODUCTION

University cafeterias are high-traffic service points that must serve large numbers of students within narrow time windows - particularly the short breaks between lectures. United States International University – Africa (USIU-A), as a dynamic and growing institution, hosts a diverse student population that relies on the campus cafeteria as a primary source of meals throughout the day. Yet despite the cafeteria's central role in daily campus life, there is no digital system in place to manage the ordering process. Without a dedicated ordering platform, students must physically queue at the counter during peak hours, often with no information about what is available, how long the wait will be, or whether they will be served before their next lecture begins. Cafeteria staff, on the other hand, have no advance visibility into demand and cannot begin preparation until a student is standing at the counter.

The widespread adoption of smartphones and cloud-based services has made it increasingly feasible to address this kind of operational inefficiency through mobile technology. This project seeks to design and develop a Cafeteria Ordering System for USIU-A - a native Android application that enables students to browse the menu, place orders remotely, track order status in real time, schedule meals in advance, and manage payments through an in-app wallet. The goal is to reduce queue time, improve the dining experience, and give cafeteria staff the demand visibility they currently lack.

1.1 History

The evolution of food ordering systems closely mirrors the broader trajectory of digital technology and mobile computing. In the early decades of institutional food service, ordering was an entirely manual process: students and staff would join a physical queue, select from whatever was displayed or described at the counter, and pay in cash. While straightforward, this model placed the entire burden of demand management on the moment of service, with no mechanism for advance planning on either the customer or provider side. The introduction of digital point-of-sale (POS) terminals in the 1990s marked the first significant shift, replacing handwritten tickets and cash drawers with electronic transaction records. Fast food chains such as McDonald's and KFC were early adopters, using POS systems to accelerate throughput and capture sales data. However,

POS terminals addressed speed at the counter - they did not eliminate the queue or provide customers with any form of advance ordering.

The 2000s brought online ordering to the mainstream. Platforms such as Domino's and Pizza Hut launched web-based ordering systems that allowed customers to place orders from home before arriving for collection or delivery. This was the first widespread separation of the ordering moment from the service moment, demonstrating that demand could be captured and prepared in advance. The concept was extended in the 2010s with the rise of dedicated mobile applications - McDonald's, Starbucks, and a wave of food delivery platforms including Uber Eats and Jumia Food introduced app-based ordering with real-time status tracking and, in some cases, integrated digital payment. In the Kenyan context, mobile money through M-Pesa fundamentally transformed the payment landscape, normalizing cashless transactions even at informal food vendors. Food delivery services such as Jumia Food and Glovo Kenya demonstrated that mobile ordering was viable and in demand in the local market. Nonetheless, the institutional cafeteria setting - particularly within universities - has largely been left behind. Most Kenyan university cafeterias continue to operate on fully manual processes: physical queuing, cash-only transactions, and no digital menu visibility.

This history reveals a consistent pattern: the technology to solve institutional cafeteria queuing has existed for over a decade, but its application in the Kenyan university context remains almost entirely absent. The gap between what is technically possible and what is operationally deployed at institutions like USIU-A is the motivation for this project.

1.2 Problem Statement

The USIU-Africa cafeteria serves a large and growing student population across multiple meal periods each day. Despite its central role in campus life, the cafeteria operates without any digital ordering infrastructure, creating a set of interconnected problems that affect both students and cafeteria staff on a daily basis. During peak hours - particularly between 10:00 AM and 11:00 AM, and again between 1:00 PM and 2:00 PM - queues of twenty to forty students are not uncommon. These windows coincide directly with breaks between lectures, meaning students have a limited and fixed amount of time to obtain a meal and return to class. A queue of this size can easily exceed

the available break time, forcing students to choose between eating and attending their next class on time. The result is missed meals, wasted time, and unnecessary stress that undermines the academic experience. Compounding the queue problem is the complete absence of menu visibility. Students have no way to check what is available before walking to the cafeteria, which means those who arrive to find their preferred meal sold out must either settle for an alternative or leave without eating. There is no mechanism to browse the menu, check item availability, or make an informed choice before committing the time to travel to the counter.

Pre-ordering is equally absent. Food preparation does not begin until a student reaches the counter and places a verbal order, meaning preparation time is added on top of queue time. Students with fixed lecture timetables have no way to schedule a meal in advance and ensure it is ready at a specific time. A student whose lecture ends at 12:15 cannot arrange to have food waiting at 12:30 - they must queue and wait like everyone else regardless of how predictable their schedule is. From the staff perspective, the absence of a digital system means there is no advance visibility into demand. Cafeteria staff cannot know how many portions of a given item will be needed in the next hour, leading to both stock shortages during peak demand and food wastage when demand is overestimated. Without data on order volumes and timing patterns, operational planning remains reactive rather than proactive.

Finally, every transaction requires cash. Students must carry money to every meal, and staff must manage a cash float throughout the day. This adds friction for students and administrative overhead for staff, and stands in contrast to the cashless payment norms that have become standard in Kenyan commercial food service. There is a clear and pressing need for a dedicated mobile ordering system for the USIU-A cafeteria - one that eliminates physical queuing, provides menu visibility, enables advance meal scheduling, and supports cashless payment through an in-app wallet.

1.3 Conclusion

This chapter has established the context and motivation for the proposed project. It has traced the evolution of food ordering systems from manual queuing and cash transactions through digital POS terminals, online ordering, and mobile applications, demonstrating that the technology to

solve institutional cafeteria problems has been available and maturing for over a decade. It has also highlighted that despite this progress at a global and regional level, Kenyan university cafeterias - including that of USIU-Africa - continue to operate on entirely manual processes. The problem statement has identified five specific, interconnected deficiencies in the current USIU-A cafeteria operation: peak-hour queuing that exceeds available break time, no menu visibility before arrival, no pre-ordering capability, no advance demand visibility for staff, and a cash-only payment model that adds friction for all parties. These are not isolated inconveniences but systemic inefficiencies that affect the daily experience of hundreds of students and the operational effectiveness of cafeteria staff.

Together, these observations justify the development of a dedicated mobile cafeteria ordering system for USIU-A. The proposed application will address each identified problem directly - enabling remote ordering, real-time status tracking, advance meal scheduling with recurring options, and cashless payment through an in-app wallet. The subsequent chapters build on this foundation. Chapter Two reviews existing cafeteria and ordering systems globally, regionally, and locally to identify what has been done, what works, and where the gaps remain. Chapter Three formalises the aims and objectives of this project. Chapters Four through Six cover the proposed methodology, system design, and implementation. Chapters Seven and Eight present the testing results and conclusions drawn from the completed work.

2 CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

The purpose of a literature review is to examine existing knowledge, systems, and research that relate to the proposed study. By exploring global, regional, and local contexts, this chapter highlights how cafeteria and food ordering technologies have evolved, their adoption in educational institutions, and the challenges that remain unresolved. The review also identifies the specific gaps that justify the development of a dedicated mobile cafeteria ordering system for United States International University – Africa (USIU-A).

2.2 Global Perspective

Globally, food ordering technology has matured significantly over the past two decades, driven by the widespread adoption of smartphones and cloud-based services. Mobile applications such as those developed by McDonald's, Starbucks, and Uber Eats have demonstrated that customers are willing to shift order placement entirely to their devices, provided the experience is fast, reliable, and integrated with a payment mechanism. These platforms introduced concepts - real-time order tracking, push notifications, and digital wallets - that are now considered standard in commercial food service. Within universities, several institutions in North America and Asia have applied these same principles to their dining services, developing dedicated mobile platforms that allow students to browse menus, place orders, and receive notifications when their meal is ready. Research into these deployments consistently shows reductions in peak-hour congestion, improved student satisfaction, and better demand visibility for cafeteria staff. Despite this progress, most solutions at the global level are deeply integrated with institutional infrastructure and are not easily replicated without significant technical and financial investment.

2.2.1 UMass Dining App (University of Massachusetts Amherst, USA)

The University of Massachusetts Amherst operates a dedicated mobile dining application that serves as the digital interface for its campus-wide dining programme. The app allows students to browse multiple dining locations - including Franklin Dining Commons and Harvest Market - view

real-time open or closed status for each venue, and access item-level nutritional data including healthfulness scores, diet tags (Vegetarian, Halal), and full allergen and ingredient breakdowns. A notification feature lets students subscribe to specific menu items and receive an alert the next time that item is served. Students are able to check dining hall availability and menus before leaving their residence, removing the need to physically visit a location to discover what is being served.

Figure 1 above shows the UMass Dining App's main screen, listing campus dining venues with their real-time open or closed status, operating hours, and a curated Student Favorites banner highlighting popular dishes at Franklin Dining Commons.

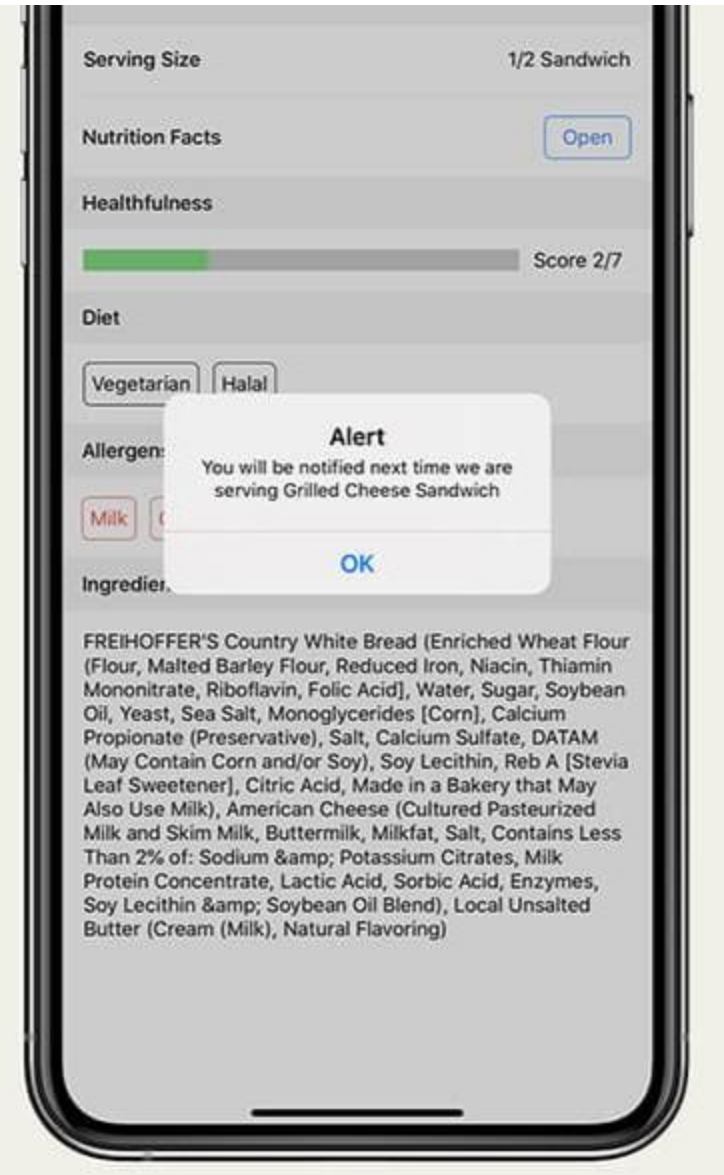


Figure 2 shows the item detail view, where a student can review a full nutrition panel, healthfulness score, dietary suitability tags, allergen labels, and a complete ingredient list, as well as subscribe to an in-app notification for when a specific item is next available.

The UMass Dining App represents a comprehensive institutional food service platform that prioritises transparency and student welfare. The item-level nutritional detail - including healthfulness scoring and allergen tagging - demonstrates a commitment to informed dietary choices that extends well beyond basic menu display. The notification subscription feature is particularly noteworthy as it directly addresses the problem of menu unpredictability, allowing

students to be alerted when a preferred item returns to the rotation without requiring them to check the app daily.

However, the app functions primarily as an information and discovery tool rather than a transactional ordering platform. Students can browse and plan, but cannot place an order in advance for a guaranteed pickup time. The system also depends on UMass's existing dining infrastructure and point-of-sale hardware, making direct adoption by another institution cost-prohibitive. For the USIU Cafeteria Ordering System, the UMass model informs the importance of menu visibility and real-time availability indicators, while highlighting the opportunity to go further with advance ordering and scheduled pre-orders.

2.2.2 NUSmart Dining (National University of Singapore, Singapore)

The National University of Singapore (NUS) developed NUSmart Dining, a mobile application dedicated to campus food ordering. Through the app, students can browse canteen stalls, place orders, save favourite items for quick reordering, and track the status of current and past orders in real time. The order history screen records each transaction with its date, time, amount, and collection status, and flags any refunded transactions clearly. A Favourites tab allows students to curate a personal list of preferred dishes from across multiple stalls - each tagged with a rating, price, and stall category - enabling repeat orders to be placed in a few taps rather than navigating the full menu each time.

Figure 3 above shows the NUSmart Dining Order Status screen, where a student can review their full transaction history - each entry displaying the date, time, item name, stall category, amount paid, and collection outcome, including a clearly labelled refund entry.

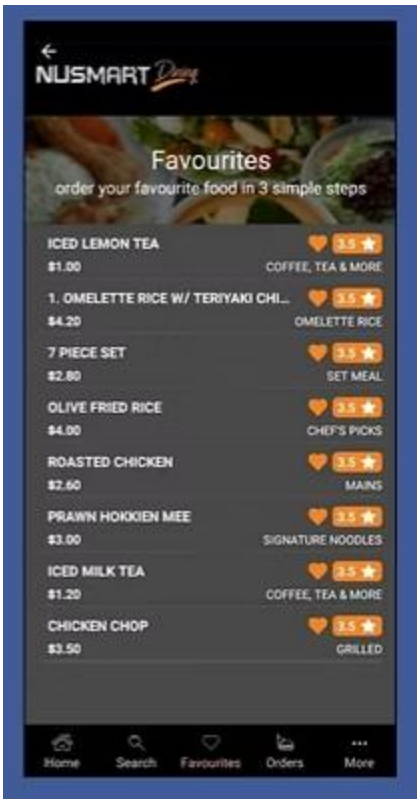


Figure 4 shows the Favourites screen, presenting the student's saved dishes with star ratings, prices, and stall categories across a scrollable list, enabling fast repeat ordering without browsing the full catalogue.

The NUSmart Dining app stands out for its combination of order tracking and personalisation. The order history with refund tracking provides students with a clear audit trail of their spending, while the Favourites feature directly reduces the friction of repeat ordering - a pattern common in cafeteria settings where students tend to reorder the same meals regularly. Both features are directly relevant to the problem this project addresses, where students currently have no record of past orders and must browse the full menu from scratch on every visit.

The system's limitations from a replication standpoint are similar to other institutionally integrated platforms: it depends on NUS's campus payment infrastructure and stall operator agreements. The app also does not appear to support advance scheduling of orders for a future meal slot. For the USIU Cafeteria Ordering System, the NUSmart Dining model informs the order history and wallet transaction tracking features, while the absence of pre-order scheduling in the NUS platform highlights an opportunity for this project to go further.

2.3 Regional Perspective

Across Africa, digital transformation in food service has been largely driven by commercial delivery platforms - Jumia Food, Glovo, and similar services have demonstrated strong mobile ordering adoption in major cities including Nairobi, Lagos, and Cape Town. However, the translation of these commercial models into institutional settings, particularly university cafeterias, has been slow. Most African universities continue to operate cafeteria services on fully manual processes, with cash transactions, physical queuing, and no digital interface between students and food service staff. The barriers to adoption are both financial and technical: purpose-built institutional food ordering platforms require ongoing maintenance, reliable internet infrastructure, and staff training. Nonetheless, several regional institutions have taken steps toward digitising aspects of their food service, offering useful models from which the USIU Cafeteria Ordering System can draw lessons about what is feasible and effective in the African university context.

2.3.1 UCT Dining App (University of Cape Town, South Africa)

The University of Cape Town (UCT) operates a dedicated mobile dining application that manages student access to campus dining halls through a QR code and meal plan system. Each student is issued a personal QR code linked to their student account, which is presented at the dining hall entrance to be scanned for admission or to redeem a meal voucher. The app's Settings section displays the student's personal details - including their residence hall and account status - alongside a meal plan summary indicating which meals (Breakfast, Lunch, Dinner) are covered under their plan. The bottom navigation provides quick access to Home, Vouchers, Scan, Show, and Settings, keeping all dining functions within a single interface.

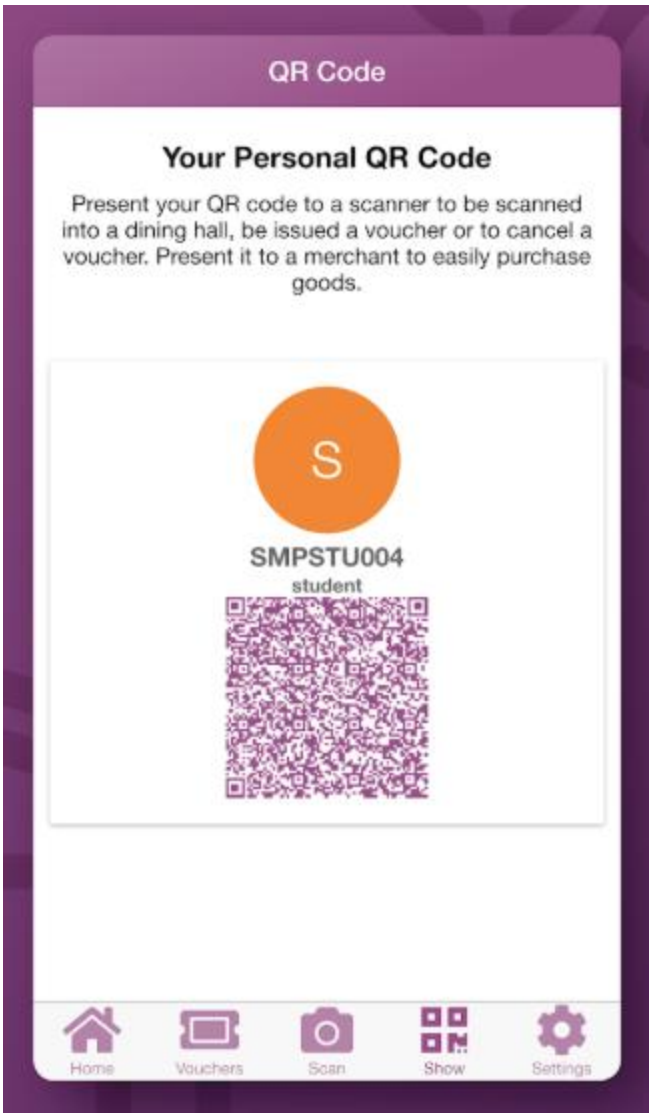


Figure 5 above shows the UCT Dining App's QR Code screen, where the student's personal QR code is displayed ready to be scanned at a dining hall entrance or merchant point - the primary access mechanism linking the student's identity to their dining entitlement.

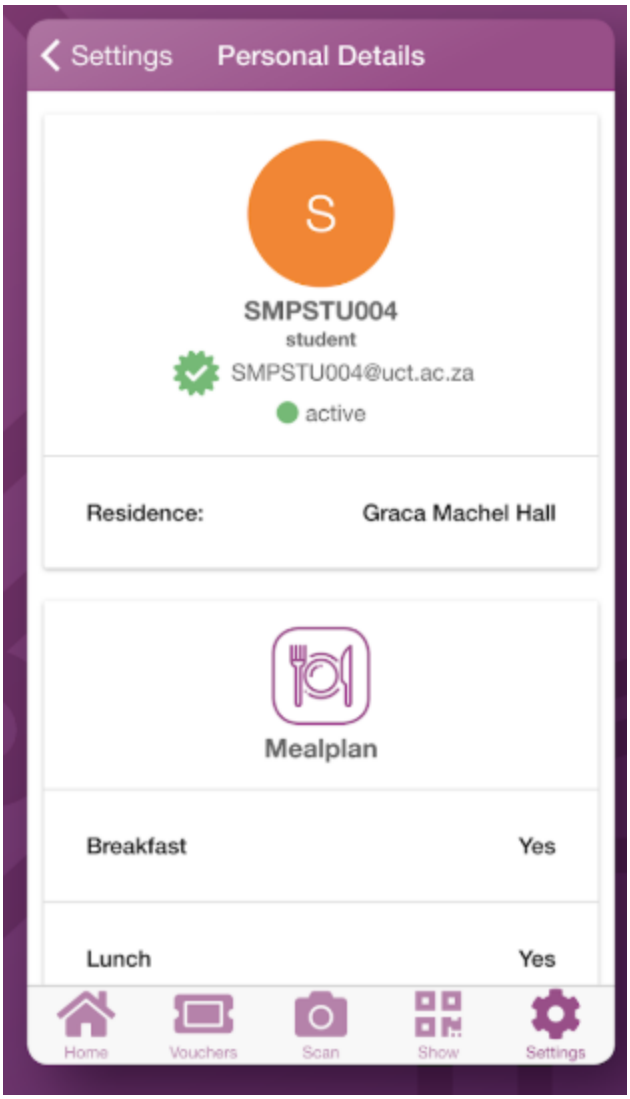


Figure 6 shows the Personal Details screen, confirming the student's account status as active, their assigned residence (Graca Machel Hall), and their meal plan coverage, with Breakfast and Lunch both enabled under the current plan.

The UCT Dining App demonstrates a mature institutional dining model built around identity-linked access control and structured meal plans. The QR code mechanism eliminates the need for a physical student card at every dining transaction, while the meal plan display gives students clear visibility of their covered meals. The voucher system adds flexibility for ad-hoc purchases beyond the standard plan.

The system's focus is primarily on access and entitlement management rather than advance ordering or queue reduction. Students present their QR code on arrival rather than placing orders in advance, meaning the app does not address peak-hour queuing directly. There is no pre-ordering, estimated wait time, or push notification for order readiness. For the USIU Cafeteria Ordering System, the UCT model informs the importance of linking a student's identity to their dining balance, while highlighting pre-ordering, real-time order tracking, and push notifications as features that a more transactional platform can deliver.

2.3.2 Campus Gusto (Nigeria)

Campus Gusto is a subscription-based mobile meal application designed specifically for university students in Nigeria. Rather than operating a single institutional cafeteria, the platform connects students to multiple vendors located near their campus. Students subscribe to their preferred vendors, browse available menus, and schedule daily meal deliveries to their location, eliminating the need to physically queue at a canteen. Orders are tracked through a dedicated Orders tab, where each entry displays the vendor name, scheduled date and time, and current order status. An Account section provides access to personal information, login and security settings, and app support resources.

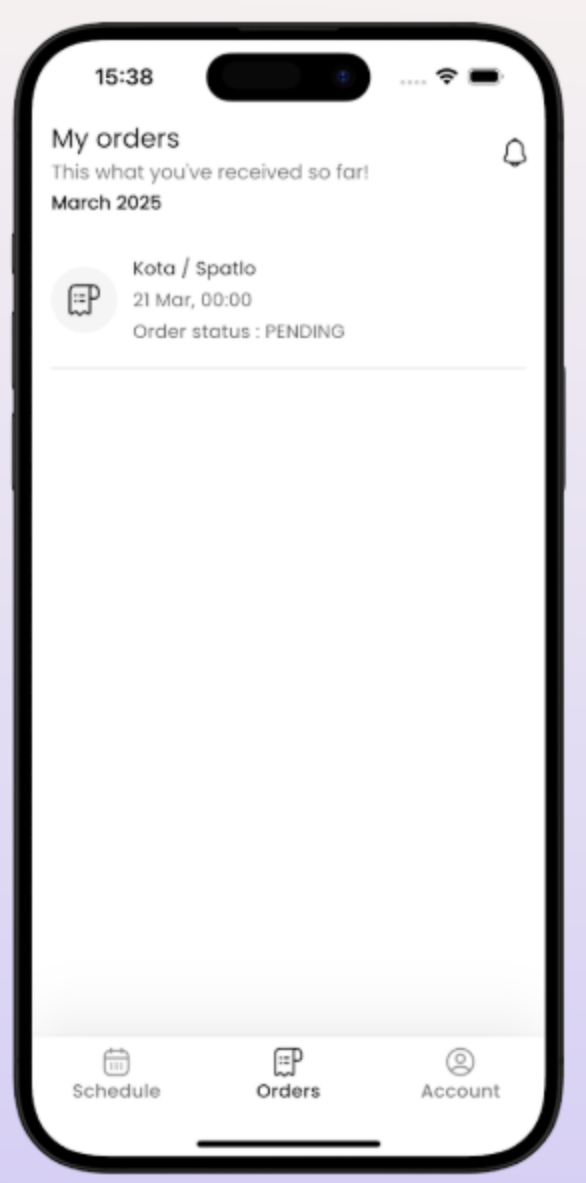


Figure 7 above shows the Campus Gusto My Orders screen, where a student can review their scheduled orders - each displaying the vendor name, scheduled delivery date, and current status (Pending) - giving a clear view of upcoming and recent meal requests.

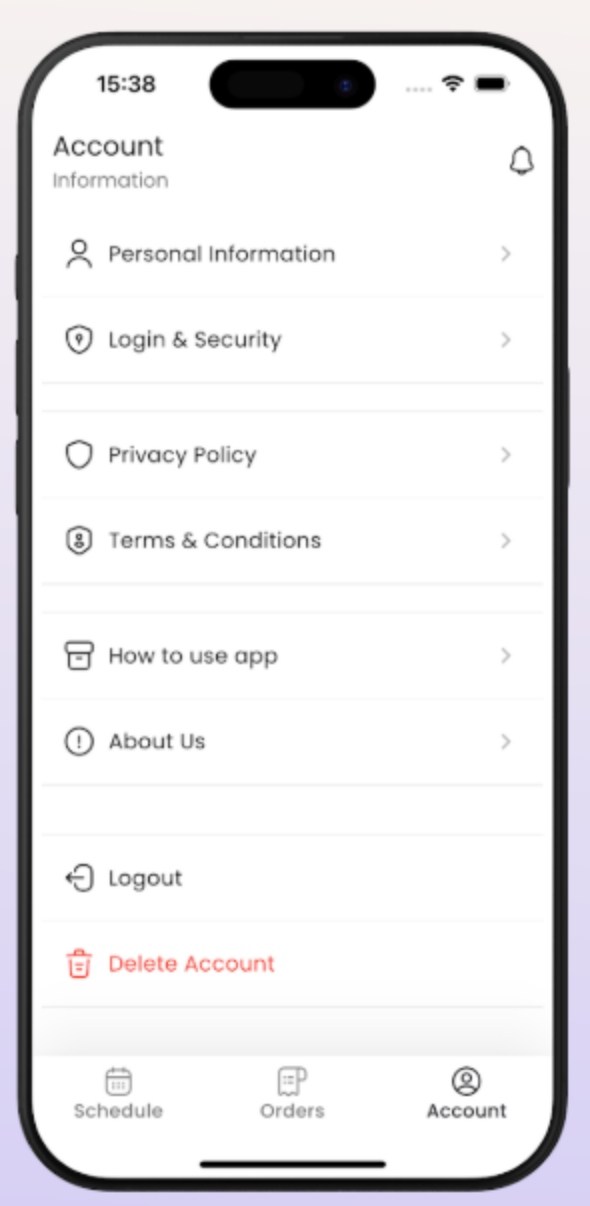


Figure 8 shows the Account screen, which consolidates personal information management, login and security settings, privacy policy, and support links into a single profile area, alongside Logout and Delete Account options.

Campus Gusto addresses one of the core problems common to African university food service: long physical queues at peak hours. By moving order placement off the counter and into a mobile scheduling interface, the app allows kitchen preparation to begin before the student arrives, and delivery scheduling removes the student from the queue entirely. The multi-vendor model also

reflects the practical reality of many African campuses, where food service is distributed across multiple informal or semi-formal vendors rather than a single institutional cafeteria.

The platform's limitations in the context of this project are primarily structural. Campus Gusto is a commercial third-party service rather than an institutionally managed system, meaning it does not integrate with a university wallet or student account. Payment and vendor relationships are managed externally. There is no in-app wallet, no staff-facing order management interface, and no push notification for order readiness visible in the reviewed version. For the USIU Cafeteria Ordering System, Campus Gusto validates the demand for mobile meal scheduling in the Nigerian - and by extension African - university context, while the absence of an institutional wallet and real-time staff coordination underscores the features this project adds.

2.4 Local Perspective

In Kenya, the food service landscape has been shaped significantly by the rapid growth of mobile money and delivery platforms, with M-Pesa integration normalising cashless transactions even at small food vendors. However, this commercial progress has not translated meaningfully into university cafeteria operations. Kenyan universities, including some of the country's most digitally progressive institutions, continue to rely on manual queuing and cash payment in their cafeterias. The following two local examples illustrate both the current state of university food service in Kenya and the limited steps that have been taken toward digitisation

2.4.1 University of Nairobi (UoN) Cafeteria, Kenya

The University of Nairobi operates cafeteria facilities across its main and satellite campuses that function entirely on manual processes. Students queue at the counter, order verbally, pay in cash, and wait for preparation. There is no digital menu, no mobile ordering interface, and no advance visibility into what is available on a given day. The cafeteria at the main Nairobi campus serves a very large student population and is known for significant peak-hour congestion, particularly during the lunch period. UoN represents the current operational baseline for university food service in Kenya - functional but entirely analogue.

Uji
AVAILABLE
Free juice
on weekend
after meals
pepsi

[Figure 9: UoN main campus cafeteria - physical queuing at the counter during peak lunch hour]

The UoN cafeteria, despite serving one of East Africa's largest university communities, has not been the subject of any documented digitisation effort. Its continued reliance on cash and manual queuing reflects both the resource constraints common to public universities in Kenya and the absence of a strong institutional mandate for modernising food service. From a systems perspective, it offers no technical model to build upon - its value in this review is as a baseline that defines what the status quo looks like for most Kenyan university cafeterias.

The scale of the queuing problem at UoN, serving thousands of students with limited counter capacity and no pre-ordering, directly mirrors the problem at USIU-A, albeit at greater magnitude. This parallel reinforces the relevance of the proposed system within the local university context. If a smaller, more resource-capable institution like USIU-A does not address this problem proactively, the trajectory points toward the same entrenched manual operations seen at UoN. The proposed USIU Cafeteria Ordering System is designed to break from that trajectory by introducing mobile ordering, advance scheduling, and cashless payment from the outset.

2.4.2 Strathmore University Cafeteria, Kenya

Strathmore University, one of Kenya's most technology-forward private universities, introduced QR-code-based digital menu boards in its cafeteria following the COVID-19 pandemic. Students can scan a QR code posted at the entrance to view the day's menu on their phone - a static PDF that is updated each morning. This eliminated the need for printed daily menus and gave students the ability to check offerings before joining the queue. No ordering functionality is provided through the QR code; it serves purely as a display mechanism. Queuing, ordering, and payment at Strathmore remain entirely manual.

[Figure 10: Strathmore University cafeteria - QR code menu board at the entrance with static daily menu PDF]

The Strathmore QR menu represents the most advanced local example of digital engagement with university food service, yet it illustrates how limited that engagement remains. The single innovation - moving the menu from a physical board to a phone screen - does provide genuine value: students can check availability before committing to the queue. However, it addresses only one of the five problems identified in this project's problem statement. Queuing, pre-ordering, staff demand visibility, and cashless payment are all left entirely unchanged.

Strathmore's initiative is nonetheless instructive in two ways. First, it demonstrates that even a minimal digital intervention (a QR code linking to a PDF) is adopted and used by students, confirming that the target audience is willing to engage with digital food service tools on their phones. Second, it establishes the immediate local benchmark - the most any Kenyan university has currently done is display a menu digitally. The USIU Cafeteria Ordering System is designed to move well beyond this baseline, delivering a fully interactive, order-placed, wallet-enabled, and pre-schedulable mobile platform that addresses the complete problem rather than a single dimension of it.

2.5 Strengths and Weaknesses of the Selected Systems

Section 2.5 presents a critical evaluation of the strengths and weaknesses of the six selected cafeteria and food ordering systems. By analysing solutions across global, regional, and local contexts, this section identifies the core functionalities each system delivers, the limitations each leaves unaddressed, and the cumulative gap that the proposed USIU Cafeteria Ordering System is designed to fill.

Table 1: Strengths and Weaknesses of the Selected Cafeteria and Food Ordering Systems

System	**Strengths**	**Weaknesses**
MIT Dining App (USA)	- Full menu visibility across all dining halls	- Requires deep institutional IT integration
	- Nutritional information per item	- No advance order scheduling
	- Meal plan balance management	- High cost to replicate without existing infrastructure
	- Real-time seat availability	- No estimated wait time for order pickup
NUS uNivUS (Singapore)	- Mobile ordering with queue status visibility	- Dependent on NUS campus card ecosystem
	- Digital payment via linked student card	- No recurring scheduled orders

System	Strengths	Weaknesses
	- Estimated wait time before ordering	- Complex integration with individual stall operators
	- Multi-service campus app	
UCT Hungry Rhino (South Africa)	- Web-based pre-ordering with collection slots	- Web-only, no native mobile app
	- Advance demand visibility for kitchen staff	- No push notifications for order ready
	- Directly addresses peak-hour queuing	- No persistent wallet - per-transaction payment only
	- Validated in African university context	- No recurring scheduled orders
UNILAG Cafeteria (Nigeria)	- Cashless NFC card payment at counter	- No mobile ordering interface
	- Digital menu board for basic visibility	- Physical queuing unchanged
	- Reduces cash handling for staff	- No pre-ordering or scheduling

System	Strengths	Weaknesses
	- Accepted by large student population	- Card top-up requires physical visit
UoN Cafeteria (Kenya)	- Establishes the local manual baseline	- Fully manual - no digital component
	- Reflects common operational context	- Cash only, no menu visibility
	- Large-scale evidence of queuing problem	- No ordering, tracking, or scheduling
Strathmore Cafeteria (Kenya)	- QR code menu adopted by students	- Display only - no ordering functionality
	- Proves students will use phone for food info	- Queuing and payment remain manual
	- Low-cost, low-friction digital step	- No wallet, no pre-order, no staff visibility
Proposed USIU System	- Native Android app - mobile-first	- Requires internet connectivity
	- Real-time order tracking and status updates	- Initial setup and menu data entry required

System	Strengths	Weaknesses
	- Estimated wait time from live queue depth	- Staff training needed for order management
	- Smart Pre-order with recurring scheduling	
	- In-app wallet with automated cut-off deduction	
	- Role-based access for students and staff	
	- Built specifically for USIU-A context	

Analysis

The systems reviewed span a wide spectrum from fully manual operations to sophisticated mobile platforms. Global systems such as the MIT Dining App and NUS uNivUS demonstrate what is possible when institutional resources and infrastructure are aligned behind a digital food service strategy. Both deliver genuine improvements to student experience through menu visibility, digital payment, and order tracking. However, neither offers advance meal scheduling or recurring order functionality, and both are too deeply embedded in their respective institutional ecosystems to serve as direct models for a student project at a mid-sized university.

Regional systems, particularly UCT Hungry Rhino, validate the core concept of this project - pre-ordering with collection slots demonstrably reduces peak-hour congestion in an African university cafeteria. This is the most directly relevant precedent in the literature. However, its web-only

delivery, lack of push notifications, and absence of a persistent wallet leave clear room for improvement in a native mobile implementation.

Local systems reveal the extent of the gap in the Kenyan university context. UoN represents the status quo: entirely manual, with no digital engagement whatsoever. Strathmore represents the current frontier of local innovation: a QR code menu that proves student willingness to use their phone for food-related information, but offers no ordering capability. No Kenyan university cafeteria currently offers mobile ordering, digital payment through an app, real-time order tracking, pre-scheduling, or an in-app wallet.

The proposed USIU Cafeteria Ordering System addresses all six dimensions identified in the gap analysis - pre-ordering, recurring scheduling, real-time tracking, native mobile delivery, in-app wallet, and local university context - making it the first system of its kind in the Kenyan university landscape.

2.6 Conclusion

The literature reviewed in this chapter demonstrates that while food ordering technology has advanced considerably at the global level, its application within university cafeterias - particularly in the African and Kenyan context - remains severely limited. Global platforms such as MIT Dining and NUS uNivUS prove that mobile ordering, digital wallets, and real-time tracking are technically achievable and operationally effective in university dining environments. Regional and local systems, however, reveal that these capabilities have not yet reached Kenyan institutions, with the most advanced local example being a static QR code menu that offers no ordering functionality. This gap is both a challenge and an opportunity. It confirms that the problem identified in Chapter One - physical queuing, no menu visibility, no pre-ordering, cash-only transactions, and no staff demand visibility - is not unique to USIU-A, but is in fact the norm across local universities. It also confirms that no existing local system has attempted to address this problem comprehensively through a mobile-first platform.

The proposed USIU Cafeteria Ordering System is designed to close this gap by delivering capabilities that exceed those of every local and regional system reviewed: native Android ordering, real-time status tracking with estimated wait time, advance meal scheduling with

recurring options, automated wallet deduction at cut-off, and role-based access for both students and staff. The next chapter formalises the aims and objectives that guide this system's development.

3 CHAPTER THREE: AIMS AND OBJECTIVES

3.1 Introduction

A successful system development project requires a clear sense of direction, which can only be achieved through well-defined aims and objectives. These act as the guiding framework that shapes the overall purpose of the study, the methods adopted, and the outcomes expected. Without properly established aims and objectives, projects risk deviating from their intended purpose, resulting in inefficiency and lack of focus. In the context of this study, which seeks to address the operational inefficiencies of the USIU-Africa cafeteria through a mobile ordering system, the aims and objectives provide the foundation for ensuring that the proposed solution directly responds to the problems identified in the earlier chapters. The development of a cafeteria ordering application requires not only a broad aim that captures the overall vision of the project but also specific objectives that outline the practical steps necessary to realise this vision. This section therefore introduces the general aim of the study and breaks it down into specific objectives that will guide the design, development, testing, and documentation of the system. Together, they serve as a structured roadmap that ensures the project achieves its intended goals in a systematic and measurable manner.

3.2 General Aims and Objectives

The general aim of this project is to design, develop, and evaluate a cafeteria ordering system for United States International University – Africa (USIU-A), comprising a native Android application for students and cafeteria staff, and a web-based admin panel for system administrators. This initiative seeks to directly address the documented inefficiencies faced by students and cafeteria staff in the current manual ordering process - eliminating physical queuing, providing menu visibility before arrival, enabling advance meal scheduling, and introducing a cashless in-app wallet for frictionless payment. By replacing the existing counter-based, cash-only process with a mobile-first platform supported by centralised administrative control, the project aims to reduce peak-hour congestion, improve the dining experience for students, and give cafeteria staff

the real-time demand visibility they currently lack. Ultimately, this endeavor contributes to the broader goal of building a smarter, more connected campus at USIU-A.

A comprehensive review of existing literature will be conducted to establish a strong theoretical and practical foundation for this project. This survey will critically analyse prior research and existing implementations of food ordering and cafeteria management systems, with a specific focus on their application within educational institutions. It will evaluate the approaches taken by global, regional, and local systems, assess their respective strengths and limitations, and identify the usability principles and technical patterns most relevant to a university cafeteria context. This process will ensure the project is informed by established practice and will clearly define the gap that the proposed USIU-A system is designed to fill.

The design and development of the system will be guided by a structured architectural framework to ensure robustness, usability, and data integrity. The system will be delivered as two components: a native Android application serving students and cafeteria staff, and a web-based admin panel built with Next.js for system administrators. The backend will use Firebase Firestore as the cloud-based real-time database, Firebase Authentication for user management across all three roles, and Firebase Cloud Messaging for push notifications. Key functionalities include a browsable and filterable menu, a shopping cart with wallet or cash payment selection, real-time order status tracking with estimated wait time, a Smart Pre-order feature with recurring scheduling and automated cut-off processing via Supabase Edge Functions, an in-app wallet managed by cafeteria staff, and admin-controlled menu management and staff account creation. The design process will prioritise a user-centred approach, ensuring the system is tailored to the specific operational context of the USIU-A cafeteria.

The final phase of the project will involve rigorous testing and evaluation of the developed prototype to validate its effectiveness and readiness for use. Testing will encompass both technical performance and user acceptance. Technically, the application will be assessed for order flow reliability, real-time synchronization speed, wallet transaction atomicity, pre-order cut-off accuracy, and performance under load. For user-centric evaluation, representative students and cafeteria staff will be recruited to participate in acceptance testing and provide feedback on ease of use, reliability, and overall satisfaction. The insights gained from this evaluation will be used to

demonstrate the extent to which the system meets its core objective of transforming the USIU-A cafeteria experience.

3.3 Specific Aims and Objectives

The specific aims and objectives of this project are to:

1. Conduct a literature survey on existing representative cafeteria and food ordering systems in order to identify their strengths and weaknesses

2. Design and develop a prototype Android application that provides real-time order placement and tracking, advance meal scheduling with automated wallet deduction, and role-based access in a secure, cloud-based Firebase architecture.

3. Test and evaluate the prototype cafeteria ordering system in order to assess the extent to which it resolves the issues above.

3.4 Conclusion

This chapter has outlined the aims and objectives that will guide the development of the proposed cafeteria ordering system for United States International University – Africa (USIU-A). The general aim of the study was established as the creation of a mobile-based solution to address the operational inefficiencies of the current manual cafeteria process. This broad aim was then broken down into specific objectives that focus on surveying existing systems, designing and developing a functional prototype with real-time ordering, advance scheduling, and an in-app wallet, and finally testing and evaluating the system against the problems identified in Chapter One. The formulation of these aims and objectives provides a structured roadmap for the project. They ensure that each stage of the design and development process remains aligned with the central problem while laying the groundwork for the practical decisions that follow. The objectives highlight the project's emphasis on user-centred design, technical robustness through Firebase's real-time and transactional capabilities, and operational relevance to the specific context of the USIU-A cafeteria. The following chapters translate these objectives into a concrete plan, system design, and implementation.

4 CHAPTER FOUR: PROPOSED PROJECT

4.1 Introduction

The development of any software system requires a structured and systematic approach that ensures the final product meets both user needs and technical requirements. One of the most effective ways of achieving this is by organizing the project into distinct phases that guide the progression of work from the initial idea to the final deployment. Each phase plays a critical role in reducing risks, clarifying objectives, and ensuring that the project maintains focus throughout its lifecycle. For the proposed cafeteria ordering system at United States International University – Africa (USIU-A), the project has been divided into three phases that align with standard software development practices while also reflecting the unique requirements of a Firebase-backed, Android-native application. These phases provide a step-by-step pathway that begins with research and system design, advances through implementation of the ordering, wallet, and pre-order features, and concludes with testing, evaluation, and documentation.

The purpose of this chapter is to present these proposed phases in detail, highlighting the activities carried out in each stage, their significance, and the expected outcomes. By structuring the project into clear phases, it becomes possible to manage resources effectively, monitor progress, and ensure that the final system is robust, user-friendly, and operationally ready.

4.2 Project Phases

The project is organized into three sequential phases, each with distinct objectives, activities, and deliverables designed to progress systematically from concept to deployed system. This phased approach provides structure to the development process, enables checkpoint evaluations ensuring quality before proceeding, and accommodates the iterative nature of software development where later discoveries may inform refinements to earlier decisions.

4.2.1 Phase 1: Research, Requirements Gathering, and System Design (Weeks 1-3)

Objective: Establish comprehensive understanding of user needs, technical requirements, and system architecture before beginning implementation, ensuring development efforts build the right system correctly.

Duration: 3 weeks (approximately 45-60 hours of effort)

Key Activities:

The first phase begins with completion of background research documented as Chapters 1-2 of this report-understanding the problem space through observation of current cafeteria operations, informal interviews with students and cafet

eria staff to validate assumptions, and literature review of existing solutions establishing best practices and common pitfalls. This research phase, largely completed concurrently with early project planning, informs all subsequent design decisions.

Requirements gathering formalizes user needs into specific, testable system requirements. Functional requirements define what the system must do: users shall be able to browse menus, add items to cart, place orders, make payments, earn loyalty points, track order status, and view history. Non-functional requirements specify quality attributes: performance (menu load time <2 seconds), security (encrypted payment information, secure authentication), usability (intuitive interface requiring minimal training), and compatibility (support Android 8.0+, iOS 12+ through Flutter compilation). Requirements are documented in a structured format enabling traceability from need through implementation to testing.

System design translates requirements into technical architecture. The high-level architecture defines major components and their relationships: Flutter mobile application (presentation tier), Firebase Cloud Firestore (data tier), M-Pesa Daraja API (payment integration), and React web dashboard (administrative interface). Detailed design specifies data models for users, menu items, orders, loyalty points, and transactions; defines API contracts between application and backend services; plans database schema including collections, document structures, and indexing strategy; and creates UI/UX wireframes for key screens establishing visual hierarchy and user flows before coding begins.

Technology stack finalization occurs during this phase. While Flutter and Firebase are predetermined strategic choices, specific decisions about state management approach (Provider, Riverpod, or BLoC pattern), navigation framework, UI component libraries, and third-party packages require evaluation and selection based on project needs. A prototype development

environment is established and configured: Flutter SDK installation, Android Studio or VS Code setup with Flutter extensions, Firebase project creation and configuration, Daraja API sandbox account registration, and git repository initialization for version control.

Phase 1 Deliverables:

Formalized requirements document with functional and non-functional specifications

System architecture diagram showing major components and data flows

Database schema design for Cloud Firestore collections

UI/UX wireframes for minimum 8-10 key screens (login, menu browsing, cart, checkout, order tracking, profile, loyalty, admin dashboard)

Technical specification document detailing technology choices and rationale

Configured development environment ready for coding

Chapters 1-3 of project report (introduction, literature review, objectives)

Success Criteria: Design documents are reviewed and approved by project supervisor, wireframes validated with potential users showing interface comprehensibility, development environment successfully builds and runs a "Hello World" Flutter application connecting to Firebase.

4.2.2 Phase 2: Development and Implementation (Weeks 4-8)

Objective: Transform design specifications into fully functional mobile application and administrative dashboard, implementing core features iteratively with continuous testing ensuring quality throughout development.

Duration: 5 weeks (approximately 75-100 hours of effort)

Key Activities:

The development phase follows an iterative approach building features incrementally rather than attempting complete implementation before any testing. This reduces risk of late-stage discovery of fundamental flaws and enables early user feedback incorporation. Development begins with foundational infrastructure: Firebase project configuration including Firestore security rules,

authentication setup, and cloud function deployment if needed; basic Flutter project structure establishing folder organization, navigation framework, and state management architecture; and common UI components (buttons, cards, input fields) following consistent design system.

User authentication implementation provides the entry point: Firebase Authentication integration for email/password registration and login, user profile creation in Firestore upon registration, session management maintaining logged-in state across app restarts, and basic profile viewing/editing functionality. This foundational feature enables all subsequent user-specific functionality (order history, loyalty points, personalized preferences).

Core ordering functionality development proceeds through logical increments. Menu browsing displays cafeteria offerings retrieved from Firestore with categories, images, descriptions, and prices; search and filtering enable users to find items efficiently. Shopping cart management allows adding/removing items, adjusting quantities, viewing subtotals including tax/fees if applicable, and applying loyalty point discounts. Order placement captures delivery/pickup preferences, special instructions, and scheduled pickup times, submits orders to Firestore creating order documents, and triggers appropriate workflows (staff notifications, inventory updates).

Payment integration represents the most technically complex component. M-Pesa integration via Daraja API requires: obtaining API credentials and understanding authentication flows (OAuth), implementing STK Push for customer-initiated payments where users receive payment prompts on their phones, handling payment callbacks confirming transaction success/failure, recording payment transactions in Firestore linked to orders, and implementing proper error handling for network failures, insufficient funds, or timeout scenarios. Sandbox testing with test M-Pesa wallets validates payment flows before any real money transactions.

Loyalty points system implementation tracks earned points (automated when payments complete), displays point balances in user profiles, enables point redemption creating discounts on orders, and establishes points expiration policies if required. Order tracking functionality shows real-time order status progression (Received → Preparing → Ready), sends push notifications on status changes using Firebase Cloud Messaging, and provides order history viewing with reorder capability for convenience.

The administrative dashboard development creates web interface for cafeteria staff: order queue display showing pending orders requiring preparation, status management enabling staff to update order states, menu management for adding/editing/removing items and marking items unavailable, basic analytics showing daily orders, revenue, and popular items, and user management if needed for staff access control.

Phase 2 Deliverables:

Functional Flutter mobile application with all core features implemented

M-Pesa payment integration working in sandbox environment

Loyalty points earning and redemption functional

Order tracking with real-time status updates operational

React/Next.js administrative dashboard deployed

Firestore database running with proper security rules

Firebase Authentication active with user registration/login working

Version control repository with regular commits documenting progress

Chapter 6 technical documentation (implementation details, code explanations, challenges)

Success Criteria: Application runs on physical Android devices without crashes, complete ordering workflow (browse → add to cart → checkout → payment → tracking) functions end-to-end, M-Pesa payments process successfully in sandbox, loyalty points calculate and apply correctly, administrative dashboard displays orders and enables status updates.

4.2.3 Phase 3: Testing, Evaluation, and Delivery (Weeks 9-10)

Objective: Validate system functionality, performance, and user satisfaction through comprehensive testing, gather user feedback, assess objective achievement, and finalize project documentation for submission.

Duration: 2 weeks (approximately 30-40 hours of effort)

Key Activities:

The testing phase executes systematic validation across multiple dimensions. Functional testing verifies each feature works correctly: all screens load and display appropriate content, buttons and interactions respond as expected, data saves and retrieves accurately from Firestore, payment processing completes successfully, loyalty points calculate properly, and order status updates reflect in app. Test cases are documented with expected vs. actual outcomes, identifying defects requiring correction.

Performance testing measures system responsiveness: app launch time from tap to first screen display, menu loading time across varying item counts, order submission processing duration, payment API response times, and overall application memory and CPU usage. These metrics are compared against targets defined in Chapter 3 objectives (e.g, <3 second launch time) with optimization if targets aren't met.

Usability testing recruits actual users (minimum 15 student volunteers per Objective 8) to complete realistic scenarios: discover and download the app, create an account, browse menus and selected preferred items, place an order with payment, track order status, and provide feedback via surveys. Observations identify user interface confusion points, navigation difficulties, or workflow friction not apparent to the developer. Satisfaction ratings quantify overall user experience measuring against 75% satisfaction target.

Security testing verifies proper authentication (unauthorized users cannot access protected features), sensitive data protection (passwords hashed, payment information encrypted), and Firestore security rules preventing unauthorized data access. While not penetration testing, basic validation ensures no obvious vulnerabilities.

Bug fixing and refinement address identified issues. Critical bugs blocking core function receive immediate priority, medium-severity usability issues are addressed if time permits, and minor cosmetic issues are documented for potential future improvement but may remain in v1.0 given time constraints. The goal is stable, functional system, not perfection.

Documentation finalization completes all project report chapters: Chapter 4 (this proposed project plan), Chapter 5 (system design diagrams and specifications), Chapter 6 (implementation documentation with code explanations), Chapter 7 (testing results and evaluation), and Chapter 8

(conclusion and reflection). User documentation is created including basic app usage guide and administrative dashboard manual if stakeholders request.

Deployment preparation packages the application for distribution: generating signed APK for Android installation, creating Play Store listing if publishing (optional given academic context), deploying Firebase backend to production environment from development, and documenting deployment process for reproducibility. Final demo/presentation materials are prepared for project presentation.

Phase 3 Deliverables:

Comprehensive test results documentation (functional, performance, usability, security)

User testing feedback from 15+ participants with satisfaction metrics

Bug reports with resolution status

Completed project report chapters 4-8

User documentation and quick-start guides

Deployed application (APK or app store link)

Project presentation slides/demo materials

Final reflection on objective achievement against Chapter 3 definitions

Success Criteria: All critical bugs resolved or documented with workarounds, performance targets from Chapter 3 met or variance explained, user satisfaction ≥75%, all objectives from Chapter 3 assessed with evidence, project report complete and ready for academic submission.

4.3 Program of work

The following tables provide granular week-by-week activity breakdowns for each project phase, complementing the narrative descriptions in Section 4.2.

4.3.1 Phase One: Research and Design

i. Research

Objective:

To understand the cafeteria ordering challenges at USIU and review existing global and regional food ordering systems.

Tasks:

Review global systems; review regional systems; observe cafeteria operations; conduct student interviews; conduct staff interviews.

Deliverables:

Research findings report; identified gaps in current solutions.

Timeline:

Week 1

ii. Requirements Gathering

Objective:

To define what the system should do.

Tasks:

Identify functional requirements; define menu browsing; define ordering; define wallet; define pre-order; define staff management; identify non-functional requirements; define performance requirements; define security requirements; define usability requirements.

Deliverables:

Software Requirements Specification (SRS); defined system requirements.

Timeline:

Week 2

iii. Proposal

Objective:

To consolidate research and requirements into a structured plan.

Tasks:

Write abstract; write background; conduct literature review; define aims; define methodology; prepare project plan.

Deliverables:

Completed and approved project proposal.

Timeline:

Week 3

iv. System Architecture and Design

Objective:

To design the system structure and workflows.

Tasks:

Create use case diagrams; create ERD; create DFDs; design system architecture; design wireframes; define database schema.

Deliverables:

System design document; architecture diagrams; wireframes.

Timeline:

Week 4

4.3.2 Phase Two: System Development

Sprint 5: Foundation and Authentication

Objective:

To set up the core system and implement user authentication.

Tasks:

Setup Firebase; implement user registration; implement user login; implement profile management; setup app navigation.

Deliverables:

Functional authentication system; users can register and login.

Timeline:

Week 4

Sprint 6: Menu and Cart

Objective:

To implement menu browsing and cart functionality.

Tasks:

Display menu from Firestore; implement category filtering; add items to cart; manage cart items.

Deliverables:

Menu browsing functional; cart system working.

Timeline:

Week 5

Sprint 7: Order Placement

Objective:

To enable users to place and confirm orders.

Tasks:

Implement checkout flow; implement pickup time scheduling; submit orders; display order confirmation.

Deliverables:

Order submission functional.

Timeline:

Week 6

Sprint 8: M-Pesa Integration

Objective:

To implement the payment system.

Tasks:

Setup Daraja API; implement STK Push; handle payment callbacks; handle payment errors.

Deliverables:

Sandbox payments successful.

Timeline:

Week 7

Sprint 9: Loyalty and Tracking

Objective:

To implement advanced features and tracking.

Tasks:

Implement points earning; implement points redemption; implement order tracking; implement push notifications; develop admin dashboard.

Deliverables:

Core system features complete.

Timeline:

Week 8

4.3.3 Phase Three: Testing, Documentation, and Reporting

i. Software Testing and Evaluation

Objective:

To validate system functionality, performance, and security.

Tasks:

Perform functional testing; perform performance testing; conduct security validation; fix system bugs; deploy admin dashboard; conduct user acceptance testing with students and staff.

Deliverables:

Testing report; validated system ready for deployment.

Timeline:

Weeks 9–10

ii. Project Write-up

Objective:

To document all project outcomes.

Tasks:

Prepare final report; document research; document system design; document implementation; document testing results; document challenges; write conclusions.

Deliverables:

Draft of final project report.

Timeline:

Week 10

iii. Final Presentation and Submission

Objective:

To present findings and deliver the final system.

Tasks:

Conduct system demo; prepare presentation slides; finalize report; submit project.

Deliverables:

Final report and presentation slides.

Timeline:

Week 10

4.4 Gantt Chart

A Gantt chart is a visual project management tool used to plan, schedule, and monitor the progress of tasks within a project. The Gantt chart for the USIU Cafeteria Ordering System outlines every stage of the project lifecycle - from research and system design through development and testing to final submission - across a 13-week timeline from January to April 2026.

USIU Cafeteria Ordering System - Project Gantt Chart

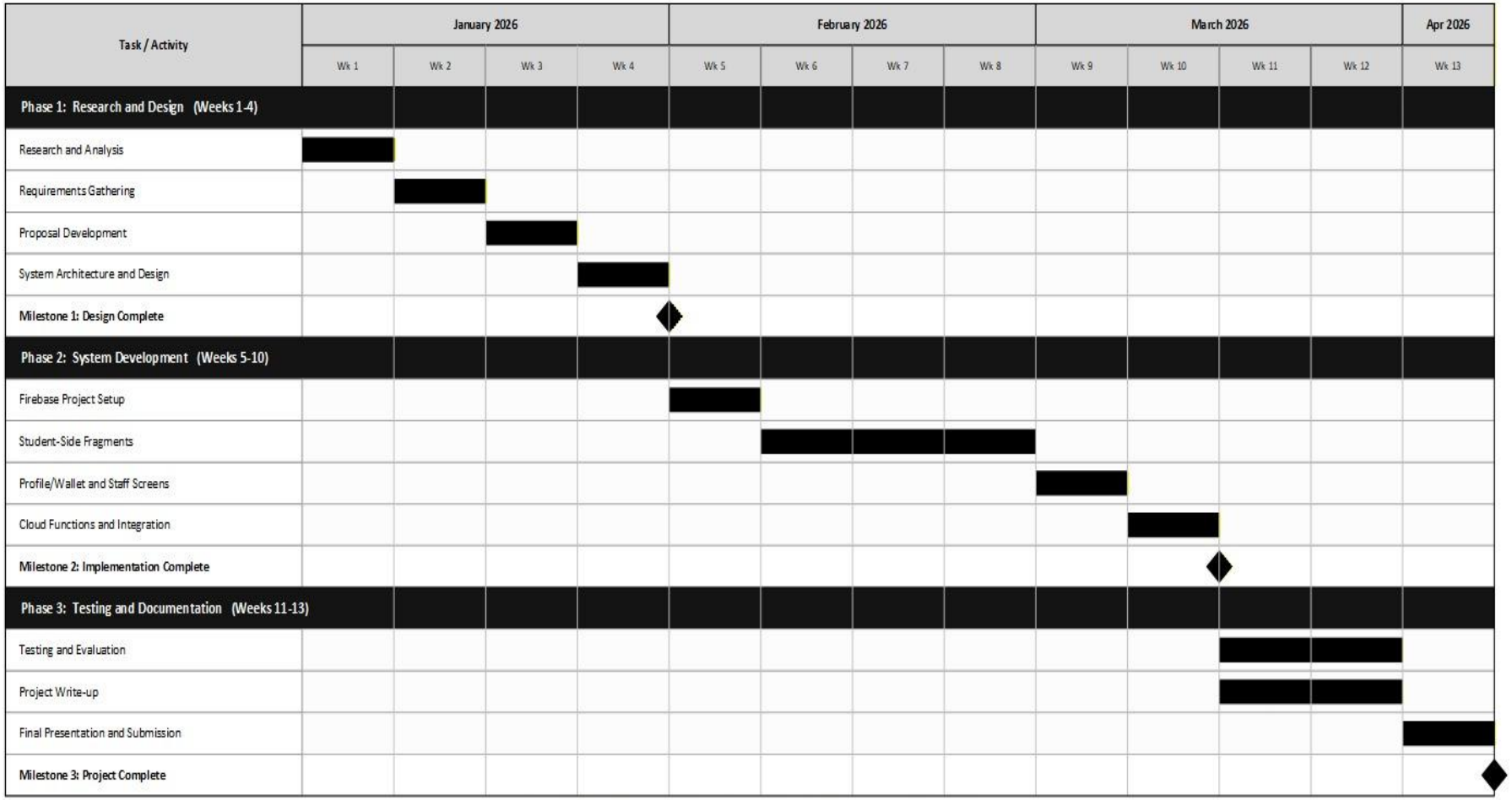


Figure 11: USIU Cafeteria Ordering System — Project Timeline (January – April 2026)

Figure 11 above shows the Gantt chart which also highlights task dependencies and overlaps, providing a visual understanding of how activities are interrelated. This helps with effective project management by identifying critical paths and ensuring that development work does not begin before the design is complete, and that testing does not begin before the full system is integrated.

Project Phases and Tasks

1. Phase One: Research and Requirements Gathering

This initial phase established the foundation of the USIU Cafeteria Ordering System by focusing on research, requirements gathering, proposal development, and system design. Research involved reviewing six existing systems and observing the USIU cafeteria during peak hours. Requirements gathering translated findings into 20 documented test cases and a complete specification of functional and non-functional requirements. Proposal development formalised the project's scope and objectives. System design produced all required diagrams - use case, ERD, DFDs, flowcharts, class diagram, and wireframes - providing a complete blueprint before any code was written. The successful completion of these tasks marked *Milestone 1: Research and System Design Completed*.

2. Phase Two: System Implementation

This phase concentrated on building all components of the cafeteria ordering system. Firebase setup established the backend infrastructure before application development began. Student-side fragments delivered the core user experience across Menu, Cart, Orders, and Pre-orders screens. The Profile/Wallet screen and staff screens completed the role-based access model. Cloud Functions integrated the server-side scheduled logic for pre-order cut-offs and push notifications. End-to-end testing confirmed that the full order lifecycle - from student placement to staff update to student notification - functioned correctly. Completion marked *Milestone 2: System Implementation Completed*.

3. Phase Three: Testing, Documentation, and Presentation

The final phase validated system reliability and prepared deliverables. All 20 test cases were executed and results documented. UAT with students and cafeteria staff provided real-world feedback. Project documentation covered all development stages, challenges encountered, and

how they were resolved. The final presentation demonstrated the system live. This marked *Milestone 3: Testing, Documentation, and Presentation Completed*.

Dependencies

The project followed a logical flow of dependencies. System design could only begin after requirements gathering was finalised. Firebase setup had to be completed before fragment development. Cloud Functions development required both Firestore collections and fragment code to be in place before end-to-end integration could be verified. Testing depended on the full system being deployed. These dependencies ensured a smooth progression between activities and minimised rework.

Milestones

Three milestones structured the project's progress. Milestone 1 marked the completion of research and design, providing a validated blueprint. Milestone 2 represented the completion of all system implementation, with every feature functional and integrated. Milestone 3 confirmed the system was fully tested, documented, and ready for submission.

4.5 Requirements

4.5.1 Hardware Requirements

The USIU Cafeteria Ordering System will require the following hardware for development and testing.

Development Machine

- *Processor:* Intel Core i5 (8th Gen or higher) or equivalent
- *RAM:* 8 GB minimum (16 GB recommended for running Android Emulator alongside Android Studio)
- *Storage:* 256 GB SSD or higher
- *Operating System:* Windows 10/11, macOS, or Linux

End-User Devices

- *Android Smartphone:* Minimum Android 7.0 (API level 24) for students and staff. A physical device is preferred for testing GPS, FCM push notifications, and real-world performance.
- *Internet Access:* Required for Firestore real-time sync, Firebase Authentication, and Cloud Messaging. The application does not support offline mode.

4.5.2 Software Requirements

The following software tools and platforms will be used in the development of the USIU Cafeteria Ordering System:

- IDE**:** Android Studio (latest stable release) - primary development environment for Java + XML Android development; VS Code for Next.js admin panel and Supabase Edge Function development
- Languages**:** Java (Android application); TypeScript (Next.js admin panel and Supabase Edge Functions)
- UI Frameworks: Material Design 3 components via AndroidX libraries (Android app); Tailwind CSS (admin panel)
- Web Framework: Next.js 14 (App Router) - web-based admin panel
- Version Control: Git & GitHub for source code management
- Firebase Services: Firestore (real-time NoSQL database), Firebase Authentication (user management for three roles: student, staff, admin), Firebase Cloud Messaging (push notifications); Firebase Spark free plan - no Cloud Functions required
- Edge Functions: Supabase CLI - deploys TypeScript/Deno edge functions (process-cutoff, notify-order-ready) on the Supabase free plan
- Scheduled Jobs: cron-job.org - free HTTP cron service that triggers the Supabase Edge Functions at 10:00 AM EAT (lunch cut-off) and 5:00 PM EAT (dinner cut-off)
- Hosting: Vercel - free hosting for the Next.js admin panel with CI/CD from GitHub

- Design Tools: Figma or draw.io for wireframes and system diagrams
- Testing: Android Emulator (functional testing), physical Android device (performance and FCM testing), browser (admin panel testing)
- Project Management: GitHub Projects or similar for task tracking

4.5.3 Server Requirements

The USIU Cafeteria Ordering System uses a combination of managed cloud services for its server infrastructure, eliminating the need for a self-managed web or database server. All services operate on free tiers - no credit card or billing account is required.

- Database: Firebase Firestore - fully managed NoSQL cloud database with real-time sync and offline SDKs. No server configuration required.
- Authentication: Firebase Authentication - managed identity service supporting email/password sign-in for three roles: student, staff, and admin.
- Push Notifications: Firebase Cloud Messaging - managed notification delivery service for order-ready and pre-order confirmation/cancellation alerts.
- Firebase Plan: Spark (free) plan. Cloud Functions are not used; all server-side scheduling logic runs on Supabase Edge Functions instead, which are available on the Supabase free tier without a billing account.
- Edge Functions: Supabase - free plan provides serverless TypeScript/Deno edge functions. Two functions are deployed: process-cutoff (pre-order cut-off deduction and cancellation logic, called by cron-job.org) and notify-order-ready (FCM notification dispatch, called directly by the Android app after a staff status update).
- Scheduled Jobs: cron-job.org - free HTTP cron service. Two jobs are configured to call the process-cutoff edge function: 10:00 AM EAT (lunch cut-off) and 5:00 PM EAT (dinner cut-off).
- Admin Panel Hosting**:** Vercel - free hosting for the Next.js admin web panel, with automatic deployment on every GitHub push to the main branch.

- Scalability**:** Firestore scales automatically with demand. Supabase Edge Functions and Vercel hosting scale within their respective free tier limits, which comfortably cover prototype-scale usage at USIU-Africa.

4.5.4 Budget Requirements

The budget for the USIU Cafeteria Ordering System is minimal. All primary development tools are free, and the entire cloud infrastructure operates on free tiers with no credit card or billing account required.

- Firebase Spark Plan: Free - used for Firestore, Authentication, and Cloud Messaging. No Cloud Functions needed, so no Blaze plan upgrade is required.
- Supabase Free Plan: Free - hosts the process-cutoff and notify-order-ready Edge Functions with no usage charges at prototype scale.
- cron-job.org: Free - provides two scheduled HTTP cron jobs for the pre-order cut-off triggers.
- Vercel Free Plan: Free - hosts and continuously deploys the Next.js admin panel from GitHub.
- Android Studio: Free (no license cost).
- VS Code: Free (no license cost).
- GitHub: Free for public and private repositories at student scale.
- Design Tools: Figma free tier sufficient for wireframes and diagrams.
- Physical Android Device: Required for FCM and performance testing - an existing personal device is sufficient; no additional purchase needed.

Total estimated cost: KES 0 for a fully functional prototype comprising the Android mobile application, the Next.js web admin panel, and all supporting cloud infrastructure.

4.6 Conclusion

This chapter has presented the methodology to be used in the design and development of the USIU Cafeteria Ordering System, a native Android application designed to eliminate physical queuing, enable advance meal scheduling, and introduce cashless wallet-based payment for USIU-Africa students. The work has been structured into three phases - Research and Design, System Development, and Testing and Reporting - each with well-defined objectives, tasks, and deliverables. The program of work and corresponding Gantt chart illustrate the 13-week timeline, task dependencies, and project milestones, ensuring that the project progresses in a systematic and manageable way.

The requirements and design artifacts produced in Phase 1 will provide a complete blueprint before any code is written, reducing rework and ensuring the implementation remains aligned with the identified problems. Phase 2 builds the system incrementally - Firebase infrastructure first, then student screens, then staff screens, then Cloud Functions - so that each layer is validated before the next depends on it. Phase 3 validates the complete system against all 20 test cases and real user feedback before final submission. The following chapter translates the design artifacts described here into a detailed system analysis, presenting the architecture, diagrams, data model, and requirements that will guide the implementation.

5 CHAPTER 5: SYSTEM ANALYSIS AND DESIGN

This chapter presents the system analysis and design of the proposed USIU Cafeteria Ordering System. It outlines the structural and functional models that define how the system operates, interacts with users, and achieves its intended objectives. The chapter introduces a series of diagrams - system architecture, use case, entity-relationship, data flow, flowcharts, activity, sequence, and class diagrams - which collectively illustrate the internal workings of the system. Wireframes are provided to showcase the user interface design across all five main screens. By integrating both conceptual and visual representations, this chapter establishes a clear foundation for the implementation phase, ensuring that the ordering, wallet, and pre-order features are both technically sound and user-friendly.

5.1 System Models

This section presents the system models developed for the USIU Cafeteria Ordering System. Each model provides a different perspective on how the system is structured, how data flows through it, and how users interact with it. Together the models form a comprehensive blueprint that guides the implementation phase and ensures every feature - ordering, wallet management, pre-order scheduling, and staff operations - is logically consistent and technically feasible.

5.1.1 System Architecture Diagram

A system architecture diagram is a visual representation that shows how the different components of a system interact and work together. It illustrates the structure, data flow, and relationships between the user interface, application logic, and data storage layers. For the USIU Cafeteria Ordering System, the diagram outlines how students, staff, and administrators - across two client applications - connect to Firebase backend services and Supabase Edge Functions to deliver real-time ordering, wallet management, and pre-order scheduling.

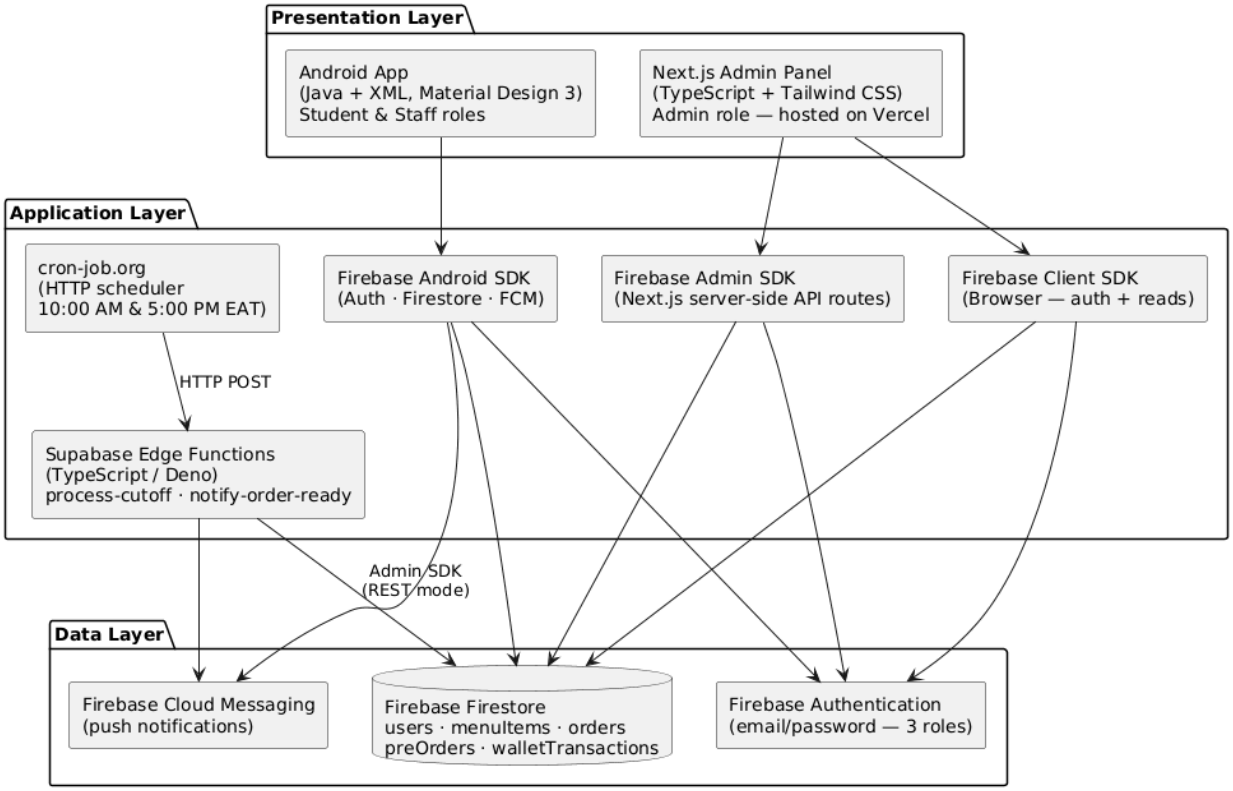


Figure 12: System Architecture Diagram

Figure 12 above shows that the architecture of the USIU Cafeteria Ordering System follows a three-tier design consisting of the Presentation Layer, Application Layer, and Data Layer.

1. Presentation Layer

This layer represents the user interface and the primary interaction point of the system. It consists of two separate client applications serving three user roles: Student, Staff (Cashier), and Admin.

The *native Android application* (Java + XML, Material Design 3) serves both Students and Staff. Students access five main screens: Menu (browse and filter by category), Cart (build order, select wallet or cash payment), Orders (track active order, view history, see estimated wait time), Pre-orders (schedule and manage advance meals), and Profile/Wallet (balance, transaction history, account details). Staff access three operational screens: Staff Orders (view and update all incoming orders), Staff Menu (toggle item availability), and Staff Wallet (top up or deduct student wallet balances and record each transaction with the staff member's ID). Role-based navigation is determined at login by reading the role field from the user's Firestore document. Staff accounts

cannot self-register - they are created by an administrator and must change their temporary password on first login.

The *Next.js web admin panel* (TypeScript, Tailwind CSS, hosted on Vercel) serves Administrators only. It provides full menu management (add, edit, delete items; toggle availability; set image URLs), staff account creation (with a temporary password and an enforced first-login password change on the Android app), and a user directory with separate views for students and staff/admin accounts.

2. Application Layer

The Application Layer sits between the client applications and the Firebase backend. For the Android app, it is implemented through the Firebase Android SDK, which manages authentication sessions, Firestore real-time listeners, and FCM token registration on the device. For the admin panel, authentication and real-time reads use the Firebase Client SDK in the browser, while privileged operations such as creating staff accounts use the Firebase Admin SDK via Next.js server-side API routes.

On the server side, two *Supabase Edge Functions* (TypeScript/Deno runtime, deployed on the Supabase free plan) replace Firebase Cloud Functions for scheduled and event-driven logic. The process-cutoff function is invoked at 10:00 AM EAT (lunch) and 5:00 PM EAT (dinner) daily by cron-job.org. It queries all preOrders documents due that day, then for each: deducts the wallet balance via a Firestore transaction and marks the pre-order confirmed, or cancels it and sends an FCM notification if funds are insufficient. The notify-order-ready function is called directly by the Android app via an authenticated HTTP POST after a staff member marks an order as Ready, dispatching an FCM push notification to the student. All Firestore writes involving wallet deductions use runTransaction() to guarantee atomicity.

3. Data Layer

The Data Layer is managed by Firebase and consists of:

- *Firebase Firestore:* The primary cloud NoSQL database storing five collections - users, menuItems, orders, preOrders, and walletTransactions - with orderItems embedded

as arrays within each order and pre-order document. Real-time snapshot listeners push changes to the Android client without polling.

- *Firebase Authentication:* Manages identity for all three roles (student, staff, admin) through email/password sign-in. Each user has a corresponding document in the users collection with a role field.
- *Firebase Cloud Messaging:* Delivers push notifications to registered FCM tokens stored in the fcmToken field of each user's document.

Supabase Edge Functions connect to Firebase using the Firebase Admin SDK configured for REST transport (gRPC is not supported in the Deno edge runtime). cron-job.org provides external scheduling for the pre-order cut-off triggers, and Vercel provides continuous deployment for the admin panel - both at zero cost.

5.1.2 Use Case Diagram

A Use Case Diagram is used to model the interactions between users (actors) and the system, illustrating the different services the application provides. For the USIU Cafeteria Ordering System, the diagram identifies three human actors and one system actor, and the use cases each is associated with.

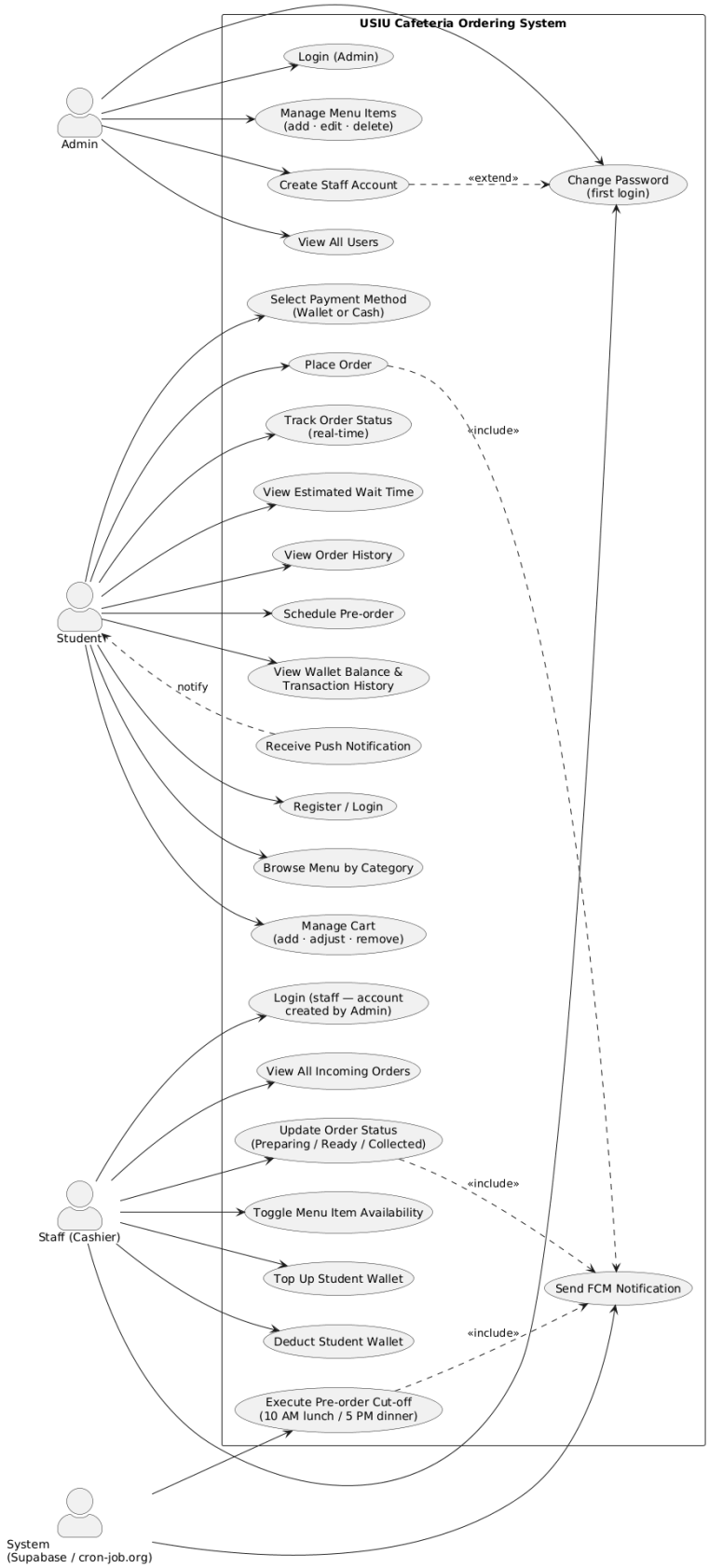


Figure 13: Use Case Diagram

Figure 13 above shows the use case diagram illustrating the key interactions between actors and the USIU Cafeteria Ordering System.

Student is the primary actor, interacting via the Android app. Use cases include: Register/Login, Browse Menu by Category, Add Item to Cart, Adjust Item Quantity, Select Payment Method (Wallet or Cash), Place Order, Track Order Status (real-time), View Estimated Wait Time, Receive Push Notification (Order Ready / Pre-order Confirmed / Insufficient Funds), View Order History, Schedule Pre-order, Set Recurring Pre-order, Cancel Pre-order (before cut-off), View Wallet Balance, and View Wallet Transaction History.

Staff (Cashier) is the second human actor, interacting via the Android app. Use cases include: Login (account created by Admin - staff cannot self-register), Change Password on First Login, View All Incoming Orders (real-time), Mark Order as Preparing, Mark Order as Ready, Mark Order as Collected, Toggle Menu Item Availability, Top Up Student Wallet (after receiving cash at counter), and Deduct Student Wallet Balance (manual charge adjustment, recorded with staff ID for audit).

Admin is the third human actor, interacting exclusively via the Next.js web admin panel. Use cases include: Login, Change Password on First Login, Create Staff Account (generates temporary password; forces first-login change on Android), Add Menu Item, Edit Menu Item, Delete Menu Item, Toggle Menu Item Availability, and View All Users (students and staff/admin separately).

System (Automated Functions) represents the Supabase Edge Functions and cron-job.org scheduler. Automated use cases include: Execute Lunch Pre-order Cut-off (10:00 AM EAT daily - triggered by cron-job.org POST to process-cutoff edge function), Execute Dinner Pre-order Cut-off (5:00 PM EAT - same logic), Send FCM Notification on Order Ready (Android calls notify-order-ready edge function after staff status update), Send FCM Notification on Pre-order Confirmed, and Send FCM Notification on Pre-order Cancelled due to Insufficient Funds.

The diagram uses «include» relationships for shared processes such as authentication (required by all login-gated use cases) and Firestore read/write (required by all data operations). «extend» relationships cover optional or conditional processes such as recurring pre-order chain creation, cancellation notifications, and forced first-login password change.

5.1.3 Entity Relationship Diagram (ERD)

Introduction

An ERD illustrates the logical structure of the database by showing entities, their attributes, and the relationships between them. For the USIU Cafeteria Ordering System, the ERD maps the five Firestore collections that store all system data - users, menuItems, orders, preOrders, and walletTransactions (with orderItems embedded as arrays) - and defines how they relate to one another.

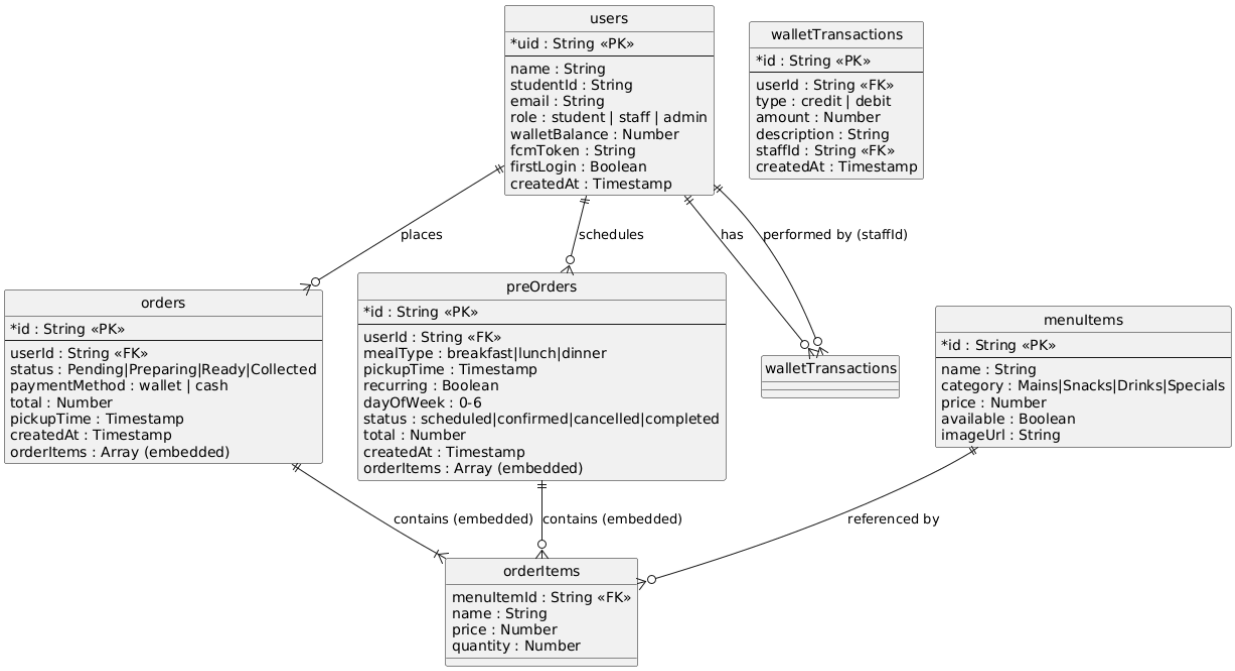


Figure 14: Entity Relationship Diagram (ERD)

Figure 14 above shows the Entity-Relationship Diagram for the USIU Cafeteria Ordering System. The five key entities are as follows:

users - Represents all registered users of the system. Attributes: uid (PK), name, studentId (students only; absent for staff and admin), email, role (student | staff | admin), walletBalance (students only; absent for staff and admin), fcmToken (renamed from deviceToken in the original plan), firstLogin (boolean; staff and admin only - set to true on account creation, cleared to false after the first-login password change), createdAt. A student user can place many orders, many pre-orders, and have many wallet transactions.

menuItems - Represents each item available in the cafeteria menu. Attributes: id (PK), name, category (Mains | Snacks | Drinks | Specials), price, available (boolean), imageUrl. A menu item can appear in many order items and many pre-order items.

orders - Represents a single order placed by a student. Attributes: id (PK), userId (FK → users), status (Pending | Preparing | Ready | Collected), paymentMethod (wallet | cash), total, pickupTime, createdAt. Each order belongs to one user and contains one or more embedded orderItems.

orderItems - Embedded within each order document as an array. Attributes: menuItemId (FK → menuItems), name, price, quantity. This denormalised structure avoids a separate collection lookup on every order read.

preOrders - Represents an advance meal scheduled by a student. Attributes: id (PK), userId (FK → users), mealType (breakfast | lunch | dinner), pickupTime, recurring (boolean), dayOfWeek (0–6, used if recurring), status (scheduled | confirmed | cancelled | completed), total, createdAt. Items are embedded as an array identical to orderItems.

walletTransactions - Represents each credit or debit event on a student's wallet. Attributes: id (PK), userId (FK → users - the student whose wallet was changed), type (credit | debit), amount, description, staffId (FK → users - the UID of the staff member who performed the transaction, for audit purposes), createdAt. Provides a complete, accountable audit trail for every wallet balance change, linking each transaction to the responsible staff member.

The ERD illustrates that all financial and ordering activity is anchored to the users entity through foreign key relationships. Firestore security rules use three helper functions - isStudent(), isStaff(), and isAdmin() - to gate access per collection, with an isPrivileged() helper combining staff and admin permissions for shared operations such as order status updates and menu management. This ensures each student sees only their own data while staff and admin can access the records necessary for their respective roles.

5.1.4 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) is a visual representation of how data moves within a system. It focuses on the flow of information between processes, data stores, and external entities, showing how inputs are transformed into outputs. The DFD for the USIU Cafeteria Ordering System is developed across three levels, from a high-level context view down to detailed sub-process breakdowns.

i. Level 0: Context Diagram

The Level 0 DFD represents the entire system as a single process and shows how it interacts with external entities at the highest level of abstraction.

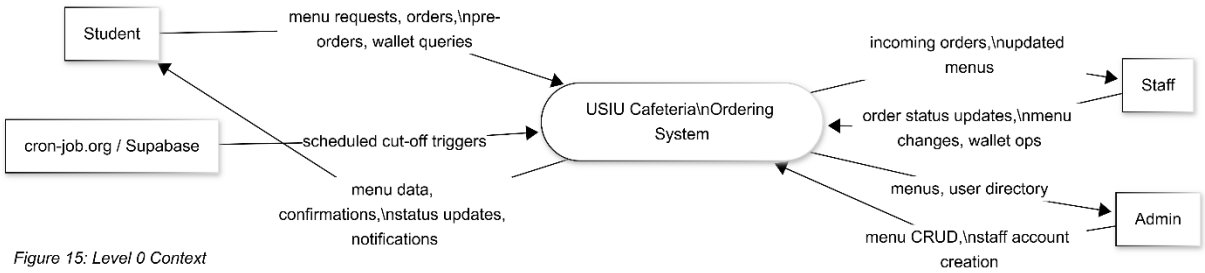


Figure 15: Level 0 Context Diagram

Figure 15 above shows the context diagram. The three external human entities are Student, Staff, and Admin. Students provide inputs - menu browsing requests, order data (items, quantity, payment method), pre-order schedules, and wallet queries. The system returns menu data, order confirmations, real-time status updates, estimated wait times, push notifications, and wallet balance information. Staff provide inputs - order status updates (Preparing/Ready/Collected), menu availability changes, and wallet top-up or deduction actions. The system returns incoming order lists and updated menu states to staff. Admin provides inputs via the web panel - menu item CRUD and staff account creation. The system returns updated menu lists and user directories to admin. The Supabase Edge Function (triggered by cron-job.org) interacts with the system as an automated entity at 10:00 AM and 5:00 PM EAT, providing scheduled cut-off processing that drives pre-order wallet deductions and cancellations.

ii. Level 1: Detailed DFD

The Level 1 DFD decomposes the single context process into the main functional sub-processes of the system.

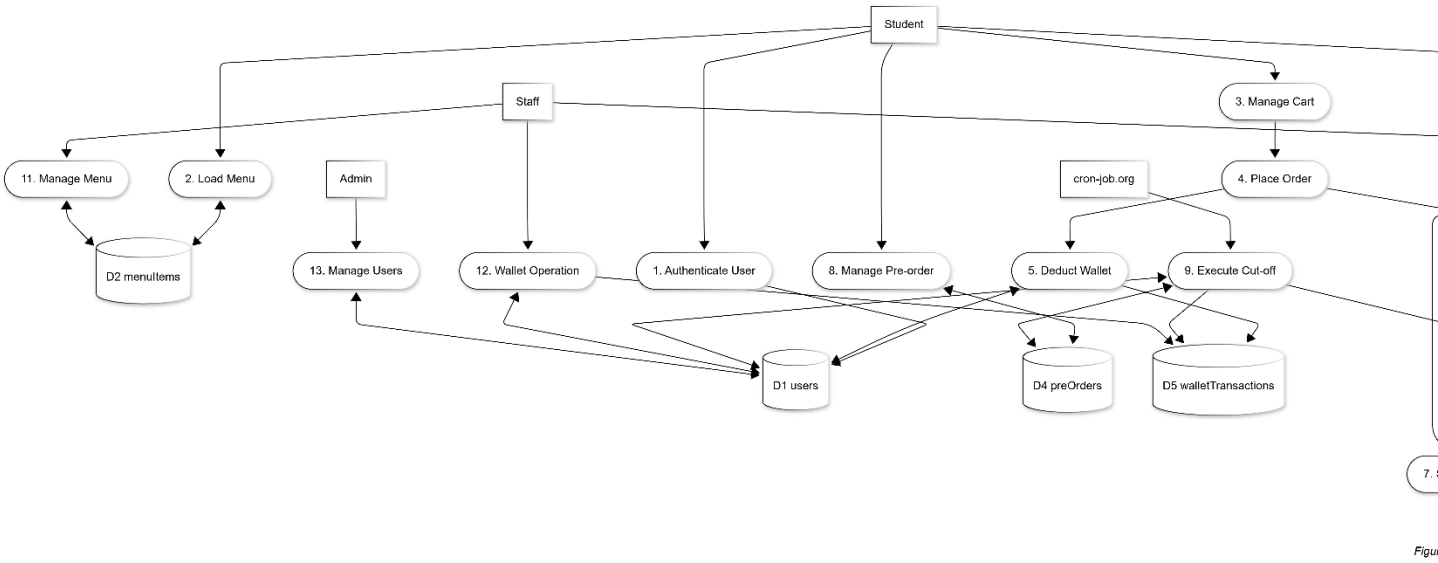


Figure 16 above shows the Level 1 DFD. The main processes are:

1. *Authenticate User* - Receives login credentials from Student/Staff, validates against Firebase Auth, returns session token.
2. *Load Menu* - Reads menuItems collection from Firestore; streams updates to student via snapshot listener.

3. *Manage Cart* - Receives add/remove/update actions from Student; maintains cart state in CartViewModel (device memory, no Firestore write until order placed).

4. *Place Order* - Receives confirmed cart and payment method from Student; writes order document to Firestore; if wallet payment, calls Deduct Wallet process atomically.

5. *Deduct Wallet* - Executes Firestore runTransaction() to deduct total from users.walletBalance and write a walletTransactions record simultaneously.

6. *Track Order* - Listens to orders collection for status changes; calculates estimated wait from count of Pending/Preparing orders ahead; streams updates to Student.

7. *Send Notification* - After the staff Android app writes status = Ready to Firestore, it sends an authenticated HTTP POST to the Supabase notify-order-ready Edge Function (fail-silent). The edge function reads the student's fcmToken from the users collection and dispatches an FCM push notification. This replaces the Firestore-trigger Cloud Function approach, which required the Firebase Blaze plan.

8. *Manage Pre-order* - Receives pre-order details from Student; writes preOrders document with status=scheduled; reads upcoming pre-orders list back to Student.

9. *Execute Cut-off* - Supabase Edge Function (process-cutoff) called by cron-job.org HTTP POST at 10:00 AM EAT (lunch) or 5:00 PM EAT (dinner): queries preOrders (mealSlot, status=scheduled, scheduledDate=today); for each match, calls Deduct Wallet via Firestore transaction; on insufficient funds, marks cancelled and sends FCM notification via Firebase Admin SDK (with preferRest: true - gRPC not supported in Deno runtime).

10. *Staff: Update Order Status* - Receives status change from Staff Android app; writes to orders collection; then app calls Send Notification (process 7).

11. *Staff: Manage Menu* - Receives availability toggle or CRUD action from Staff (Android) or Admin (web panel); updates menuItems collection; propagates to all student Menu screens via snapshot listener.

12. *Staff: Wallet Operation* - Receives student UID, amount, and operation type (credit or debit) from Staff; executes Firestore runTransaction() to update users.walletBalance; writes walletTransactions record including staffId for audit trail.

13. *Admin: Manage Users* - Admin creates staff account via web panel (Next.js API route → Firebase Admin SDK): creates Firebase Auth user, writes users document with role=staff and firstLogin=true. Admin views user directory from Firestore users collection.

iii. Level 2: Detailed DFD

The Level 2 DFD expands the two most complex processes - Place Order and Execute Cut-off - into their internal sub-steps.

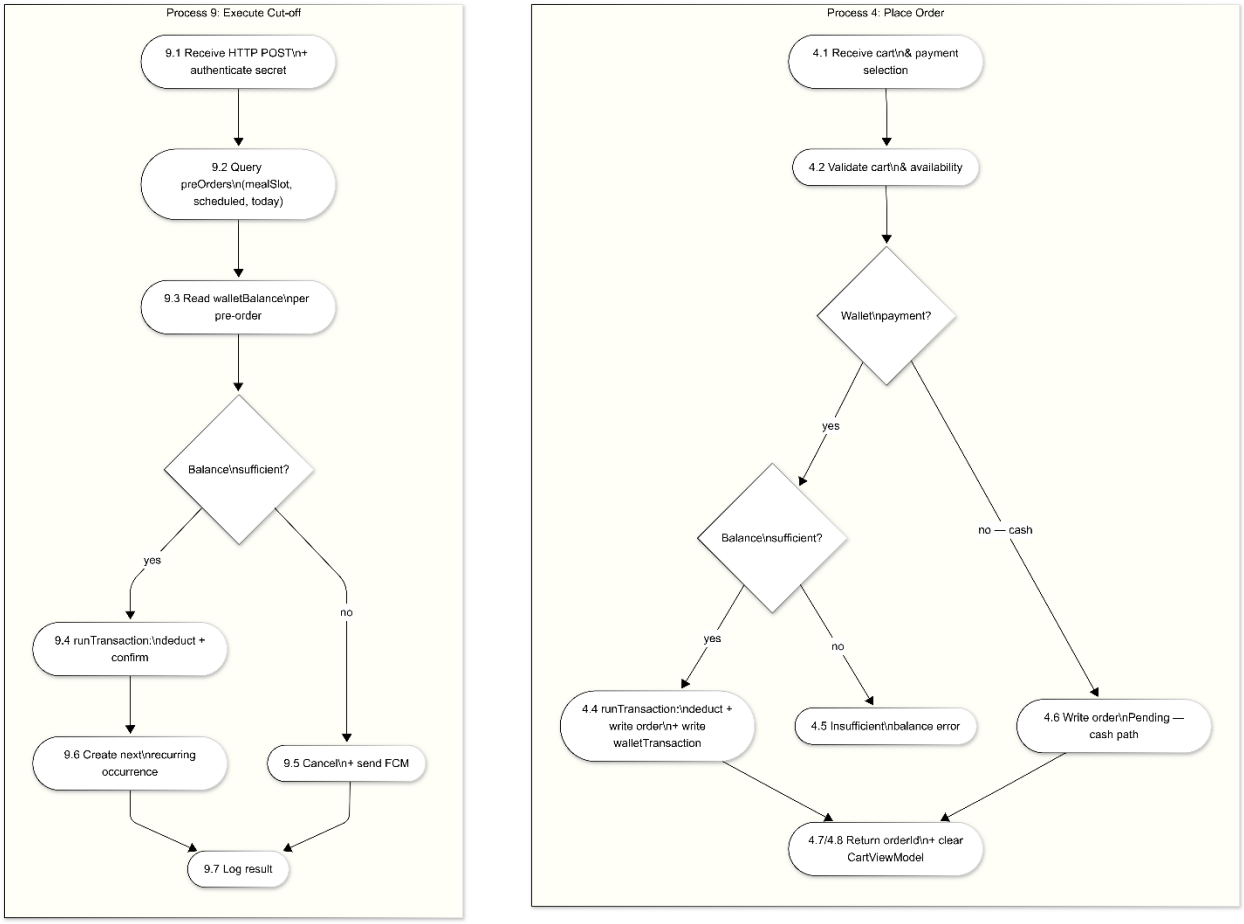


Figure 17: Level 2 DFD

Figure 17 above shows the Level 2 DFD. For *Place Order*: (1) Receive cart and payment selection from CartFragment; (2) Validate cart is non-empty and all items still available; (3) If payment = wallet, check walletBalance >= total; (4) If sufficient, call runTransaction() - deduct balance, write walletTransactions record, write order document with status=Pending; (5) If insufficient, return error to CartFragment without writing order; (6) If payment = cash, write order document directly with status=Pending; (7) Return order ID and confirmation to CartFragment; (8) Clear CartViewModel.

For *Execute Pre-order Cut-off*: (1) cron-job.org HTTP POST fires at the scheduled time (10:00 AM or 5:00 PM EAT) with mealSlot in the request body; (2) Supabase process-cutoff Edge Function authenticates the request using FUNCTIONS_SECRET header; (3) Firebase Admin SDK (REST mode - gRPC unsupported in Deno) queries preOrders where mealSlot=target, status=scheduled, scheduledDate=today; (4) For each pre-order: read users.walletBalance inside a Firestore transaction; (5) If balance >= total: runTransaction() - deduct balance from walletBalance, write walletTransactions document (with staffId omitted for system transactions), set preOrder.status=confirmed, send FCM confirmation via users.fcmToken; (6) If balance < total: set preOrder.status=cancelled, send FCM insufficient-funds notification via users.fcmToken; (7) If pre-order is recurring: create next occurrence document in preOrders for the next matching weekday; (8) Log execution result.

This Level 2 breakdown reveals the exact data stores touched in each critical operation and confirms that wallet atomicity is enforced at every deduction point through Firestore transactions.

5.1.5 Flowchart

A flowchart is a visual representation that outlines the step-by-step flow of processes within a system. It uses standardised symbols - ovals for start/end, rectangles for processes, diamonds for decisions, and arrows for flow direction. The USIU Cafeteria Ordering System has two distinct operational flows that are each represented as a separate flowchart: the regular order flow and the pre-order cut-off flow.

Flowchart A: Regular Order Flow

Figure 18a: Regular Order Flow

Figure 18a shows Flowchart A. The process begins when the Student launches the app and logs in. The system verifies credentials via Firebase Auth. If login fails, the user is prompted to retry. On success, the student is directed to the Menu screen where menu items are loaded from Firestore. The student browses the menu, selects items, and adds them to the cart. At checkout, the student

selects payment method - Wallet or Cash. If Wallet is selected, the system checks whether walletBalance >= order total. If insufficient, an error is shown and the student remains on the Cart screen. If sufficient (or Cash is selected), the order is placed - a Firestore transaction deducts the wallet balance (if applicable) and writes the order document with status=Pending. The student is navigated to the Orders screen where the live status tracker shows Pending → Preparing → Ready in real time. When the staff marks the order Ready, an FCM notification is sent to the student's device. The student collects the order at the counter.

Flowchart B: Pre-order Cut-off Flow

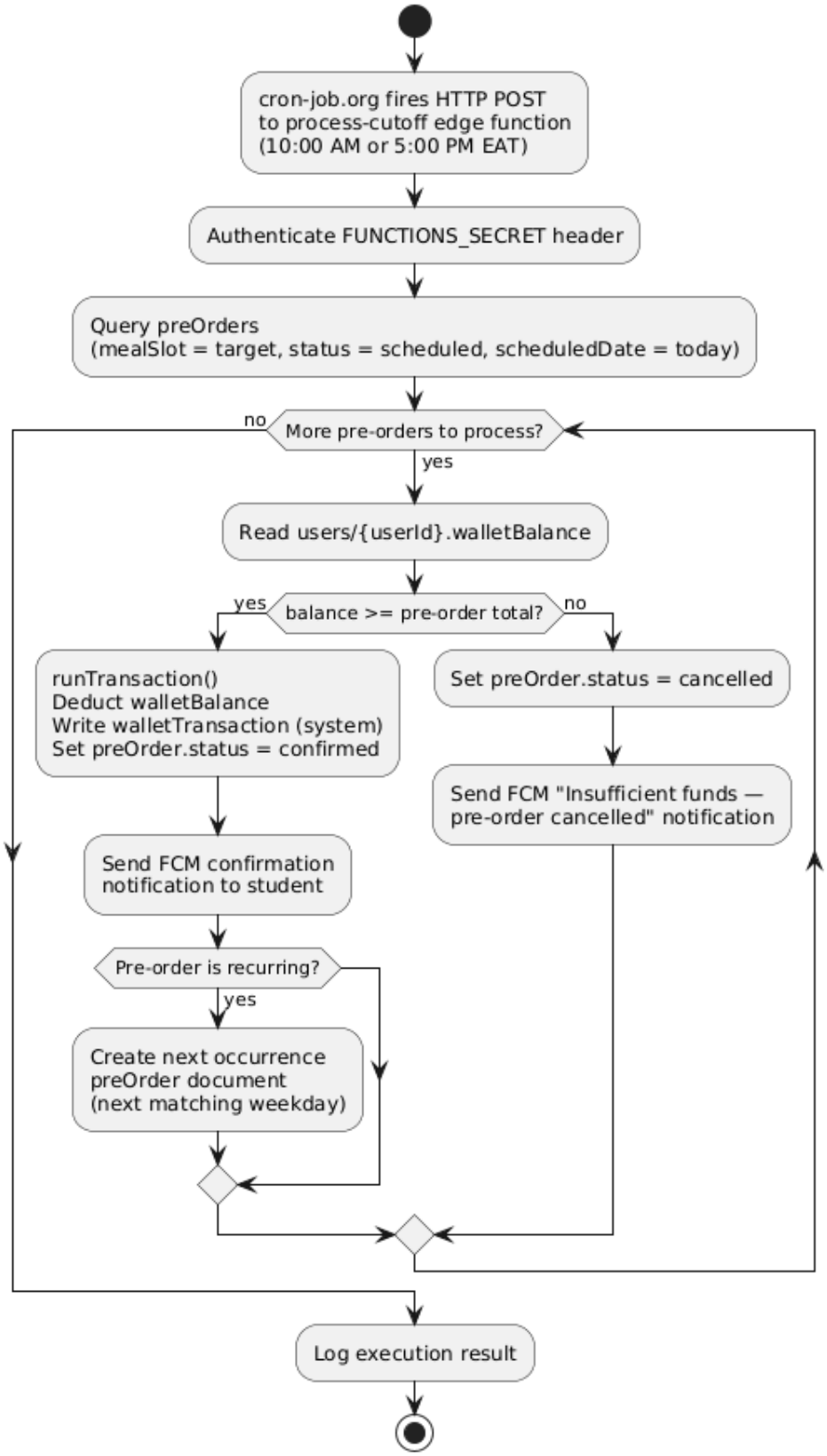


Figure 18b: Pre-order Cut-off Flow

Figure 18b shows Flowchart B. The process begins when the Firebase Cloud Function cron fires at the scheduled cut-off time (10:00 AM for lunch, 5:00 PM for dinner). The function queries all pre-orders with matching mealType, status=scheduled, and today's pickup date. For each pre-order found: the system reads the student's current walletBalance. If balance >= pre-order total, the system executes a Firestore transaction - deducts the balance, writes a walletTransactions record, and updates the pre-order status to confirmed. The cafeteria prepares the meal for the scheduled pickup time. If balance < pre-order total, the pre-order status is updated to cancelled and an FCM insufficient-funds notification is sent to the student's device. The student receives the notification and can top up their wallet and reschedule if they choose. The function logs its execution result and terminates until the next scheduled trigger.

5.1.6 Activity Diagram

An activity diagram is a type of behavioural diagram used in system modelling to represent the flow of activities and actions within a process. It focuses on how different tasks are carried out in sequence, the decisions made along the way, and how various activities connect to achieve a specific goal. For the USIU Cafeteria Ordering System, the activity diagram visualises the complete workflow across both student and staff roles, from login through to order collection, and separately for the pre-order scheduling lifecycle.

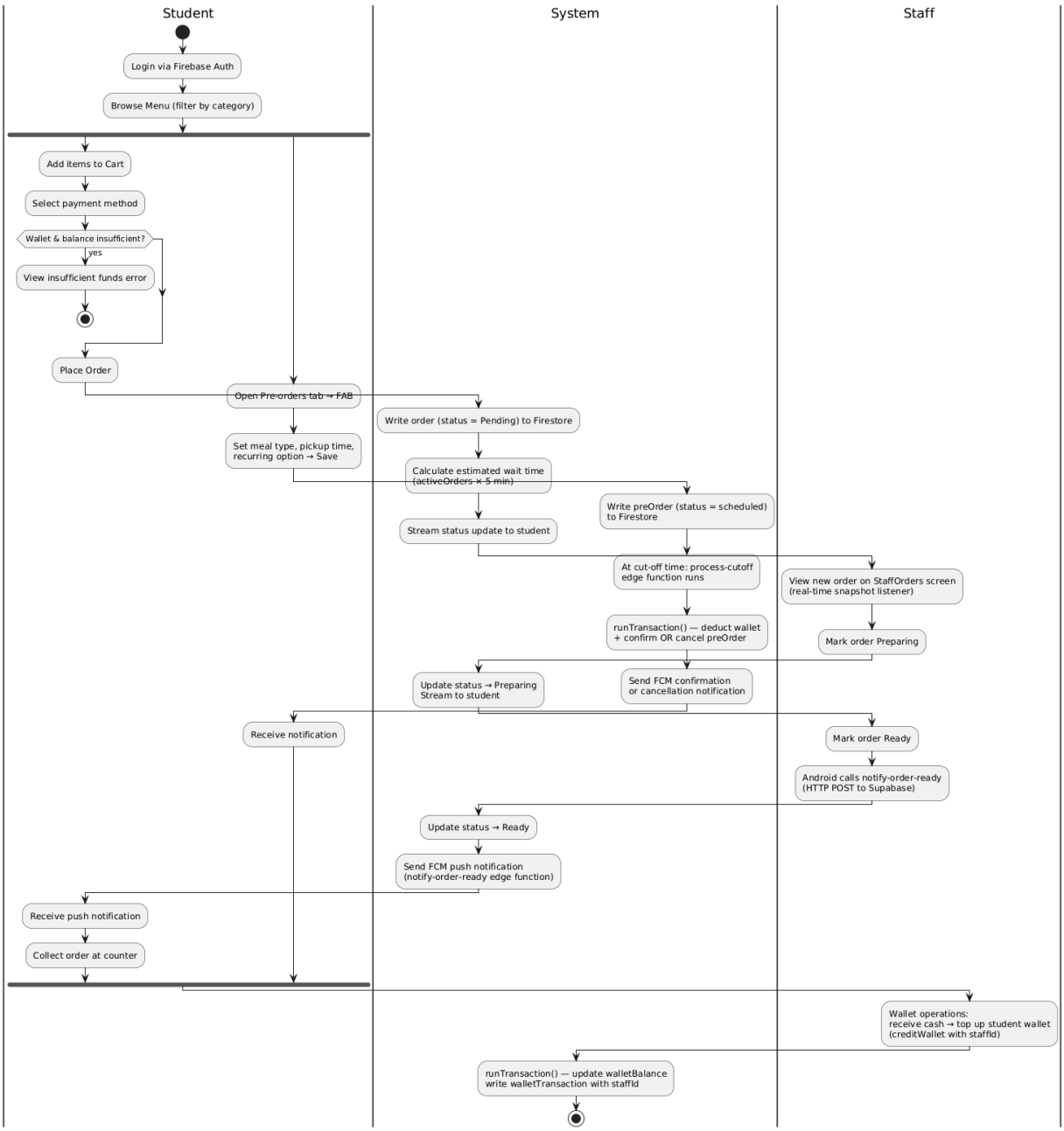


Figure 19: Activity Diagram — Order & Pre-order Lifecycle

Figure 19 above shows the activity diagram for the USIU Cafeteria Ordering System using swimlanes to separate the responsibilities of the Student, Staff, and System (Cloud Functions).

In the *Student swimlane*, activities begin with Login. On successful authentication, the student browses the menu, filtering by category if desired. The student selects items and adds them to the cart, adjusts quantities, and selects a payment method. If the chosen method is Wallet, a balance

check occurs - if funds are insufficient, the student is returned to the cart. Otherwise, the order is placed and the student waits on the Orders screen, monitoring the live status tracker and the estimated wait time indicator. When the Ready notification arrives, the student collects their meal. For the pre-order path, the student instead navigates to the Pre-orders screen, enters meal details, selects a pickup time, optionally sets a recurring day, and saves the pre-order.

In the *Staff swimlane*, activities begin with Login (staff role). The staff views the incoming orders list in real time. On receiving a new order, the staff marks it as Preparing and begins preparation. When the meal is ready, the staff marks it as Ready. For menu management, staff can toggle item availability at any point. For wallet management, staff receive cash from a student, input the amount and student ID, and submit the top-up.

In the *System swimlane*, activities include: streaming menu updates to all connected student devices when availability changes; updating the order status display on the student's Orders screen when staff writes a status change; calculating estimated wait time on each incoming order update; and executing the cut-off cron job - scanning scheduled pre-orders, deducting wallets, and sending notifications - at the designated times.

The activity diagram demonstrates that the system operates across three concurrent actors, with the System swimlane bridging the Student and Staff swimlanes through real-time data propagation.

5.1.7 Sequence Diagram

A sequence diagram is a type of interaction diagram used in system design to show how objects and components interact with each other over time. It focuses on the order of messages exchanged between different entities to accomplish a specific function. For the USIU Cafeteria Ordering System, two sequence diagrams are presented: one for the regular order flow and one for the pre-order cut-off execution.

Sequence Diagram A: Regular Order Placement

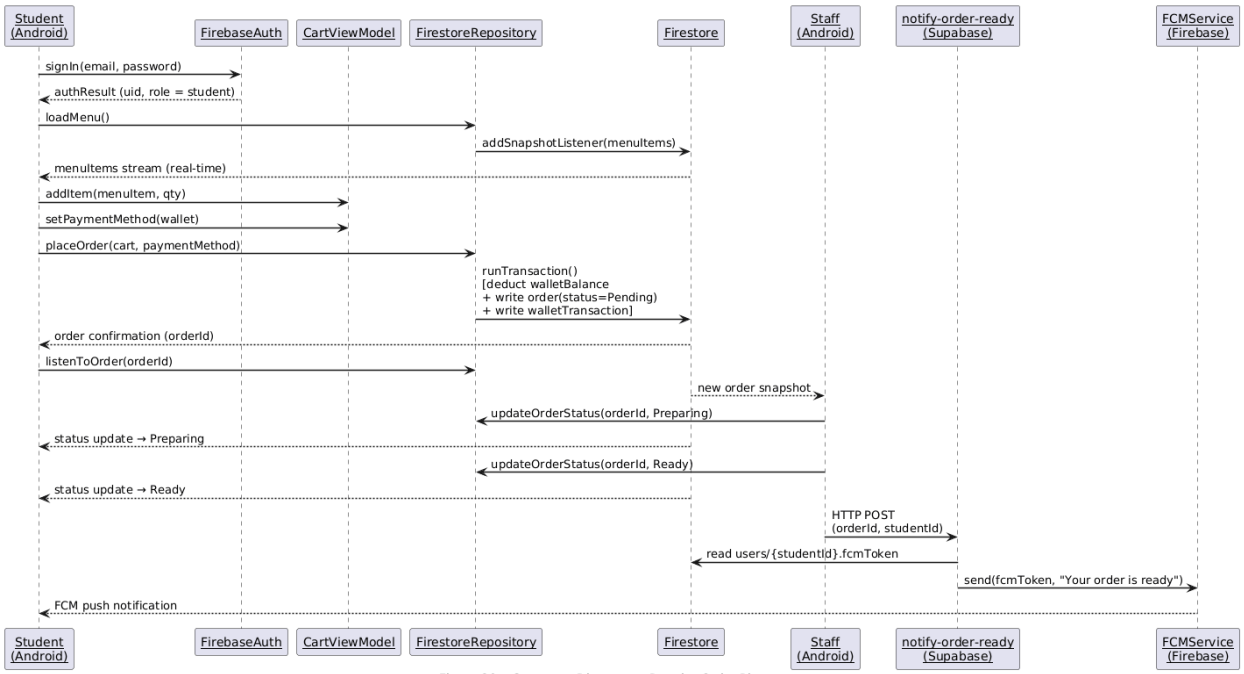


Figure 20a: Sequence Diagram — Regular Order Placement

Figure 20a shows the sequence of interactions for a regular order. The participants are: Student (Android client), FirebaseAuth, CartViewModel, FirestoreRepository, Firestore, StaffOrdersFragment, and FCMService.

1. Student → FirebaseAuth: signIn(email, password)
2. FirebaseAuth → Student: authResult (uid, role)
3. Student → FirestoreRepository: loadMenu()
4. FirestoreRepository → Firestore: addSnapshotListener(menuItems)
5. Firestore → Student: menuItems stream
6. Student → CartViewModel: addItem(menuItem, quantity)
7. Student → CartViewModel: setPaymentMethod(wallet)
8. Student → FirestoreRepository: placeOrder(cart, paymentMethod)
9. FirestoreRepository → Firestore: runTransaction() [deduct walletBalance + write order(status=Pending)]
10. Firestore → Student: order confirmation (orderId)
11. Student → FirestoreRepository: listenToOrder(orderId)

12. Firestore → StaffOrdersFragment: new order notification (snapshot)
13. Staff → FirestoreRepository: updateOrderStatus(orderId, Preparing)
14. Firestore → Student: status update (Preparing)
15. Staff → FirestoreRepository: updateOrderStatus(orderId, Ready)
16. Firestore → FCMService: onOrderStatusChanged trigger
17. FCMService → Student: FCM push notification ("Your order is ready")
18. Firestore → Student: status update (Ready)

Sequence Diagram B: Pre-order Cut-off

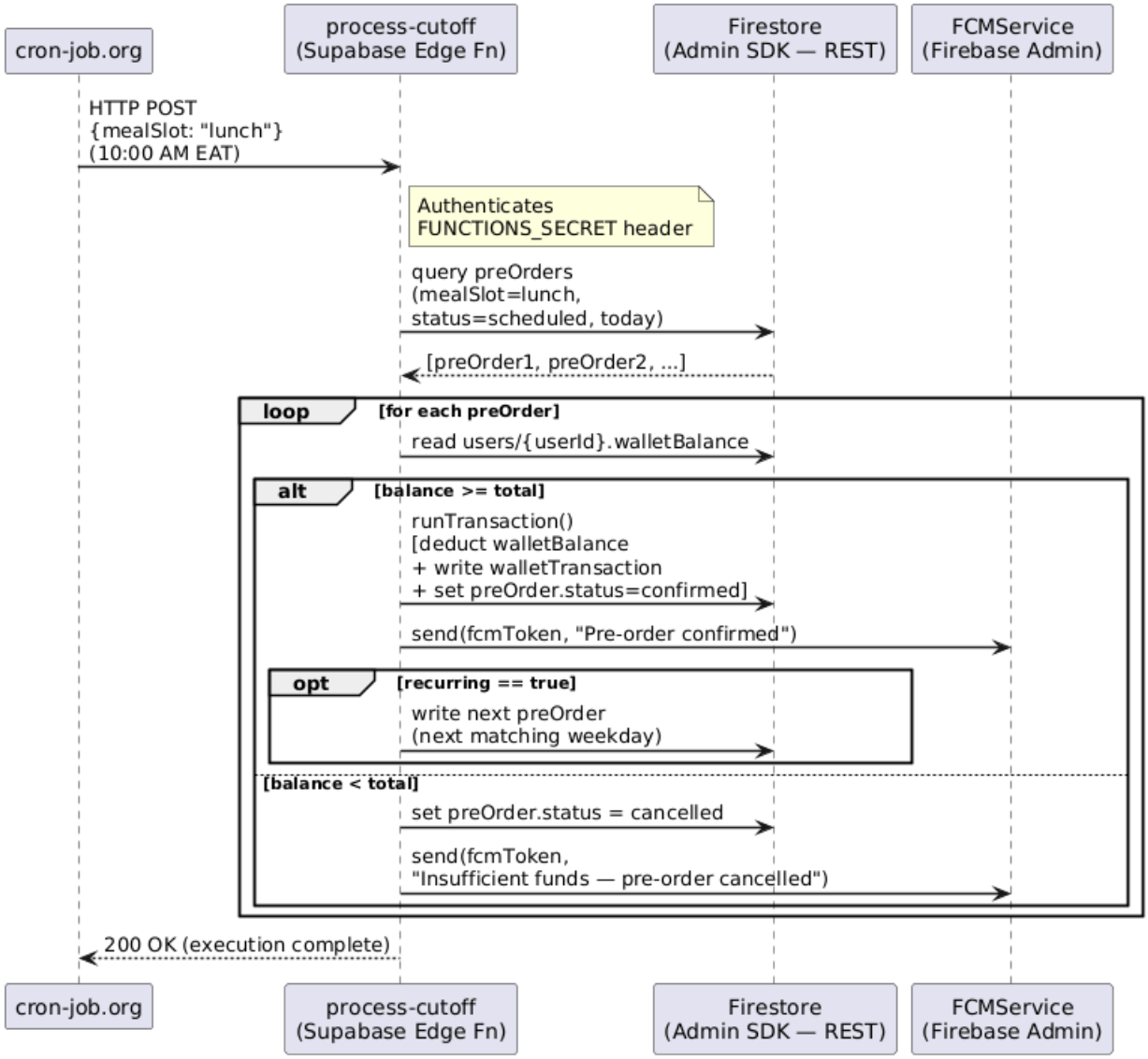


Figure 20b: Sequence Diagram — Pre-order Cut-off Execution

Figure 20b shows the sequence for the pre-order cut-off. The participants are: CloudScheduler, lunchCutoffFunction, Firestore, and FCMService.

1. CloudScheduler → lunchCutoffFunction: trigger (10:00 AM)
2. lunchCutoffFunction → Firestore: query preOrders (mealType=lunch, status=scheduled, today)
3. Firestore → lunchCutoffFunction: [preOrder1, preOrder2, ...]

4. For each preOrder: a. lunchCutoffFunction → Firestore: read users/{userId}.walletBalance b. If balance >= total:

   - lunchCutoffFunction → Firestore: runTransaction() [deduct walletBalance + write walletTransaction + update preOrder.status=confirmed] c. If balance < total:
   - lunchCutoffFunction → Firestore: update preOrder.status=cancelled
   - lunchCutoffFunction → FCMService: sendNotification(deviceToken, "Insufficient funds - pre-order cancelled")

5. lunchCutoffFunction → CloudScheduler: execution complete

5.1.8 Class Diagram

A class diagram is a type of structural diagram in system design that illustrates the blueprint of an object-oriented system by showing the system's classes, their attributes, methods, and the relationships among them. It provides a static view of the system, helping developers understand how components interact and depend on one another. For the USIU Cafeteria Ordering System, the class diagram represents the Android application's Java class structure. The system also includes a separate web admin panel (Next.js, TypeScript) whose components are not represented here as they are not Java classes; a note on this is included at the end of this section.

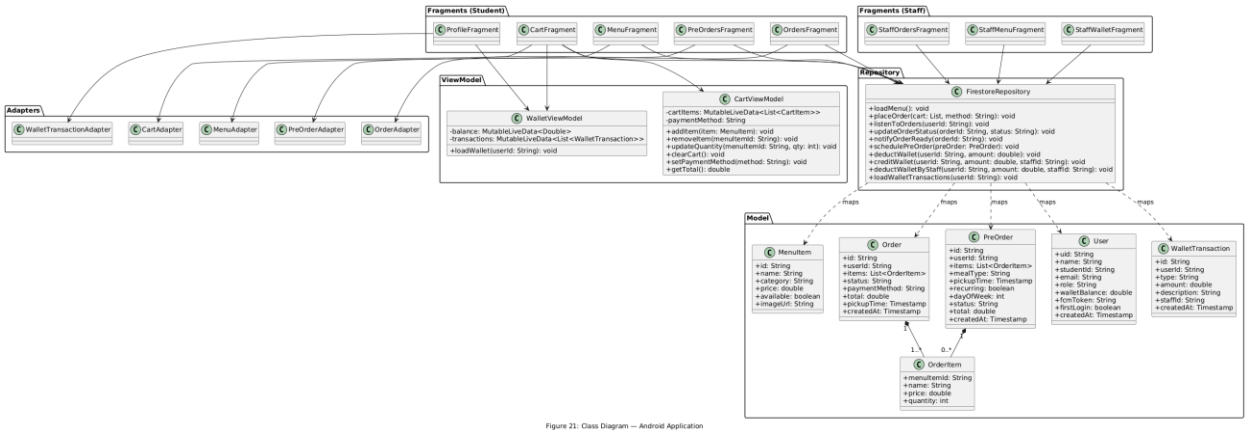


Figure 21: Class Diagram — Android Application

Figure 21 above shows the class diagram for the USIU Cafeteria Ordering System. The key classes are grouped by responsibility:

Model Classes (Data)

- *MenuItem:* id, name, category, price, available, imageUrl
- *Order:* id, userId, items (List<OrderItem>), status, paymentMethod, total, pickupTime, createdAt
- *OrderItem:* menuItemId, name, price, quantity
- *PreOrder:* id, userId, items (List<OrderItem>), mealType, pickupTime, recurring, dayOfWeek, status, total, createdAt
- *User:* uid, name, studentId, email, role, walletBalance, fcmToken, firstLogin, createdAt
- *WalletTransaction:* id, userId, type, amount, description, staffId, createdAt

Fragment Classes (UI)

- *MenuFragment:* Observes menuItems via FirestoreRepository; uses MenuAdapter and CategoryAdapter; navigates to CartFragment
- *CartFragment:* Observes CartViewModel; displays wallet balance; calls FirestoreRepository.placeOrder()
- *OrdersFragment:* Observes active orders via FirestoreRepository; calculates estimatedWait(); uses OrderAdapter
- *PreOrdersFragment:* Reads/writes preOrders via FirestoreRepository; uses PreOrderAdapter
- *ProfileFragment:* Observes WalletViewModel; displays balance and transaction history
- *StaffOrdersFragment:* Observes all orders via FirestoreRepository; calls updateOrderStatus()
- *StaffMenuFragment:* Calls FirestoreRepository.updateItemAvailability()
- *StaffWalletFragment:* Calls FirestoreRepository.creditWallet() or FirestoreRepository.deductWallet(); records staffId on each transaction

ViewModel Classes (State)

- *CartViewModel:* MutableLiveData<List<CartItem>>; addItem(), removeItem(), updateQuantity(), clearCart(), setPaymentMethod()
- *WalletViewModel:* MutableLiveData<Double> balance; MutableLiveData<List<WalletTransaction>>; loads from FirestoreRepository

Adapter Classes (RecyclerView)

- *MenuAdapter, CategoryAdapter, CartAdapter, OrderAdapter, PreOrderAdapter, WalletTransactionAdapter* Each extends RecyclerView.Adapter; binds the relevant model class to a ViewHolder layout

Repository Class (Data Access)

- *FirestoreRepository:* Central data access object; all Firestore reads and writes pass through this class. Key methods: loadMenu(), placeOrder(), listenToOrders(), updateOrderStatus(), notifyOrderReady() (HTTP POST to Supabase notify-order-ready edge function), schedulePreOrder(), deductWallet() (runTransaction), creditWallet() (runTransaction), deductWalletByStaff() (runTransaction, writes staffId), loadWalletTransactions()

Relationships: Fragments depend on FirestoreRepository and ViewModel classes. ViewModels hold LiveData observed by Fragments. Adapters are owned by Fragments. FirestoreRepository maps Firestore documents to model class instances using Firestore's toObject() method.

Note - Admin Web Panel (out of scope for this Java class diagram): The system includes a separate web-based admin panel built with Next.js and TypeScript. Its components - login page, dashboard pages for menu management and staff account creation, Next.js API routes using the Firebase Admin SDK, and Tailwind CSS UI - are TypeScript/React components and are not represented in this Java class diagram. The admin panel interacts with the same Firestore database and Firebase Authentication instance as the Android app, governed by the same Firestore security rules.

5.1.9 Wireframes

The wireframes for the USIU Cafeteria Ordering System serve as the visual blueprint for all application interfaces. They outline how each screen will be structured, what components will appear, and how users will navigate between functions. At this stage the focus is on layout and interaction flow rather than final colours or visual polish. The wireframes cover two applications: the Android mobile app (five student screens and three staff screens) and the web-based admin panel (four web pages). Together they ensure that every feature - menu browsing, cart checkout, order tracking, pre-order scheduling, wallet management, staff order processing, and admin account and menu control - is logically organised before implementation begins.

[Figure 22: Wireframes - Android app (Menu, Cart, Orders, Pre-orders, Profile/Wallet) and web admin panel (Login, Menu Management, Staff Creation, Users Directory)]
Figure 22 above represents the wireframes for the USIU Cafeteria Ordering System.

Screen 1: Menu

The Menu screen is the home screen for student users. A horizontal tab bar at the top allows filtering by category: All, Mains, Snacks, Drinks, and Specials. Below the tabs, a vertical scrollable list of item cards fills the main content area. Each card displays the item image, name, price, and an availability chip (green "Available" or red "Sold Out"). A floating action button (FAB) in the bottom-right corner shows the current cart item count and navigates to the Cart screen. A search icon in the top app bar expands an inline search field for text-based item lookup.

Screen 2: Cart

The Cart screen displays all items currently in the cart in a scrollable list. Each row shows the item name, unit price, and +/– quantity controls, with a remove button. A subtotal updates dynamically below the list. A payment method toggle allows the student to switch between Wallet (showing current balance) and Cash. If Wallet is selected and the balance is insufficient, a warning indicator appears below the toggle. A prominent "Place Order" button at the bottom is disabled if the cart is empty or if Wallet is selected with insufficient funds.

Screen 3: Orders

The Orders screen has two sections. The top section shows the active order (if one exists) with a three-step horizontal status bar: Pending → Preparing → Ready. The current step is highlighted. An estimated wait time chip (e.g, "~10 min") appears below the status bar, updating in real time as the queue changes. The bottom section is a scrollable history list of past completed orders, each showing date, items summary, total, and payment method. Tapping a past order expands it to show full details and a "Reorder" button.

Screen 4: Pre-orders

The Pre-orders screen shows a list of upcoming scheduled pre-orders, each displayed as a card with meal type, pickup time, total, recurring badge (if set), and a cancel button. A FAB opens the Schedule Pre-order dialog. The dialog contains: a meal type selector (Breakfast / Lunch / Dinner), a date and time picker for the pickup slot, an item selection interface (same as the menu), a recurring toggle with a day-of-week selector, and a wallet balance preview showing post-deduction balance at cut-off.

Screen 5: Profile / Wallet

The Profile/Wallet screen opens with a prominent wallet balance card at the top showing the current balance and a note indicating that top-ups are done at the cafeteria counter. Below the balance card is a scrollable list of recent wallet transactions (credits and debits) with date, description, and amount. Below transactions is the student's account details section (name, student ID, email) and a logout button.

Web Admin Panel (Sixth Interface)

The admin panel is a browser-based application accessible only to users with the admin role. It consists of four pages:

Admin Login Page: A centred login form with email and password fields and a "Sign In" button. On successful login, the system checks the role field - if not admin, the user is redirected. If firstLogin is true, the user is redirected to the change password page before accessing the dashboard.

Dashboard - Menu Management: A full-width data table listing all menu items with columns for name, category, price, availability toggle (switch), and image URL. An "Add Item" button above the table opens an inline form or modal with fields for name, description, category (dropdown), price, imageUrl, and availability. Each row has Edit and Delete action buttons. Changes write to the Firestore menuItems collection via the Firebase Client SDK.

Dashboard - Staff Account Creation: A simple form with fields for full name, email, and temporary password, and a "Create Staff Account" button. On submission, the Next.js API route calls the Firebase Admin SDK to create a Firebase Auth user and write a Firestore users document with role=staff and firstLogin=true. A success message shows the new account's email.

Dashboard - Users Directory: Two separate tables on one page. The first table lists all students (name, student ID, email, wallet balance, joined date). The second table lists all staff and admin accounts (name, email, role, firstLogin status). Both tables are read from the Firestore users collection, filtered by role, via the Firebase Client SDK.

5.2 Functional Requirements

Functional requirements define the specific actions and features that the Cafeteria Ordering System must provide to meet the needs of students, staff, and administrators.

Student Functional Requirements

- *FR1:* The system shall allow students to browse the cafeteria menu, filtered by category (breakfast, lunch, dinner) and item availability.
- *FR2:* The system shall allow students to add items to a cart, adjust quantities, and review the order before confirming.
- *FR3:* The system shall allow students to place a regular order and choose to pay via the in-app wallet or declare cash payment at the counter.
- *FR4:* The system shall display a real-time estimated wait time for each order, calculated from the number of pending and preparing orders ahead in the queue and the average preparation time per item.

- *FR5:* The system shall allow students to place a smart pre-order for a future meal slot (lunch or dinner), subject to cut-off times (10:00 AM for lunch, 5:00 PM for dinner).
- *FR6:* The system shall allow students to set a pre-order as recurring on selected days of the week (e.g, every Thursday at 12:30).
- *FR7:* The system shall allow students to view the status of their active and past orders in real time.
- *FR8:* The system shall allow students to view their wallet balance and a transaction history of top-ups and deductions.

Staff Functional Requirements

- *FR9:* The system shall allow staff to view all incoming orders and update their status (Pending → Preparing → Ready → Collected).
- *FR10:* The system shall allow staff to mark a cash order as paid when the student collects and pays at the counter.
- *FR11:* The system shall allow staff to manage the menu - add, edit, enable, or disable menu items and update daily availability.
- *FR12:* The system shall allow staff to credit (top up) or debit (deduct) a student's wallet balance by entering the student's ID and the amount; each transaction shall record the staff member's ID (staffId) for accountability.

Admin Functional Requirements

- *FR13:* The system shall allow administrators to create staff accounts via the web admin panel; each new staff account shall be assigned a temporary password, and the staff member shall be required to change it on first login before accessing any operational screens.
- *FR14:* The system shall allow administrators to add, edit, delete, and toggle the availability of menu items via the web admin panel, with changes reflected immediately on all student Menu screens through Firestore snapshot listeners.

- *FR15:* The system shall allow administrators to view a directory of all users, with students and staff/admin accounts displayed in separate tables.

System Functional Requirements

- *FR16:* The system shall automatically process all confirmed pre-orders at the respective cut-off time via the Supabase process-cutoff Edge Function (called by cron-job.org at 10:00 AM EAT for lunch and 5:00 PM EAT for dinner): deduct the order total from the student's wallet if funds are sufficient, or cancel the pre-order and send an FCM push notification if the wallet balance is insufficient.
- *FR17:* The system shall send FCM push notifications to students when their order status changes to Ready (via the Supabase notify-order-ready Edge Function, called by the Android app after the staff status update) and when a pre-order is confirmed or cancelled due to insufficient funds (via the process-cutoff Edge Function).
- *FR18:* The system shall enforce secure authentication via Firebase Authentication, ensuring that students (Android app), staff (Android app), and administrators (web panel) access only the data and screens appropriate to their role, as enforced by Firestore security rules using isStudent(), isStaff(), isAdmin(), and isPrivileged() helper functions.
- *FR19:* The system shall perform all wallet deductions atomically using Firestore runTransaction() to prevent race conditions or double-spending.
- *FR20:* The system shall maintain a complete, timestamped audit trail of all wallet transactions (top-ups and deductions) in the walletTransactions Firestore collection, including the staffId of the staff member who performed each transaction.

5.3 Non-Functional Requirements

Non-functional requirements describe the quality attributes and constraints that the Cafeteria Ordering System must satisfy beyond its core features.

- *NFR1: Performance Requirements* The system shall display the current menu and cart within 2 seconds on a stable mobile data or Wi-Fi connection. Real-time order status

updates shall be reflected in the app within 3 seconds of a staff status change. The estimated wait time calculation shall execute entirely on the client using a cached Firestore query result and shall complete within 1 second.

- *NFR2: Security Requirements* All users must authenticate via Firebase Authentication before accessing any system function. Firestore security rules shall enforce role-based access using three helper functions - isStudent(), isStaff(), and isAdmin() - with an isPrivileged() helper for shared staff/admin operations: students may only read and write their own orders, pre-orders, and wallet records; staff may read and update all orders and perform wallet top-ups and deductions; admin may read and write all collections. No client may directly modify another user's wallet balance without a server-side Firestore transaction. Supabase Edge Functions authenticate via a shared FUNCTIONS_SECRET header; the admin panel's privileged operations (staff account creation) execute via Next.js server-side API routes using the Firebase Admin SDK, never exposing admin credentials to the browser. Wallet deductions use Firestore runTransaction() to guarantee atomicity and prevent double-spending.

- *NFR3: Usability Requirements* The application shall follow Material Design 3 guidelines with USIU branding (navy #002147, gold #CFB991) to ensure a consistent and familiar experience. Navigation between the five main screens (Menu, Cart, Orders, Pre-orders, Profile/Wallet) shall require no more than one tap from any screen. The pre-order form shall clearly display the applicable cut-off time and wallet-only payment constraint so that students are not surprised by automatic deductions.

- *NFR4: Reliability Requirements* The system shall rely on Firebase's managed infrastructure, which provides 99.95% uptime SLA, to store all order and wallet data. Pre-order cut-off processing shall be performed by the Supabase process-cutoff Edge Function, triggered by cron-job.org at the scheduled times, rather than client devices - ensuring that deductions occur even when no student has the app open. All Firestore writes shall use offline persistence so that students can browse the menu and build a cart without an active connection, with synchronisation occurring when connectivity is restored.

- *NFR5: Scalability* The system shall use Firebase Firestore, a horizontally scalable NoSQL database, allowing the number of concurrent users, menu items, and orders to grow without schema migrations or infrastructure changes. Adding new meal slots, menu categories, or staff accounts shall require only Firestore document inserts or Firebase Authentication role assignments, with no changes to application code.

- *NFR6: Maintainability* All Firestore interactions in the Android app shall be centralised in a FirestoreRepository class, so that query logic can be updated in one place without modifying Activity or Fragment code. Business logic for pre-order cut-off shall reside exclusively in the Supabase process-cutoff Edge Function (TypeScript), making it independently deployable, testable via HTTP POST, and updatable without rebuilding the Android APK. The admin panel's privileged Firebase operations shall reside in Next.js API routes, keeping admin SDK credentials off the client.

- *NFR7: Compliance* The system shall not store any payment card or M-Pesa credentials. Wallet top-ups are performed in person (cash at the counter), and all financial records are maintained within the university's own Firestore project, reducing data-sovereignty and regulatory risk.

5.4 Conclusion

The Cafeteria Ordering System is a project designed to address the real challenges of food ordering and queue management at the United States International University Africa (USIU-Africa) cafeteria. This chapter has presented the complete system design, covering the system models, functional requirements, and non-functional requirements that together define both what the system must do and the quality standards it must meet.

The system models section documented the architecture of a three-tier Android and Firebase solution and illustrated its behaviour through a comprehensive set of diagrams: use case, entity-relationship, data flow, flowchart, activity, sequence, class, and wireframe diagrams. These diagrams confirm that the system is well-structured, with clear separation between the student-facing mobile application, the Firebase backend services, and the staff management interface.

The functional requirements capture seventeen specific capabilities spanning student ordering (browse, cart, regular order, pre-order, recurring schedule), real-time estimated wait time, staff management (order processing, menu management, wallet top-up), and system-level automation (pre-order cut-off via Cloud Functions, FCM push notifications, atomic Firestore transactions, and a complete wallet audit trail). The non-functional requirements establish the performance, security, usability, reliability, scalability, maintainability, and compliance standards that the implementation must satisfy.

In conclusion, the analysis and design presented in this chapter provide a solid and feasible blueprint for the Cafeteria Ordering System. The combination of a modern Android front end, a scalable Firebase backend, and automated pre-order processing through Cloud Functions distinguishes this system from conventional point-of-sale or manual queuing approaches. With this design in place, the implementation phase, covered in the following chapter, can proceed with clarity and confidence.

6 CHAPTER 6: IMPLEMENTATION

The implementation phase of the USIU Cafeteria Ordering System represented the transformation of the design specifications established in Chapter 5 into two fully operational, production-ready deliverables: a native Android mobile application serving students and cafeteria staff, and a web-based admin panel for system administrators. This chapter documents the development process, the technical structures adopted, and the integration patterns that govern how all components communicate, from the user interface through to the cloud backend. The Android application was built in Java using XML-based layouts and Material Design 3 components, following an MVVM architecture with a centralised FirestoreRepository for all data access. The admin panel was built as a separate Next.js 14 application using TypeScript and Tailwind CSS, sharing the same Firebase project. The backend relies on Firebase Firestore, Firebase Authentication, and Firebase Cloud Messaging, supplemented by Supabase Edge Functions for scheduled pre-order processing and push notification dispatch. All infrastructure is hosted on free-tier plans, resulting in a production-ready system at zero operational cost.

6.1 Introduction

The implementation phase of the USIU Cafeteria Ordering System represented the crucial transformation of the theoretical design specifications and comprehensive requirements documentation into fully operational, production-ready software. This development stage encompassed the systematic construction of two distinct client applications - a native Android mobile application and a web-based admin panel - along with the configuration of a Firebase cloud backend and two Supabase Edge Functions that handle automated pre-order processing and push notification dispatch. The primary objective was to deliver a robust, intuitive, and feature-complete cafeteria ordering solution addressing the needs of three user roles: students who browse the menu, place orders, and manage pre-orders and wallet balances; cafeteria staff who process incoming orders and handle student wallet operations; and administrators who manage the menu catalogue and create staff accounts. Special emphasis was placed on ensuring that all payment operations - wallet deductions, top-ups, and pre-order charges at cut-off time -

execute atomically using Firestore transactions, preventing any inconsistency in wallet balances under concurrent access.

The implementation prioritised simplicity and reliability over complexity. The Android app follows a single-activity architecture with a BottomNavigationView and fragment-based navigation using hide/show transactions to preserve scroll state. All Firestore access is centralised in a single FirestoreRepository class, and UI state is managed through ViewModel and LiveData classes following the MVVM pattern. The admin panel uses Next.js 14 App Router with server-side Firebase Admin SDK calls to securely create staff accounts and manage Firestore data. Both applications share the same Firebase project and Firestore database, governed by a unified set of security rules that enforce role-based access at the document level.

6.2 Frontend Implementation

The frontend of the USIU Cafeteria Ordering System comprises two distinct client applications. The primary deliverable is a native Android application built in Java with XML layouts, serving both students and cafeteria staff through role-specific navigation. The secondary deliverable is a web-based admin panel built with Next.js 14 and TypeScript, accessible via a desktop browser for system administrators. Both applications adhere to Material Design 3 principles and USIU branding guidelines, incorporating the university's signature navy (#002147) and gold (#CFB991) colour palette throughout all interface elements.

i. Authentication and Role Routing

Authentication is handled centrally by Firebase Authentication using email and password credentials. The LoginActivity submits user credentials via Firebase Auth's signInWithEmailAndPassword method. On successful authentication, the app immediately reads the users/{uid} Firestore document to retrieve the role field. If the role is "student", the user is directed to MainActivity with the student bottom navigation configuration. If the role is "staff", the user is directed to StaffMainActivity with the staff navigation. This role check prevents students from accessing staff screens and vice versa without requiring server-side session management. Students self-register through the RegisterActivity, which creates a Firebase Authentication user and simultaneously writes a Firestore document to the users collection with role: "student" and walletBalance: 0.0. Staff accounts cannot be self-registered; they are created

exclusively by administrators via the web admin panel, which calls a Firebase Admin SDK API route to provision both the Auth user and the Firestore document. When a staff or admin user logs in for the first time, the firstLogin field on their Firestore document is true. The app detects this and redirects to ChangePasswordActivity before granting access to any operational screen, enforcing a mandatory password change on initial access.

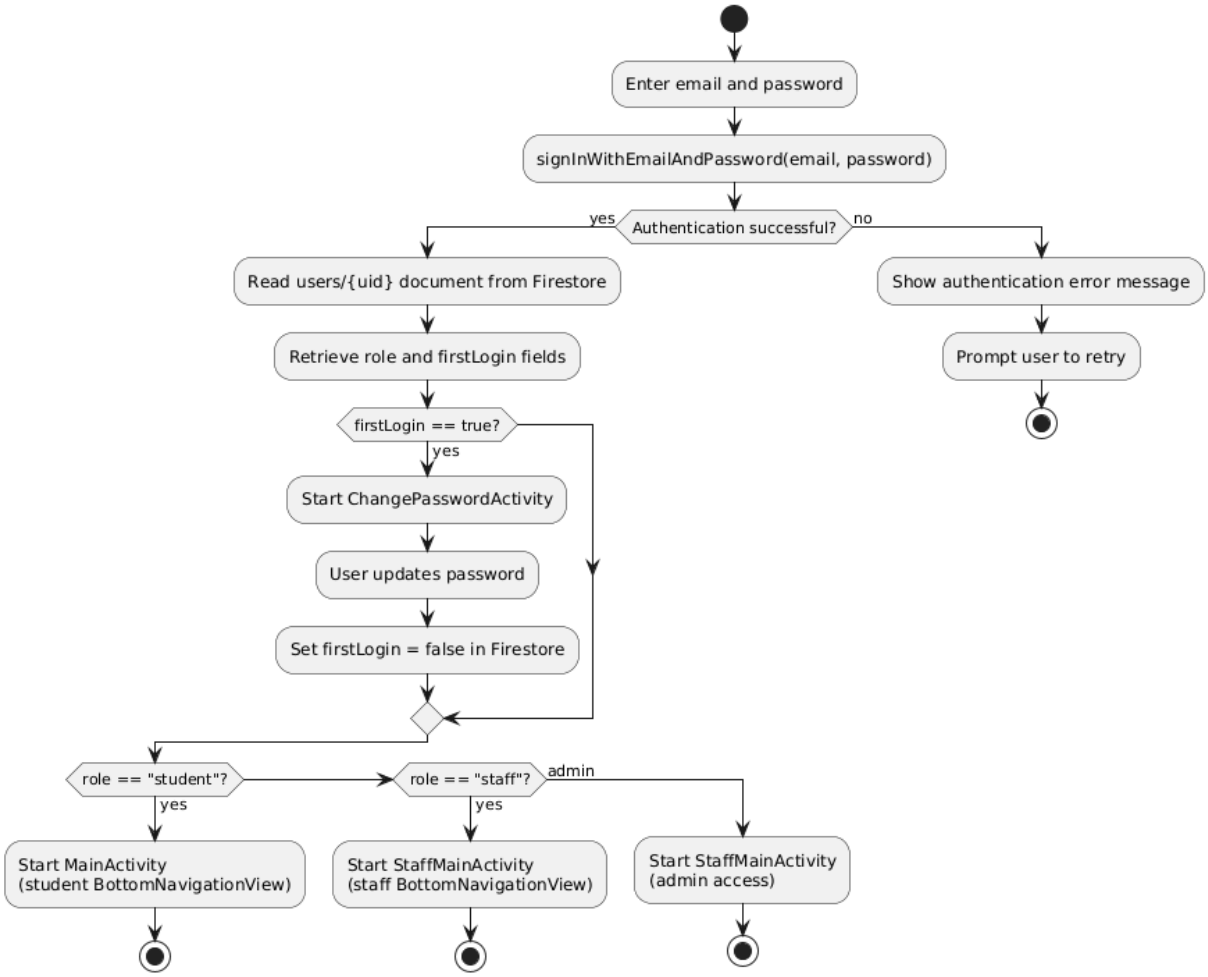


Figure 23: LoginActivity — Firebase Authentication and Role-based Routing

Figure 23 illustrates the authentication flow. On login, the signInWithEmailAndPassword callback reads the user's Firestore document to determine role and firstLogin status. If firstLogin is true, the activity starts ChangePasswordActivity and finishes itself, preventing back-navigation to the pre-change state. Once the password is updated, firstLogin is set to false in Firestore, and the user is routed to the appropriate main activity.

ii. Student Interface - Menu and Cart

The student interface is hosted within MainActivity, which manages five fragments through a BottomNavigationView. Navigation uses fragment hide/show transactions rather than replace transactions; this preserves the RecyclerView scroll position in the Menu and Orders tabs when the user switches between screens, eliminating the need to re-query Firestore on every tab switch. The MenuFragment serves as the home screen for student users. It presents a horizontal ChipGroup at the top of the screen allowing the user to filter items by category: All, Breakfast, Lunch, and Dinner. Below the chips, a RecyclerView backed by a MenuAdapter displays item cards. Each card shows the item image (loaded asynchronously using Glide with a placeholder drawable), the item name, price in KES, and a short description. Items with available == false render with a "Sold Out" chip and a disabled, greyed-out "Add to Cart" button, ensuring that unavailable items are visible but not orderable. Tapping the "Add to Cart" button dispatches the item to CartViewModel and shows a snackbar confirming the addition. Category filtering is performed in-memory on the already-loaded LiveData list, requiring no additional network call.

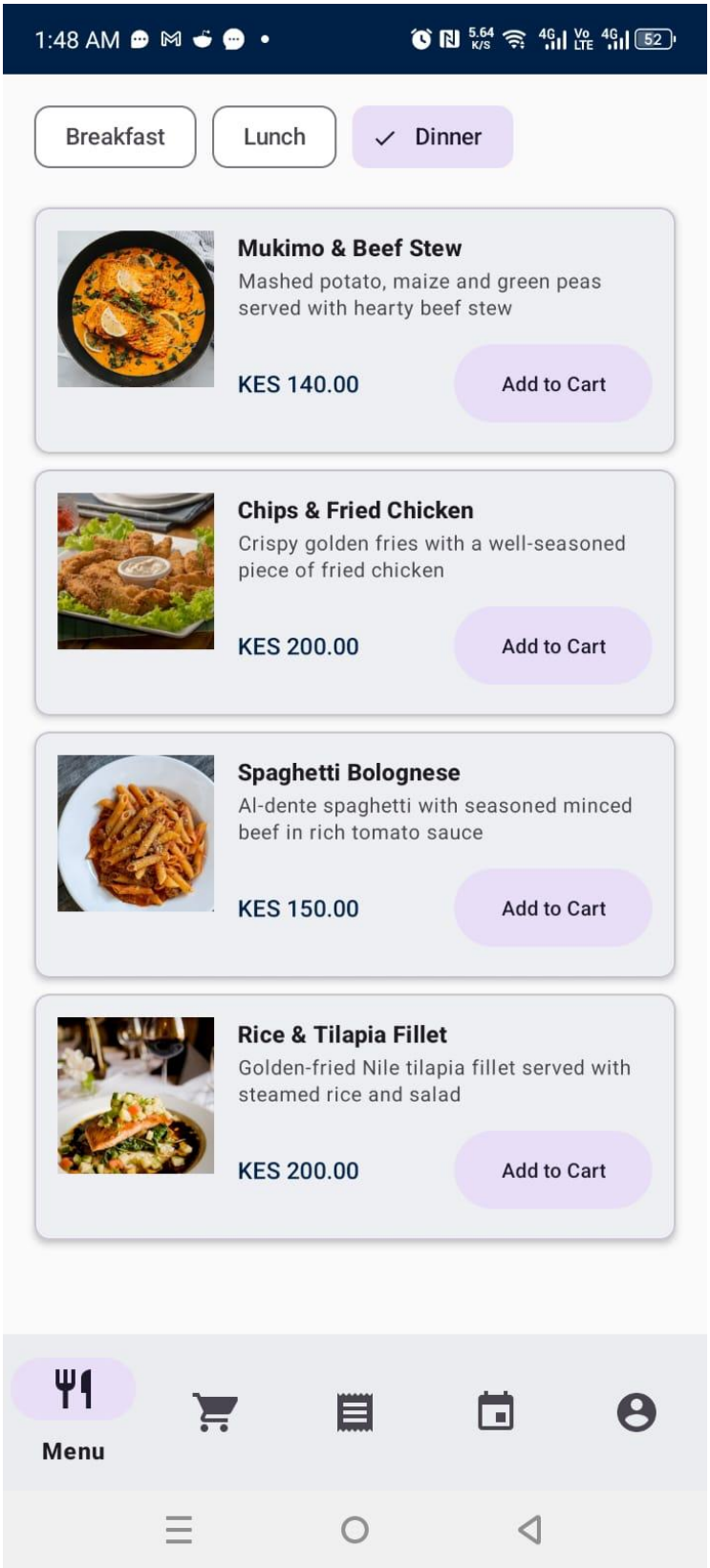

1:48 AM
Breakfast
Lunch
Dinner
Mukimo & Beef Stew
Mashed potato, maize and green peas served with hearty beef stew
KES 140.00
Add to Cart
Chips & Fried Chicken
Crispy golden fries with a well-seasoned piece of fried chicken
KES 200.00
Add to Cart
Spaghetti Bolognese
Al-dente spaghetti with seasoned minced beef in rich tomato sauce
KES 150.00
Add to Cart
Rice & Tilapia Fillet
Golden-fried Nile tilapia fillet served with steamed rice and salad
KES 200.00
Add to Cart
Menu

Figure 24 shows the Menu screen. The RecyclerView data originates from FirestoreRepository.listenToMenuItems(), which attaches a Firestore real-time snapshot listener. Menu changes made by staff or admin are reflected on the student's screen within seconds without any manual refresh.

The CartFragment displays all items currently held in CartViewModel, which is a shared ViewModel that survives fragment switches. Each row in the cart RecyclerView shows the item name, unit price, and a quantity stepper with minus and plus buttons. Tapping minus on an item with quantity 1 removes it from the cart entirely. A summary section below the list shows the subtotal and an estimated wait time. The estimated wait is computed by querying the count of orders currently in "pending" or "preparing" status and multiplying by a constant of five minutes per order (minimum displayed is five minutes if the queue is empty). A RadioGroup beneath the summary lets the student toggle between wallet payment and cash payment. When wallet is selected, the current balance from WalletViewModel is displayed alongside the option. If the balance is insufficient, a warning label appears and the "Place Order" button remains disabled until the student switches to cash or the balance is topped up. Tapping "Place Order" triggers the appropriate Firestore transaction based on the selected payment method.

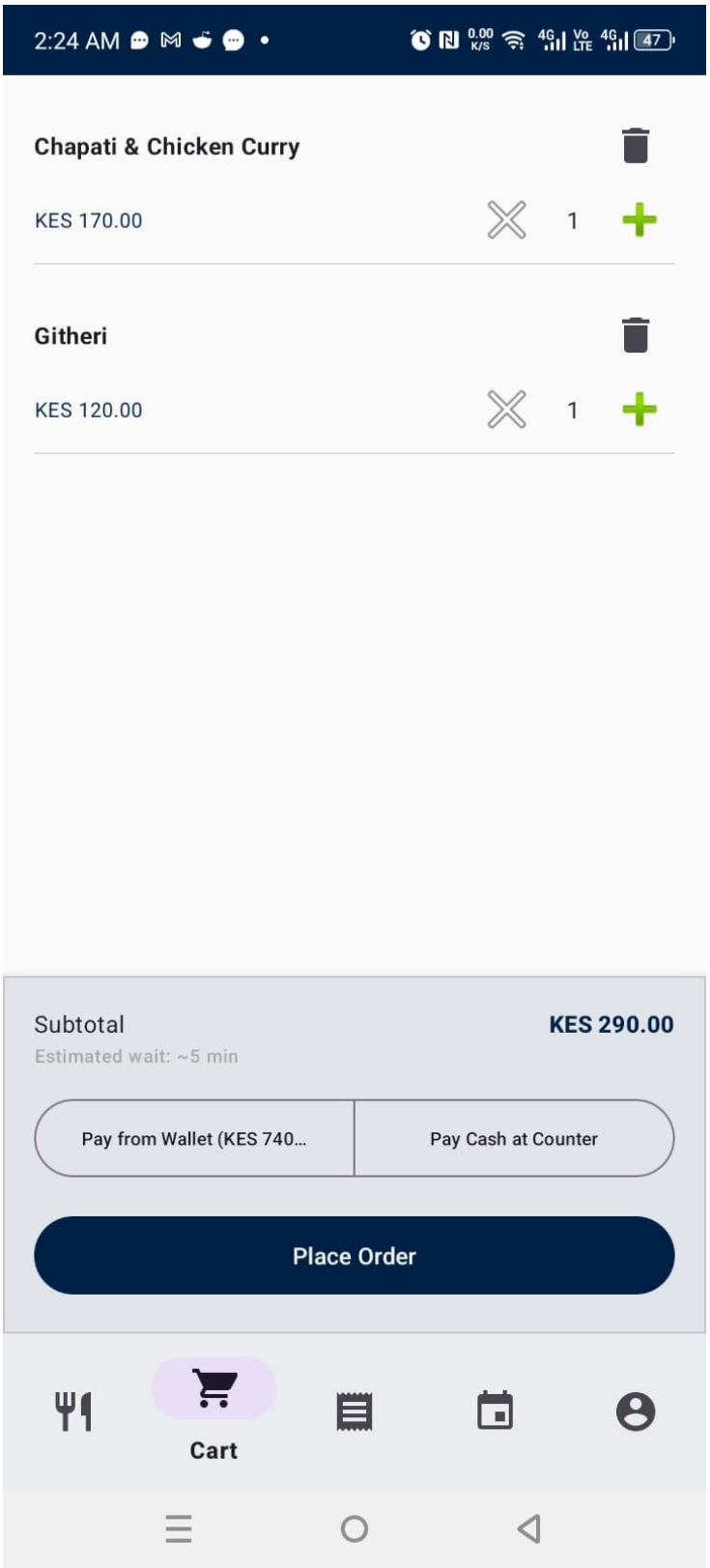

2:24 AM
Chapati & Chicken Curry
KES 170.00
1
Githeri
KES 120.00
1
Subtotal
KES 290.00
Estimated wait: ~5 min
Pay from Wallet (KES 740...
Pay Cash at Counter
Place Order
Cart

Figure 25 shows the Cart screen. The wallet balance on the payment toggle is sourced from WalletViewModel's real-time Firestore listener and updates automatically if a top-up occurs while the student is reviewing their cart. The insufficient-balance guard disables the Place Order button locally - the check runs against the LiveData balance before any Firestore write is attempted, so no server round-trip is needed to enforce it.

iii. Student Interface - Orders, Pre-orders, and Wallet

The OrdersFragment presents a two-tab layout. The "Active" tab shows orders where the status is "pending", "preparing", or "ready", each displayed as a card with a status chip coloured amber (pending), blue (preparing), or green (ready). A real-time Firestore snapshot listener attached in FirestoreRepository.listenToStudentOrders() ensures that when cafeteria staff advance an order's status, the chip on the student's screen updates without requiring a manual refresh. When an order reaches "ready", the student also receives a Firebase Cloud Messaging push notification. The "History" tab displays orders with "collected" status in reverse chronological order.

The PreOrdersFragment shows the student's scheduled pre-orders as a scrollable list. A floating action button opens a bottom sheet dialog where the student selects a meal slot (Lunch or Dinner), a scheduled date, and items from the menu. A recurring toggle presents day-of-week checkboxes for weekly repetition. Validation ensures the selected date is not in the past and that, if today is chosen, the current time is before the cut-off for the selected slot (10:00 AM for Lunch, 5:00 PM for Dinner). Students may cancel a pre-order only while its status is "scheduled", before the cut-off processing has occurred. No wallet deduction takes place at scheduling time; deduction occurs at the cut-off via the Supabase Edge Function. The ProfileWalletFragment displays the student's current wallet balance in a prominent card at the top of the screen, with an informational note indicating that top-ups are handled at the cafeteria counter. Below the balance card, a RecyclerView backed by a WalletTransactionAdapter lists all wallet transactions in reverse chronological order. Each row shows a directional icon (green arrow for credits, red arrow for debits), the transaction description (e.g, "Top-up by staff" or "Order #abc123"), the amount, and the timestamp. A logout button at the bottom calls FirebaseAuth.signOut() and starts LoginActivity with the back stack cleared.

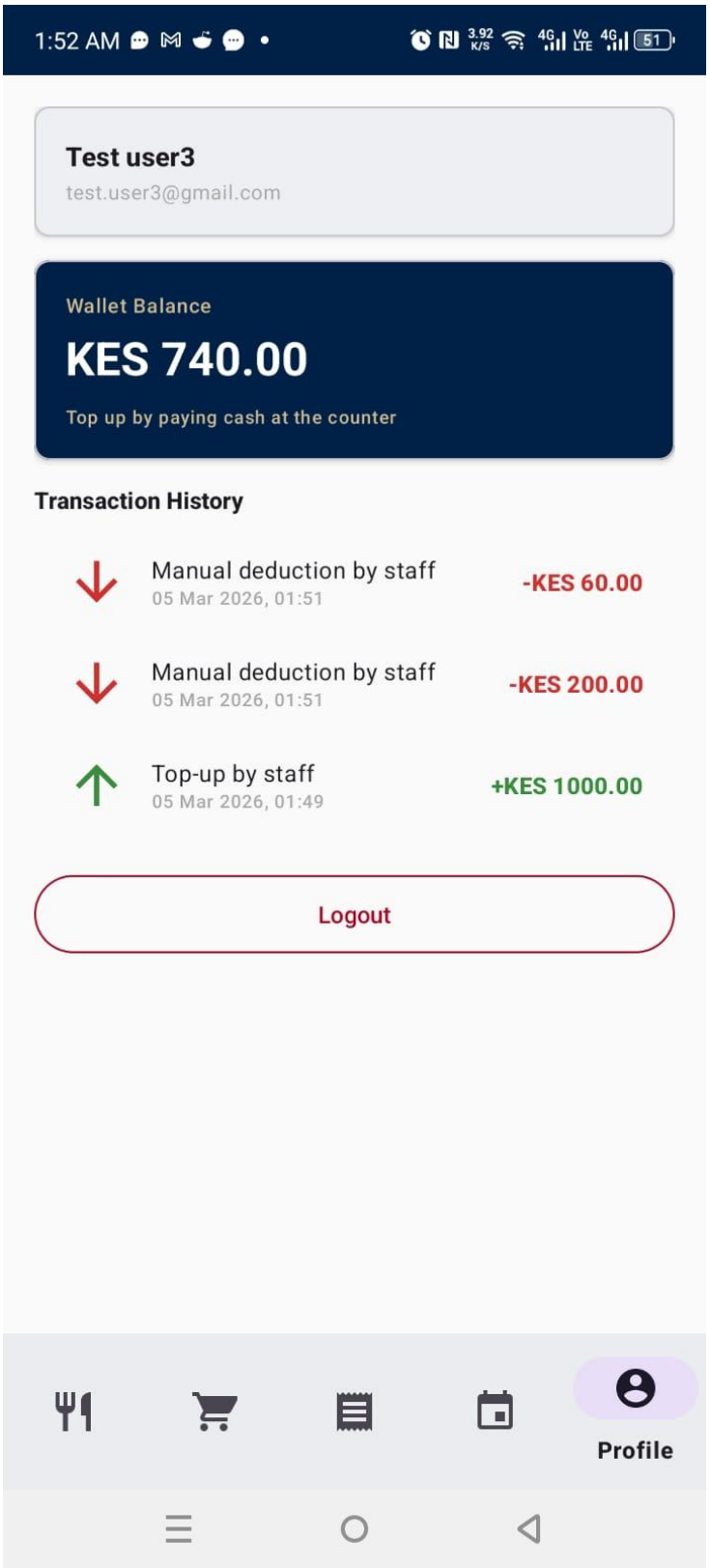
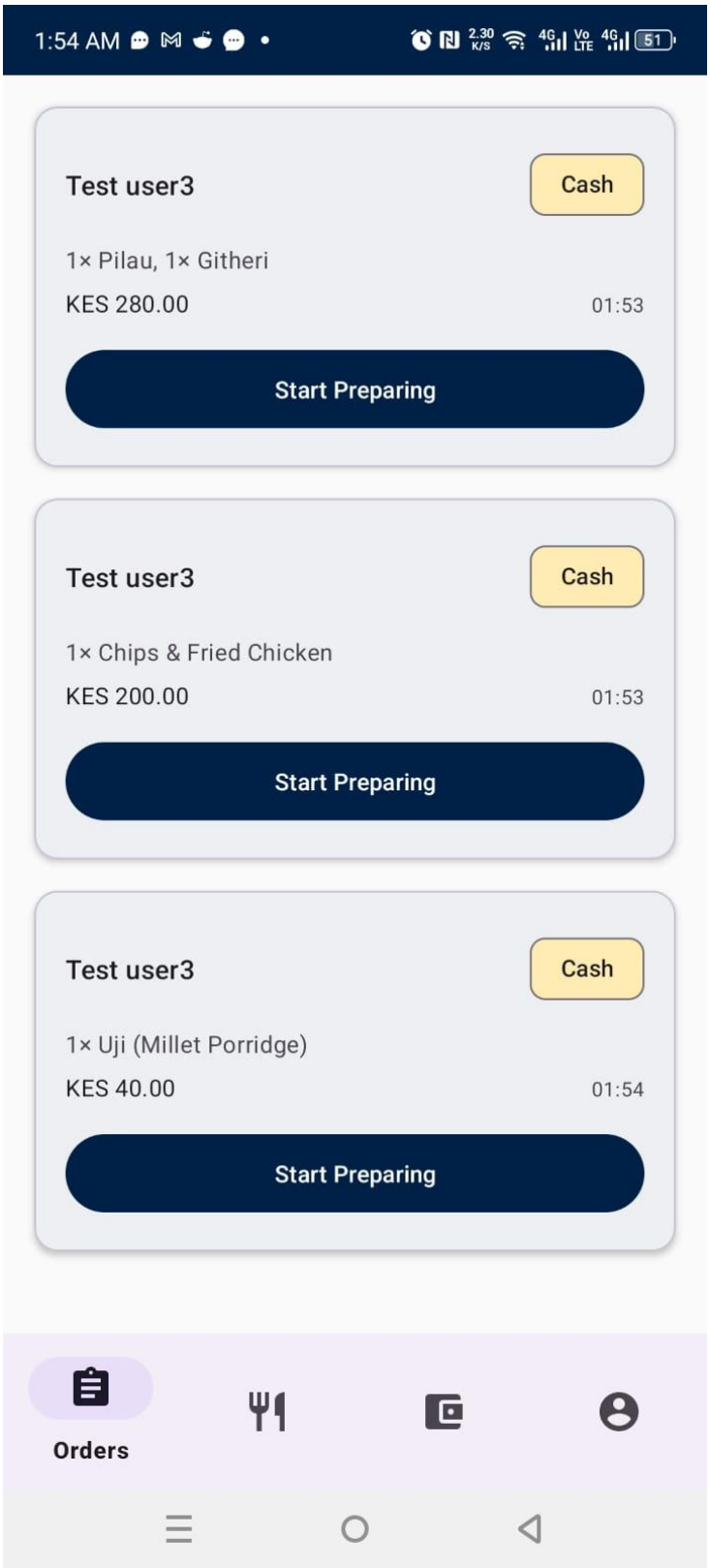

1:52 AM
Test user3
test.user3@gmail.com
Wallet Balance
KES 740.00
Top up by paying cash at the counter
Transaction History
Manual deduction by staff
05 Mar 2026, 01:51
-KES 60.00
Manual deduction by staff
05 Mar 2026, 01:51
-KES 200.00
Top-up by staff
05 Mar 2026, 01:49
+KES 1000.00
Logout
Profile

Figure 26 shows the Profile/Wallet screen. The balance card sources its value from the same WalletViewModel instance shared across the app, so it reflects any top-up or deduction that occurred during the session without requiring the student to navigate away and return. Each transaction row shows a directional icon, description, amount, and timestamp, giving the student a full audit trail of their wallet activity.

iv. Staff Interface

The staff interface is hosted in a separate StaffMainActivity with its own BottomNavigationView presenting three tabs: Orders, Menu, and Wallet. This separation ensures that staff users never see student navigation items and vice versa, reducing cognitive load and preventing accidental interactions. The StaffOrdersFragment is the primary operational screen for cafeteria staff. It displays all active orders across all students - those in "pending", "preparing", or "ready" status - in a single real-time RecyclerView. Each card shows the student name, a list of ordered items with quantities, the total amount, the payment method (wallet or cash), and the current status. An action button on each card advances the order to its next state: "Start Preparing" moves a pending order to preparing; "Mark Ready" moves a preparing order to ready and simultaneously triggers an HTTP POST to the notify-order-ready Supabase Edge Function, which sends a push notification to the student's device. "Mark Collected" finalises the order. For cash orders, the "Mark Collected" step reminds the staff member to collect physical payment before confirming.


1:54 AM
Test user3
Cash
1× Pilau, 1× Githeri
KES 280.00
01:53
Start Preparing
Test user3
Cash
1× Chips & Fried Chicken
KES 200.00
01:53
Start Preparing
Test user3
Cash
1× Uji (Millet Porridge)
KES 40.00
01:54
Start Preparing
Orders

The StaffMenuFragment provides a CRUD interface for menu items. Each item card includes an availability toggle (Switch) that staff can flip to immediately mark an item as unavailable without reloading the screen. A floating action button opens an "Add Item" dialog with fields for name, description, category, price, image URL, and availability. Tapping an existing item opens a pre-filled edit dialog. All changes are written to the menuItems Firestore collection and immediately reflected on student devices through their active snapshot listeners.

The StaffMenuFragment provides a CRUD interface for menu items. Each item card includes an availability toggle (Switch) that staff can flip to immediately mark an item as unavailable without reloading the screen. A floating action button opens an "Add Item" dialog with fields for name, description, category, price, image URL, and availability. Tapping an existing item opens a pre-filled edit dialog. All changes are written to the menuItems Firestore collection and immediately reflected on student devices through their active snapshot listeners.

The StaffWalletFragment allows staff to manage student wallet balances. The screen presents a lookup form where the staff member enters a student ID. Tapping "Find Student" queries the users collection and displays the student's name and current balance. The staff member then enters an amount and selects either "Credit" (top-up) or "Debit" (deduction, for cases such as correcting an overpayment). Tapping "Confirm" calls the appropriate FirestoreRepository method - creditWallet() or deductWalletByStaff() - both of which use Firestore runTransaction() for atomicity and record the staff member's UID in the staffId field of the resulting walletTransactions document for audit purposes.

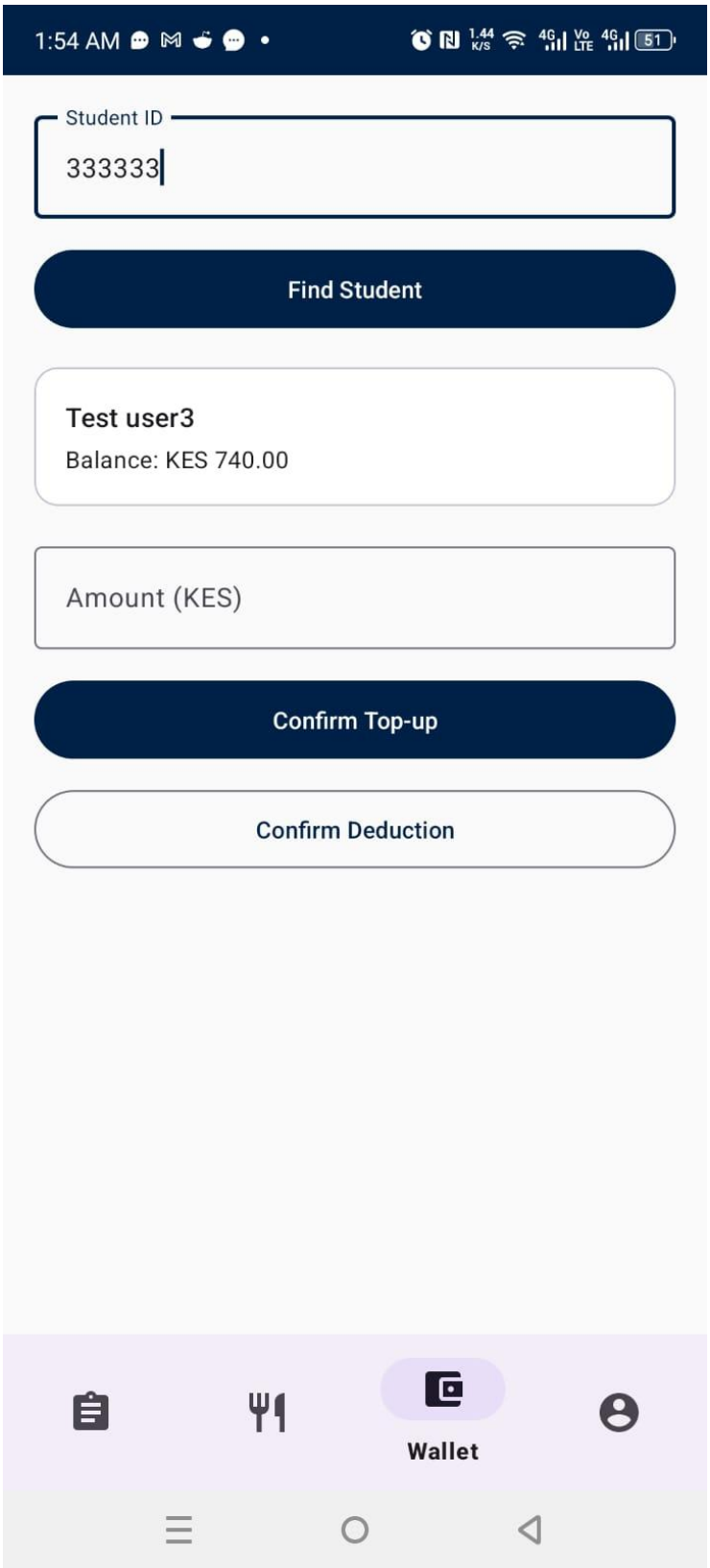

1:54 AM
Student ID
333333
Find Student
Test user3
Balance: KES 740.00
Amount (KES)
Confirm Top-up
Confirm Deduction
Wallet

Figure 28 shows the Staff Wallet screen. The student ID lookup queries the users collection before any balance operation is initiated, ensuring the staff member confirms the correct student name and current balance before committing a credit or debit. Both operations use Firestore runTransaction for atomicity and embed the staff member's UID in the resulting walletTransactions document, providing a full audit trail.

v. Admin Web Panel

The web-based admin panel is a separate Next.js 14 application deployed independently of the Android app. It is accessible only via desktop browser and is role-gated: login is handled through the Firebase Client SDK, and upon authentication the panel reads the user's role field; any non-admin user is redirected to the login page. Admin accounts with firstLogin: true are redirected to a change-password page before accessing the dashboard. The dashboard consists of four pages. The Menu Management page presents a full-width data table of all menu items with inline availability toggles, edit buttons, and delete buttons. The Staff Account Creation page provides a form for entering the new staff member's name, email, and temporary password; submission calls a Next.js API route (/api/create-staff) which uses the Firebase Admin SDK to create the Firebase Auth user and write the Firestore document with role: "staff" and firstLogin: true. The Users Directory page displays two separate tables - one for students and one for staff and admin accounts - both read from Firestore, filtered by role, providing the administrator with a complete view of all system users.

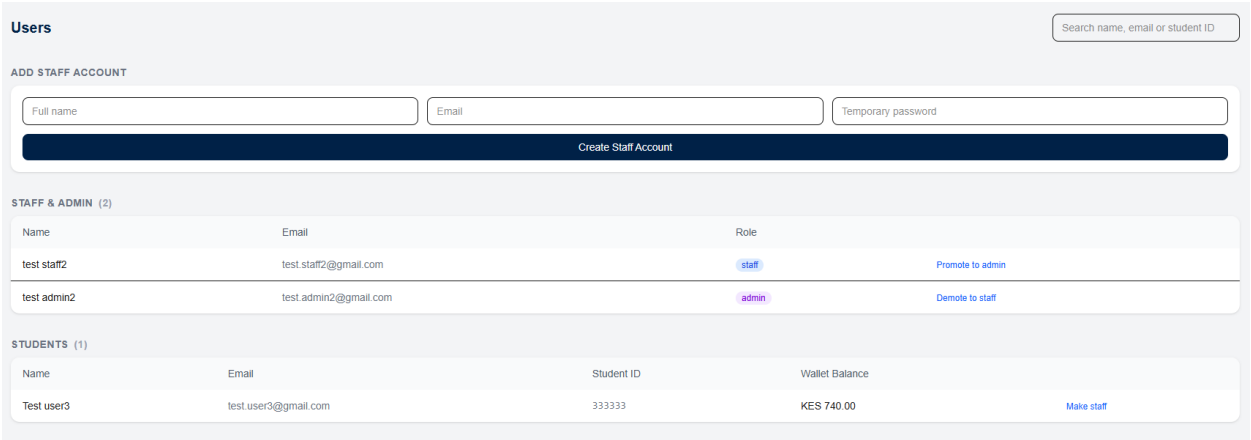


Figure 29 shows the admin panel's staff account creation interface. The Admin SDK API route runs server-side, keeping the service account credentials off the client and ensuring that only the server can create Auth users with arbitrary roles.

6.3 Backend Implementation

The backend architecture of the USIU Cafeteria Ordering System is built on Firebase's managed cloud services supplemented by Supabase Edge Functions for scheduled automation. Unlike a traditional server-client model requiring a dedicated application server, the system uses Firebase Firestore as both the real-time database and the source of truth for all application state, Firebase Authentication for identity management, and Firebase Cloud Messaging for push notifications. Supabase Edge Functions - serverless TypeScript functions running on the Deno runtime - handle the two automation tasks that require a server-side execution context: processing pre-order cut-offs on a schedule, and dispatching push notifications when an order status changes to "ready". The entire backend operates on free-tier plans with no ongoing operational cost.

i. Firestore Data Model

The system's data is organised across five Firestore collections at the root of the database. The users collection stores one document per registered user, identified by the Firebase Auth UID. Each document contains the user's name, email, studentId, role ("student", "staff", or "admin"), walletBalance (present only for students), fcmToken (the device's current FCM registration token), and firstLogin (a boolean that is true until the user completes their first-login password change). Staff and admin users do not have a walletBalance field; their documents contain only identity and role data.

The menuItems collection holds one document per menu item, with fields for name, description, price, category ("breakfast", "lunch", or "dinner"), imageUrl, and available (a boolean availability flag). The orders collection stores customer order documents, each containing an embedded list of ordered items (name, price, quantity per item), the total amount, payment method, status, and an estimatedWaitMin value calculated at order creation time. The preOrders collection holds pre-order documents with a scheduledDate field (formatted "YYYY-MM-DD"), mealSlot, recurring flag, recurringDays list, and a status field that transitions through "scheduled", "confirmed", and

"cancelled". The walletTransactions collection records every wallet credit and debit, with a staffId field on staff-initiated transactions providing an audit trail linking every balance change to the staff member who performed it.

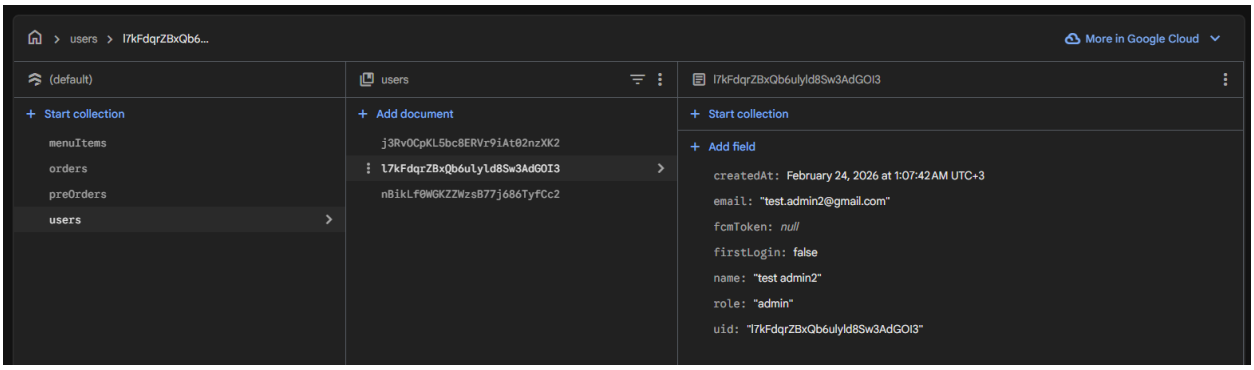


Figure 30 shows the Firestore database structure. Composite indexes are deployed for the five most performance-critical query patterns: student order history (userId ASC + createdAt DESC), staff order queue (status ASC + createdAt ASC), student pre-order listing (userId ASC + createdAt DESC), pre-order cut-off query (mealSlot ASC + scheduledDate ASC + status ASC), and wallet transaction history (userId ASC + createdAt DESC). These indexes were created via the Firebase Console and are required for Firestore to execute compound queries efficiently without throwing "index required" errors at runtime.

ii. FirestoreRepository Pattern

All Firestore reads and writes in the Android application are routed through a single class, FirestoreRepository. No Fragment or ViewModel accesses the Firestore SDK directly; they call methods on the repository, which returns LiveData objects or accepts callbacks. This pattern enforces a strict separation between UI logic and data access, makes all database operations auditable in one place, and simplifies future testing and maintenance. The repository exposes distinct methods for each data operation. listenToMenuItems() attaches a Firestore addSnapshotListener to the menuItems collection and pushes updates to a MutableLiveData list. listenToStudentOrders(userId) queries the orders collection filtered by userId, ordered by createdAt descending, and streams results via LiveData. listenToAllActiveOrders() serves the staff screen, querying for orders where status is in "pending", "preparing", or "ready". placeOrderWithWalletDeduction(order) executes a Firestore runTransaction that atomically

deducts the order total from walletBalance and writes the order and wallet transaction documents in a single server-side operation. creditWallet(userId, amount, staffId) and deductWalletByStaff(userId, amount, staffId) both use runTransaction and embed the staffId in the resulting walletTransactions document. notifyOrderReady(orderId, userId) makes an HTTP POST to the Supabase notify-order-ready Edge Function after a staff member marks an order as "ready".

```
FirestoreRepository
-db : FirebaseFirestore
-auth : FirebaseAuth
+listenToMenuItems() : LiveData<List<MenuItem>>
+listenToStudentOrders(userId : String) : LiveData<List<Order>>
+listenToAllActiveOrders() : LiveData<List<Order>>
+getActiveOrderCount() : Task<Integer>
+placeOrderWithWalletDeduction(order : Order) : Task<Void>
+placeOrderCash(order : Order) : Task<Void>
+updateOrderStatus(orderId : String, status : String) : Task<Void>
+notifyOrderReady(orderId : String, userId : String) : void
+creditWallet(userId : String, amount : Double, staffId : String) : Task<Void>
+deductWalletByStaff(userId : String, amount : Double, staffId : String) : Task<Void>
+listenToWalletBalance(userId : String) : LiveData<Double>
+loadWalletTransactions(userId : String) : LiveData<List<WalletTransaction>>
+updateItemAvailability(itemId : String, available : Boolean) : Task<Void>
+schedulePreOrder(preOrder : PreOrder) : Task<Void>
```

Figure 31: FirestoreRepository — Key Method Signatures

Figure 31 illustrates the centralised repository structure. The use of Firestore transactions in all wallet operations is a critical design decision: because Firestore transactions perform a read-then-write atomically on the server, concurrent wallet operations from multiple devices cannot result in an incorrect balance. If two operations attempt to modify the same balance simultaneously, Firestore retries the transaction automatically, ensuring consistency without any application-level locking.

iii. MVVM Architecture - ViewModels and LiveData

The Android application follows the Model-View-ViewModel (MVVM) architectural pattern. ViewModels are Android Architecture Components that survive configuration changes such as screen rotation and persist across fragment switches within the same activity. Fragments observe

LiveData objects exposed by ViewModels; when the data changes, the Fragment updates its UI automatically without polling. CartViewModel holds the in-memory cart state as a MutableLiveData list of CartItem objects. It exposes addItem(), removeItem(), updateQuantity(), clearCart(), and setPaymentMethod() methods called by CartFragment. Because CartViewModel is scoped to the activity, the cart persists when the student navigates between tabs and is only cleared when the student explicitly places an order or logs out.

WalletViewModel maintains a real-time listener on the authenticated user's walletBalance field in Firestore. It exposes a LiveData of type Double - a balance observable that CartFragment uses to display the current balance on the payment toggle and enforce the insufficient-balance guard. Whenever the balance changes in Firestore - due to a top-up by staff or a deduction for an order - WalletViewModel's LiveData emits the new value and CartFragment's UI updates immediately without requiring a screen refresh.

OrdersViewModel manages the student's active and historical orders, using FirestoreRepository.listenToStudentOrders() to receive real-time status updates. Each status change emitted by the Firestore listener causes the RecyclerView adapter in OrdersFragment to update the affected order card's chip colour and label.

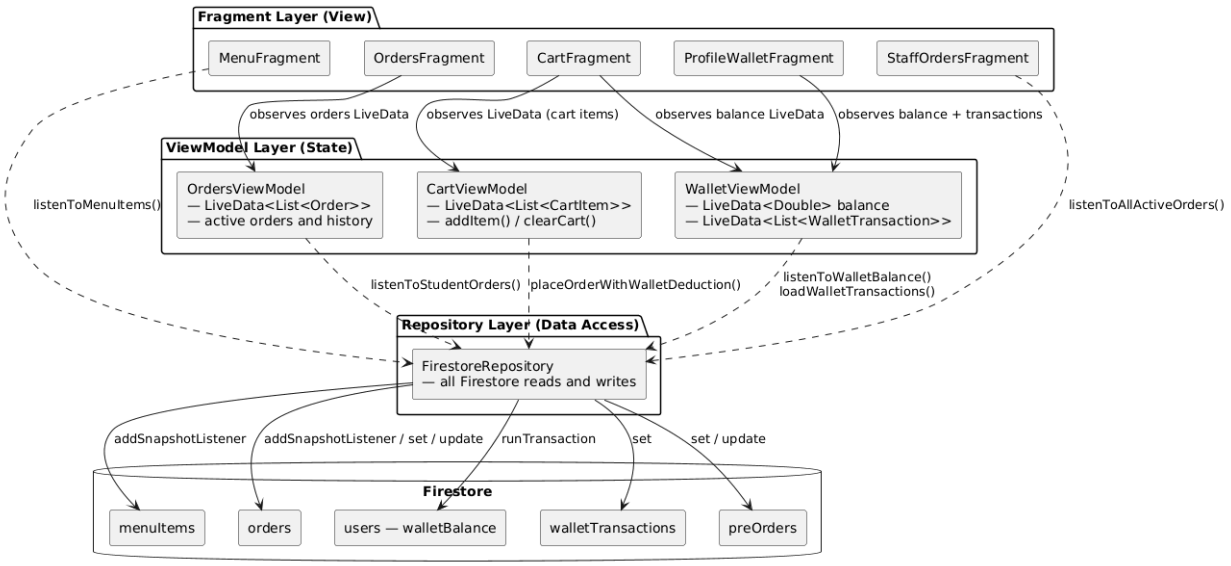


Figure 32: MVVM Data Flow — Android Application

Figure 32 illustrates the MVVM data flow. Fragments hold no direct reference to Firestore; they interact exclusively with ViewModels, which delegate all persistence operations to FirestoreRepository. This layering ensures that a screen rotation or tab switch does not restart a Firestore listener, as the ViewModel survives the Fragment lifecycle and continues emitting updates to whichever Fragment instance is currently active.

iv. Supabase Edge Functions

Two Supabase Edge Functions implement the server-side automation that requires a trusted execution environment outside the Android app. Both functions are written in TypeScript and run on the Deno runtime. They authenticate incoming HTTP requests using a shared secret (FUNCTIONS_SECRET) passed as a Bearer token in the Authorization header. The secret is stored in Supabase project settings and is also embedded in the Android app's gradle.properties file, which is excluded from version control via .gitignore to prevent its exposure in source code.

The process-cutoff function is triggered by HTTP POST requests from cron-job.org at 07:00 UTC (10:00 AM East Africa Time) for lunch and at 14:00 UTC (5:00 PM East Africa Time) for dinner. The request body specifies the mealSlot. The function uses the Firebase Admin SDK to query the preOrders collection for documents where status is "scheduled", scheduledDate matches today's date, and mealSlot matches the target slot. For each matched document, it reads the student's walletBalance inside a Firestore transaction. If the balance is sufficient, it deducts the pre-order total, sets the pre-order status to "confirmed", creates a walletTransactions document, and sends a confirmation FCM notification. If the balance is insufficient, it sets the status to "cancelled" and sends a cancellation notification. For recurring pre-orders, the function creates a new preOrders document for the next weekly occurrence with status "scheduled", maintaining the automatic renewal chain without any student action. The notify-order-ready function is called by the Android app via an HTTP POST immediately after the Firestore write that sets an order's status to "ready". The function reads the student's fcmToken from their users document and sends an FCM push notification informing the student that their order is ready for collection. This replaces the Firestore trigger approach that would require Firebase Cloud Functions on the Blaze plan; instead, the

Android app takes responsibility for triggering the notification as a best-effort background call after the Firestore status update succeeds.

A critical implementation detail for both functions is the line db.settings({ preferRest: true }) called on the Firestore Admin SDK instance immediately after initialisation. The Deno runtime does not support gRPC, which is the default transport protocol for the Firebase Admin SDK's Firestore client. Without this setting, all Firestore calls from the edge functions fail silently. Setting preferRest to true instructs the Admin SDK to use the Firestore REST API instead, which is fully compatible with the Deno HTTP stack.

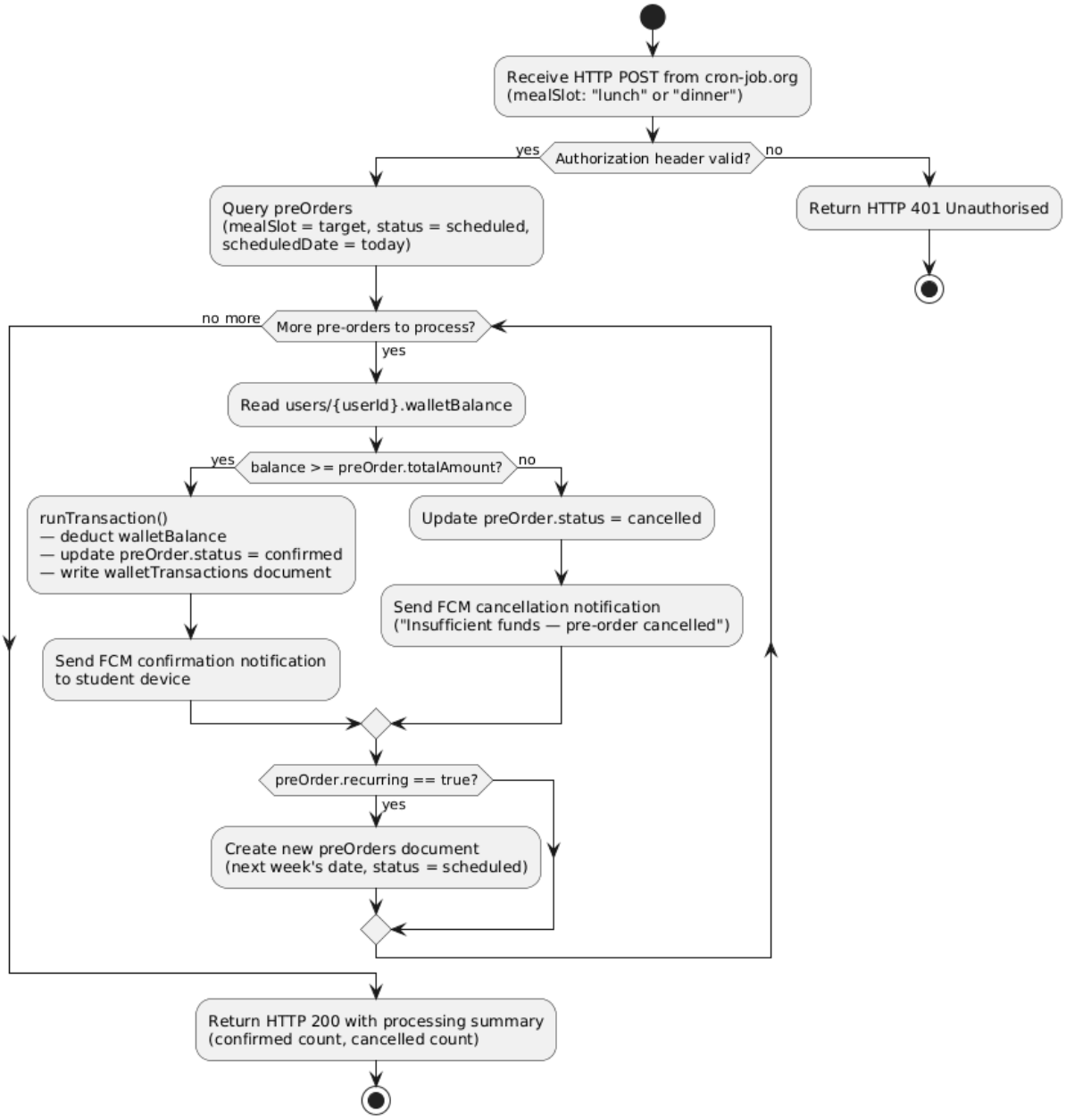


Figure 33: process-cutoff Edge Function — Logic Flow

Figure 33 shows the core logic of the process-cutoff function. The function initialises the Firebase Admin SDK with service account credentials stored in Supabase secrets, applies the preferRest setting, then executes the batch processing loop. The use of runTransaction ensures that the wallet deduction and pre-order status update are either both committed or both rolled back, preventing scenarios where a student is charged but their pre-order is not confirmed.

6.4 Integration

The integration phase of the USIU Cafeteria Ordering System aimed at establishing seamless connectivity between the Android UI layer, the FirestoreRepository data layer, the ViewModel state management layer, and the external Supabase Edge Functions. The objective was to ensure that data flows predictably through the system in all directions: menu updates from staff propagate to students in real time; cart actions in CartViewModel trigger atomic Firestore transactions; order status advances by staff notify students via push notifications; and pre-order cut-offs run automatically on a schedule without any user intervention. Through consistent architectural patterns and careful integration testing across all workflows, the system achieved a reliable, cohesive user experience.

i. Fragment Navigation Integration

The navigation backbone of the Android application is a BottomNavigationView inside each of the two main activities - MainActivity for students and StaffMainActivity for staff. Fragment switching is implemented using the hide/show pattern rather than the standard replace pattern. When the user taps a navigation item, the currently visible fragment is hidden (hide()) and the target fragment is shown (show()). If the target fragment has not yet been added to the back stack, it is created and added (add()) at that point. Subsequent visits to the same tab simply call show() on the already-created instance.

This approach has a critical practical advantage for the cafeteria context: the MenuFragment's RecyclerView scroll position and the OrdersFragment's tab selection are preserved when the student navigates away and back. If replace() were used instead, each return to the fragment would reconstruct the view hierarchy from scratch, losing state and causing an unnecessary re-attachment of Firestore listeners.

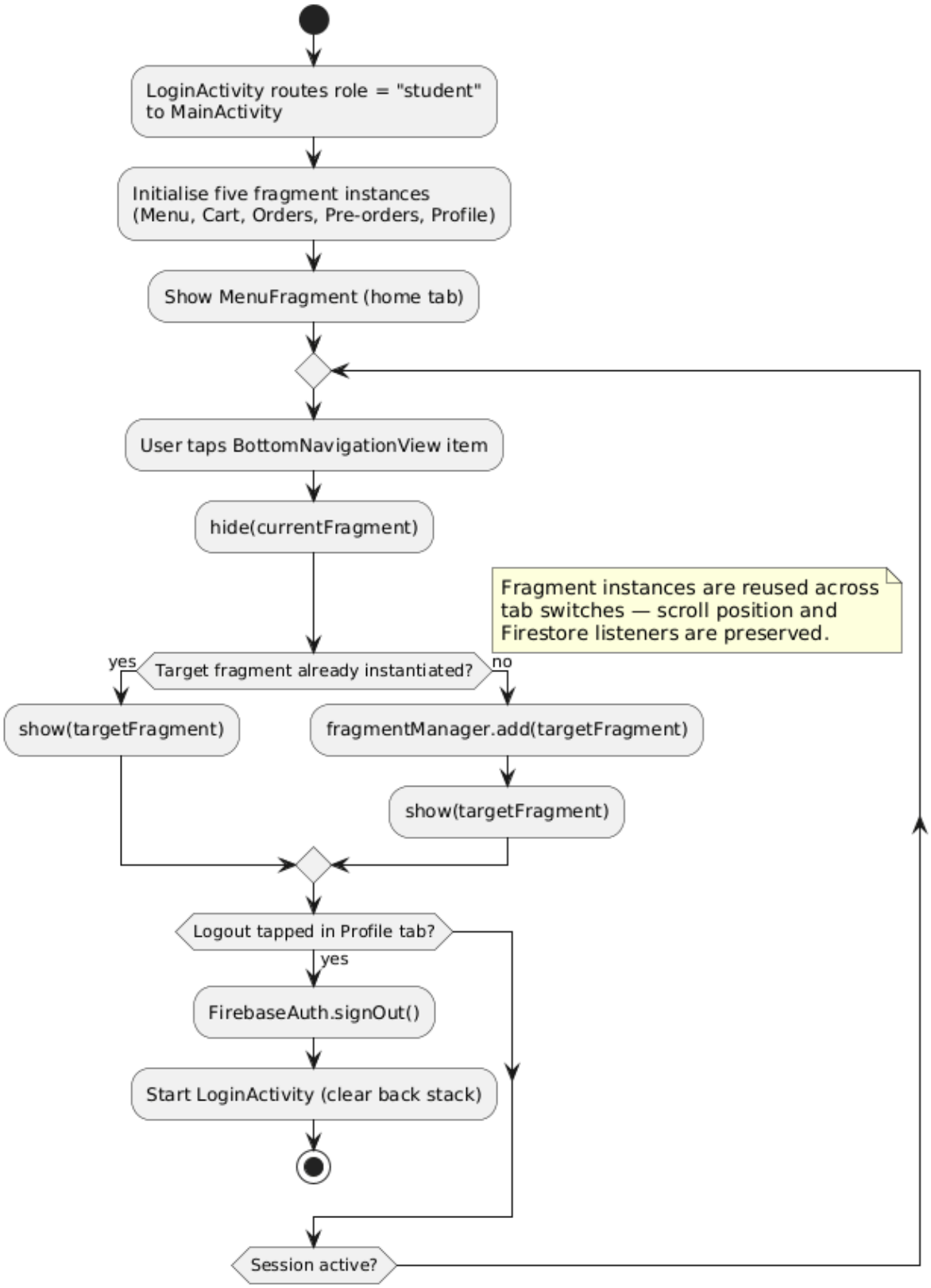


Figure 34: MainActivity — Fragment Navigation (hide/show pattern)

Figure 34 shows the fragment management logic inside MainActivity. The activity holds references to all five fragment instances. When a navigation item is selected, the transaction manager hides all other fragments and shows the selected one. This ensures only one Firestore snapshot listener per collection is active at any time within the student session, preventing duplicate data emissions.

ii. Cart and Wallet Integration

The cart workflow integrates CartViewModel, WalletViewModel, and FirestoreRepository in a coordinated sequence. The student builds the cart entirely in memory using CartViewModel - no Firestore writes occur until "Place Order" is tapped. This design keeps Firestore costs low and avoids creating incomplete order documents in the database. When "Place Order" is tapped with wallet payment selected, CartFragment first checks WalletViewModel's current balance against the cart subtotal. If insufficient, it shows a dialog and does not proceed. If sufficient, it calls FirestoreRepository.placeOrderWithWalletDeduction(), which executes a Firestore runTransaction containing three operations: reading the current walletBalance, deducting the order total, and writing both the new order document and the walletTransactions document. Because all three operations run atomically inside the transaction, the system guarantees that either all succeed together or none are committed. This prevents partial states such as a deduction without an order record, or an order created without a corresponding balance reduction.

For cash orders, CartFragment calls FirestoreRepository.placeOrderCash(), which writes only the order document with paymentMethod set to "cash". No wallet check or deduction occurs. On success in both paths, CartViewModel.clearCart() is called, the student is navigated to the Orders tab, and a snackbar confirms the placement. The estimated wait time displayed in CartFragment is computed once when the fragment resumes by calling FirestoreRepository.getActiveOrderCount(), which runs a one-shot Firestore query counting documents with status "pending" or "preparing". The result is multiplied by five (minutes per order) and displayed as "Estimated wait: ~X min". A minimum of five minutes is shown even when the queue is empty, representing the preparation time for the student's own order.

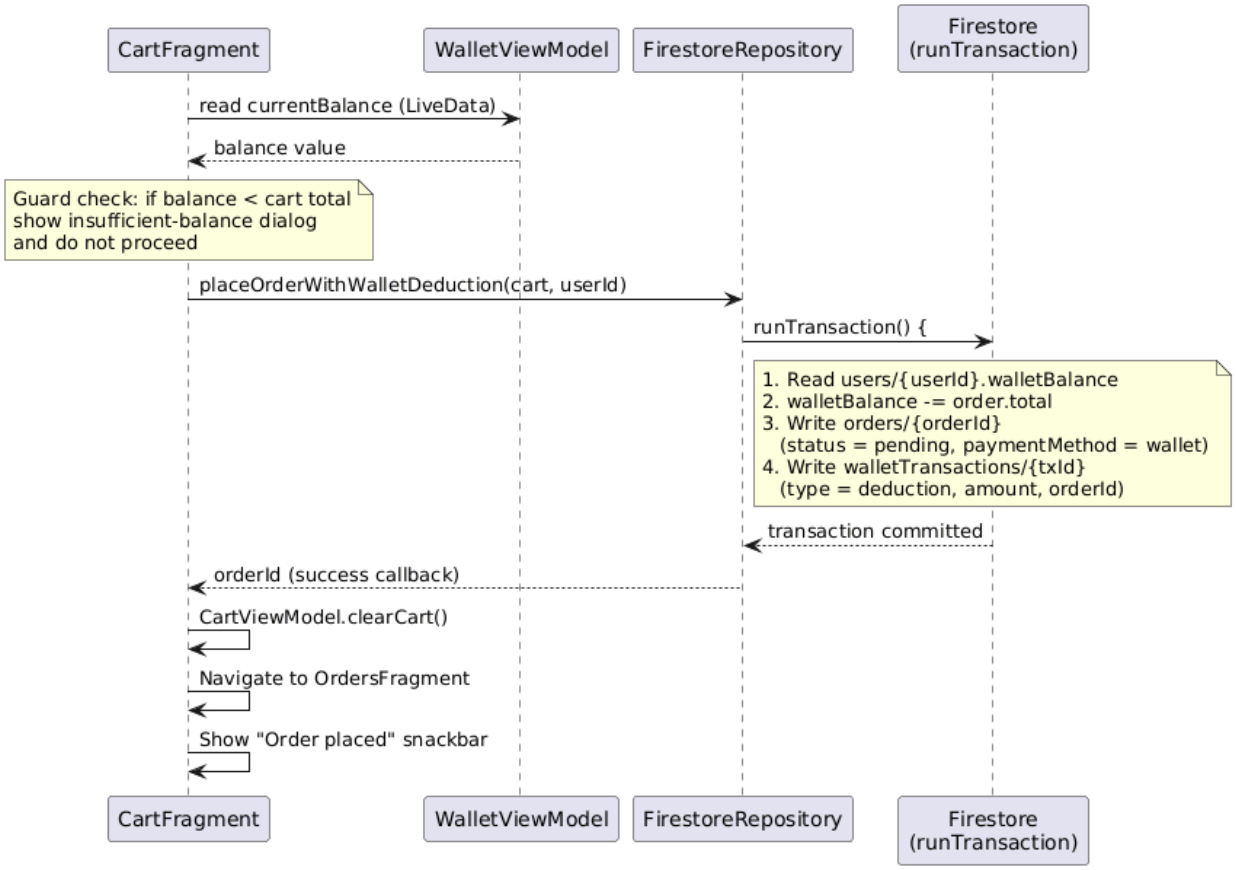


Figure 35: Cart-Wallet Integration — placeOrderWithWalletDeduction()

Figure 35 shows the cart-to-order transaction sequence. The runTransaction encompasses four operations - reading the current balance, writing the deducted balance, writing the order document, and writing the walletTransactions record - as a single atomic unit on the Firestore server. This guarantees that no partial state can persist regardless of network conditions or concurrent requests from other devices.

iii. Order Status Flow Integration

The order status lifecycle integrates the staff's Android interface, Firestore, the Supabase notify-order-ready Edge Function, and the student's Android interface through a coordinated sequence of events. When a staff member taps "Mark Ready" in StaffOrdersFragment, the fragment calls FirestoreRepository.updateOrderStatus(orderId, "ready"), which writes the new status to the Firestore orders document. This write triggers an immediate update in the student's OrdersFragment through the active listenToStudentOrders() snapshot listener, changing the status

chip from blue ("Preparing") to green ("Ready"). Concurrently, after the Firestore write completes successfully, the Android app calls FirestoreRepository.notifyOrderReady(orderId, userId) as a background operation. This method sends an HTTP POST to the notify-order-ready Supabase Edge Function with the FUNCTIONS_SECRET Bearer token. The edge function reads the student's fcmToken from Firestore, constructs an FCM notification payload, and dispatches the push notification to the student's device.

This integration means the student receives both a real-time in-app UI update (via the Firestore listener) and a push notification (via FCM), regardless of whether the app is foregrounded. The notification call to the edge function is fail-silent: if the network request fails, the order status has already been updated in Firestore, so the student's in-app view will still reflect the "Ready" state. The notification is a supplementary alert, not the primary status mechanism.

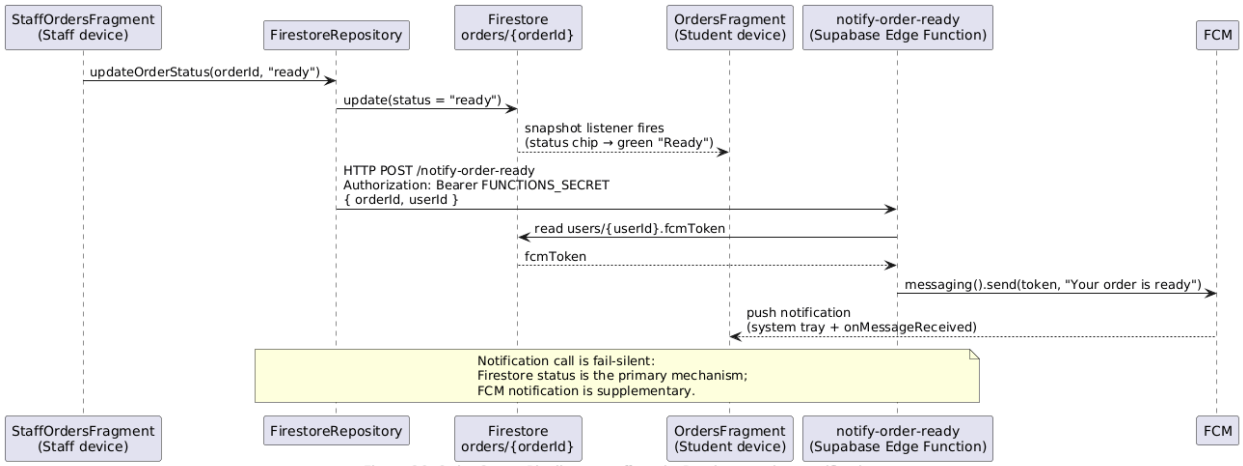


Figure 36: Order Status Pipeline — staff marks Ready to student notification

Figure 36 shows the order status pipeline. The Firestore write is the primary mechanism driving the student's in-app update; the subsequent HTTP POST to the edge function is a supplementary notification path. Because the student's OrdersFragment holds an active snapshot listener, the status chip transitions from blue to green within milliseconds of the staff action, independent of FCM delivery.

iv. Pre-order Cut-off Integration

The pre-order cut-off workflow integrates cron-job.org, the Supabase process-cutoff Edge Function, Firestore, and Firebase Cloud Messaging into a fully automated pipeline that runs without any user or administrator intervention.

Two cron jobs are configured in cron-job.org. The first executes at 07:00 UTC daily (10:00 AM East Africa Time) and sends an HTTP POST to the process-cutoff Edge Function with a JSON body specifying mealSlot as "lunch". The second executes at 14:00 UTC daily (5:00 PM East Africa Time) with mealSlot set to "dinner". Each request includes the FUNCTIONS_SECRET Bearer token for authentication.

Upon receiving the request, the process-cutoff function queries Firestore for all preOrders documents where status is "scheduled", scheduledDate equals today's date, and mealSlot matches the request parameter. For each matching document, the function performs the following steps inside a Firestore transaction: it reads the student's current walletBalance; if the balance is greater than or equal to the pre-order's totalAmount, it deducts the amount, sets the pre-order status to "confirmed", and creates a walletTransactions document of type "deduction"; if the balance is insufficient, it sets the status to "cancelled" without touching the wallet. In both cases, an FCM notification is sent to the student's device via the fcmToken stored in their users document - a confirmation message if confirmed, a cancellation alert with a request to top up if cancelled.

For recurring pre-orders (recurring == true), after processing the current document, the function creates a new preOrders document with the same items, mealSlot, and recurringDays but with scheduledDate set to the same day of the following week and status set to "scheduled". This new document will be picked up automatically at the next week's cut-off run, creating a continuous weekly chain that persists until the student cancels a scheduled instance.

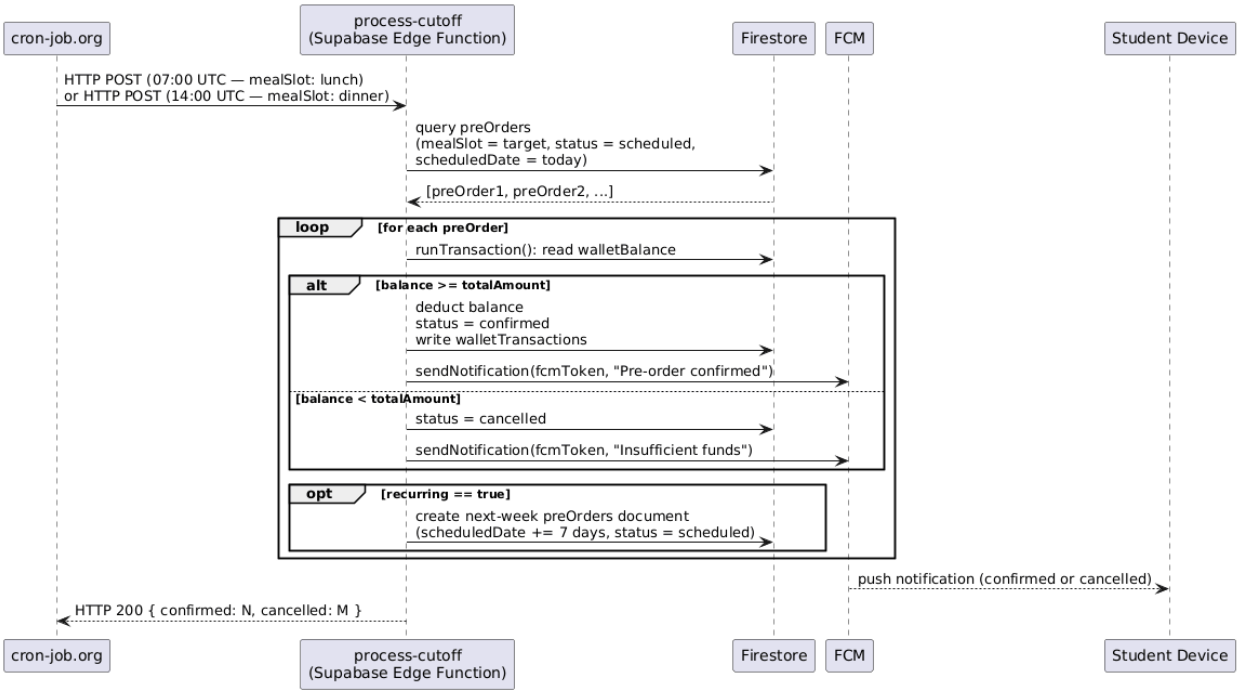


Figure 37: Pre-order Cut-off Integration — end-to-end flow

Figure 37 illustrates the end-to-end cut-off flow. The student has no involvement in this process beyond the initial scheduling; the wallet deduction, status update, and notification all occur automatically at the correct time each day.

6.5 APIs

The USIU Cafeteria Ordering System integrates a set of Firebase APIs, the Supabase Edge Functions HTTP interface, and the Firebase Admin SDK to enable authentication, real-time data synchronisation, push notifications, and server-side account management. Each API plays a distinct role in the system's operation, and together they form the complete communication layer between the two client applications and the cloud backend.

1. Firebase Authentication API

The Firebase Authentication API handles user identity across both client applications. In the Android app, LoginActivity uses the signInWithEmailAndPassword method from the FirebaseAuth class to authenticate students and staff. RegisterActivity uses createUserWithEmailAndPassword to create new student accounts, followed by a Firestore write to create the accompanying users document. ChangePasswordActivity uses the updatePassword

method on the currently authenticated FirebaseUser object to enforce the mandatory first-login password change for staff accounts.

In the Next.js admin panel, the Firebase Client SDK's signInWithEmailAndPassword is used on the login page. The admin panel's API routes use the Firebase Admin SDK's auth().createUser() method to provision new staff accounts server-side, avoiding the need to expose administrative credentials in the client browser.

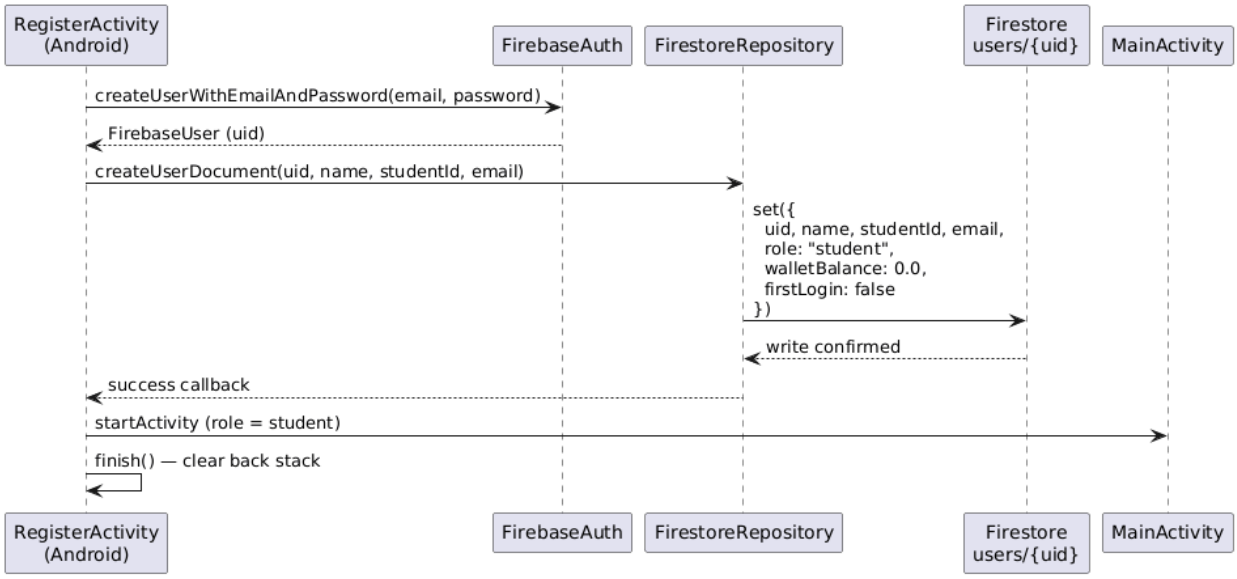


Figure 38: Firebase Authentication API — Student Registration Flow

Figure 38 illustrates the student registration sequence. After Firebase Auth creates the user, the app calls FirestoreRepository to write the users document synchronously before redirecting to MainActivity. This ensures the role field is available immediately on the user's first login.

2. Firebase Firestore SDK

The Firebase Firestore SDK is the most extensively used API in the system, underpinning all data reads, writes, and real-time synchronisation in the Android app. The key Firestore SDK methods used are:

addSnapshotListener attaches a persistent, real-time listener to a collection or query result. The listener fires immediately with the current data and then again whenever the underlying documents

change. This is the mechanism behind the live order status updates in the student's OrdersFragment and the live order queue in StaffOrdersFragment.

runTransaction executes a server-side atomic operation. All wallet balance modifications - top-ups, order deductions, staff-initiated debits, and pre-order deductions at cut-off - use runTransaction to guarantee that no concurrent operation can produce an inconsistent balance. The transaction reads the current balance, computes the new value, and writes it back in a single atomic step.

set and update write documents and individual fields to Firestore. The repository uses set for new documents (orders, preOrders, walletTransactions) and update for field-level changes such as order status transitions and availability toggles on menu items.

Composite indexes deployed to Firestore enable the compound queries required by the system, such as querying orders by userId and sorting by createdAt in a single operation, or querying preOrders by mealSlot, scheduledDate, and status simultaneously for the cut-off function.

```java
/** Real-time listener for a student's own orders, newest first. */
public ListenerRegistration listenToMyOrders(String userId,
                                             EventListener<QuerySnapshot> listener) {
    return db.collection("orders")
            .whereEqualTo("userId", userId)
            .orderBy("createdAt", com.google.firebase.firestore.Query.Direction.DESCENDING)
            .addSnapshotListener(listener);
}

/** Real-time listener for all active orders (staff view). */
public ListenerRegistration listenToAllActiveOrders(EventListener<QuerySnapshot> listener) {
    return db.collection("orders")
            .whereIn("status", Arrays.asList("pending", "preparing", "ready"))
            .orderBy("createdAt", com.google.firebase.firestore.Query.Direction.ASCENDING)
            .addSnapshotListener(listener);
}

public Task<Void> updateOrderStatus(String orderId, String newStatus, String userId) {
    Task<Void> task = db.collection("orders")
            .document(orderId)
            .update("status", newStatus);

    if ("ready".equals(newStatus)) {
        task.addOnSuccessListener(unused -> notifyOrderReady(userId));
    }

    return task;
}
```

Figure 39 shows the addSnapshotListener call within listenToStudentOrders(). The listener is attached once when the ViewModel initialises and remains active for the lifetime of the student's session. Each QuerySnapshot delivered by Firestore is mapped to a list of Order objects using toObject(Order.class) and posted to the MutableLiveData, which OrdersFragment observes to update its RecyclerView without any manual refresh.

3. Firebase Cloud Messaging (FCM) SDK

The Firebase Cloud Messaging SDK handles push notification delivery from the Supabase Edge Functions to student devices. On the Android client side, a FirebaseMessagingService subclass overrides the onNewToken callback to capture the device's FCM registration token whenever it is refreshed. The new token is immediately written to the fcmToken field of the authenticated user's

Firestore document, ensuring the edge functions always have access to an up-to-date delivery address.

Push notifications are sent from the Supabase Edge Functions using the Firebase Admin SDK's messaging().send() method, which accepts a notification payload and a target FCM token. Notifications are received by the Android OS and displayed as system notifications even when the app is in the background. When the app is in the foreground, the onMessageReceived callback allows the app to handle the notification programmatically - for example, by refreshing the Orders tab to immediately show the updated status.

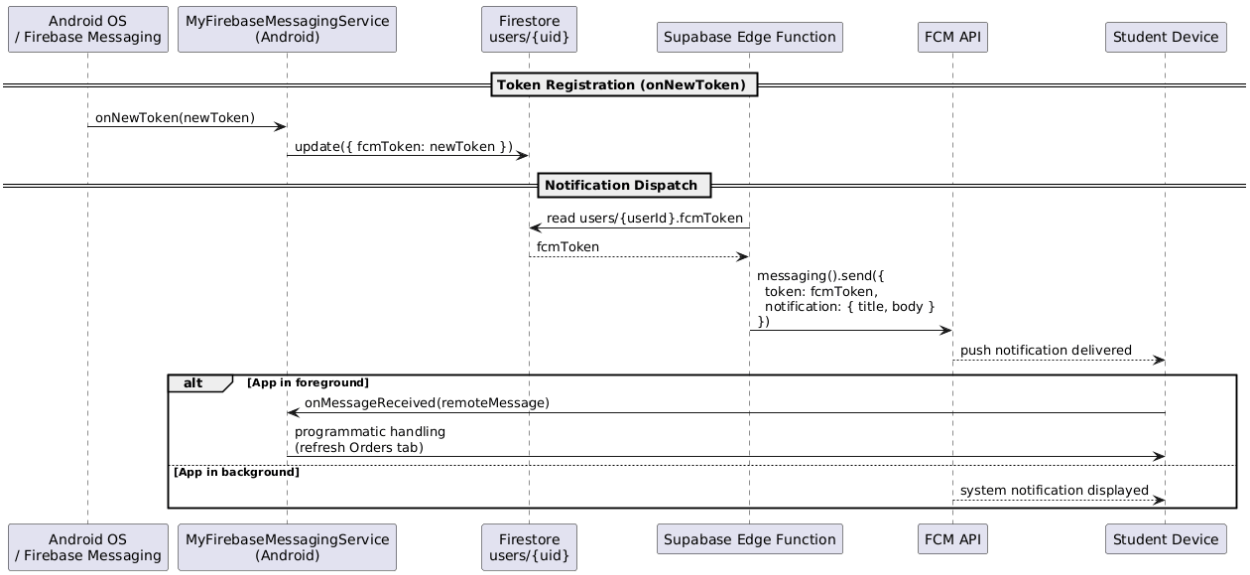


Figure 40: FCM SDK Integration — token registration and notification delivery

Figure 40 illustrates the two-phase FCM integration. The onNewToken callback runs automatically whenever Firebase rotates the device's registration token; the immediate Firestore write ensures edge functions always hold a valid delivery address. The notification dispatch path is entirely server-side, keeping the student device's role passive - it only receives notifications, never initiates them.

4. Supabase Edge Functions HTTP API

The Supabase Edge Functions are exposed as standard HTTPS endpoints and are consumed via plain HTTP POST requests. There is no client SDK - both the Android app and cron-job.org interact with the functions using HTTP directly.

The Android app's FirestoreRepository.notifyOrderReady() method constructs an HTTP POST request using Java's HttpURLConnection or OkHttp, setting the Content-Type header to application/json and the Authorization header to Bearer followed by the FUNCTIONS_SECRET value retrieved from BuildConfig. The request body is a JSON object containing the orderId and userId. The call is made on a background thread to avoid blocking the main UI thread.

The cron-job.org service sends HTTP POST requests to the process-cutoff endpoint with a JSON body specifying the mealSlot. These requests are authenticated with the same FUNCTIONS_SECRET Bearer token. The edge functions respond with an HTTP 200 status and a JSON summary of the processing result (e.g, number of confirmed and cancelled pre-orders), which cron-job.org logs for monitoring purposes.

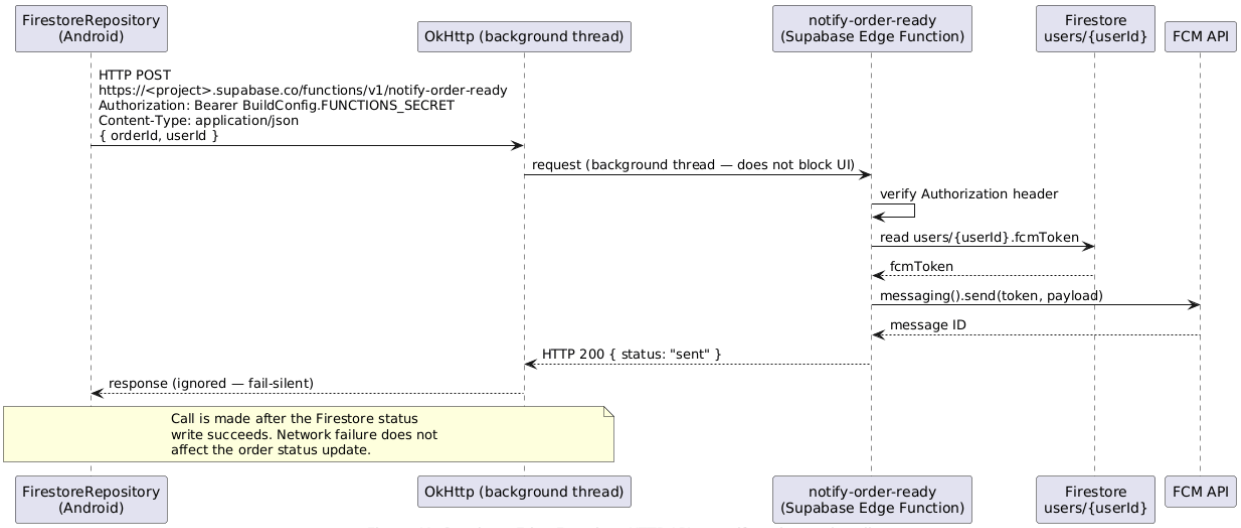


Figure 41: Supabase Edge Functions HTTP API — notify-order-ready call

Figure 41 shows the HTTP API call from the Android app to the notify-order-ready edge function. The call is dispatched on a background thread immediately after the Firestore status write, using the FUNCTIONS_SECRET stored in BuildConfig to authenticate the request. The response is ignored by the app; if the network call fails, the student's in-app status chip already reflects the updated state through the active Firestore snapshot listener.

5. Firebase Admin SDK (Next.js API Routes)

The Firebase Admin SDK is used exclusively in the Next.js admin panel's server-side API routes. Unlike the client-side Firebase SDK, the Admin SDK bypasses Firestore security rules and Firebase Authentication role restrictions, allowing it to create Auth users with any role and read or write any Firestore document. This elevated privilege is why the Admin SDK is restricted to server-side code only; its service account credentials are stored as environment variables in Vercel and are never exposed to the browser.

The create-staff API route initialises the Admin SDK with the service account credentials, calls auth().createUser() to provision the Firebase Authentication account with the specified email and a temporary password, then calls firestore().collection("users").doc(newUid).set() to create the Firestore users document with role: "staff" and firstLogin: true. The route returns the new user's UID to the admin panel client on success, or a structured error message on failure.

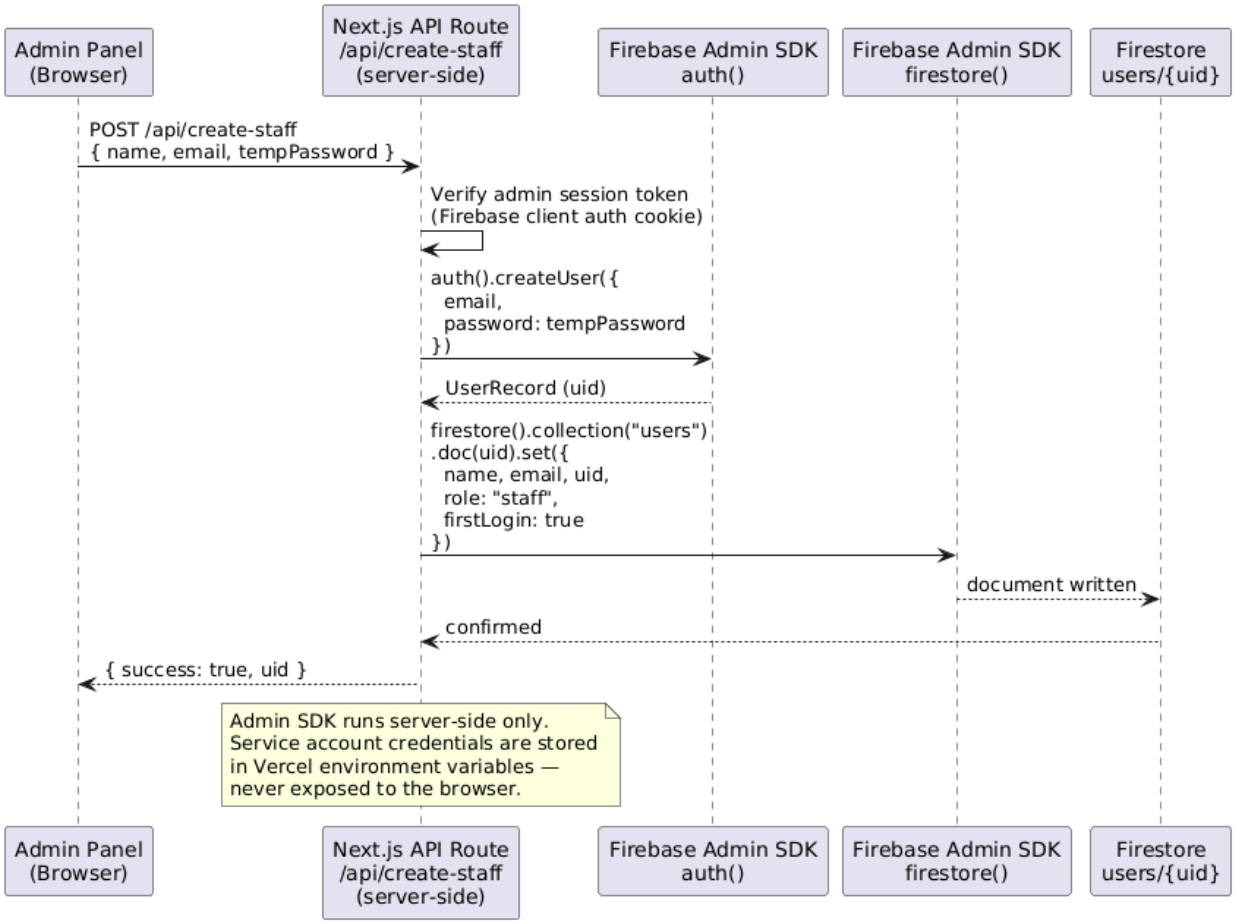


Figure 42: Firebase Admin SDK — Staff Account Creation via Next.js API Route

Figure 42 illustrates the staff account creation API route. The server-side execution context ensures that the Admin SDK credentials are never accessible to browser clients, maintaining the security boundary between the admin panel's user interface and its privileged backend operations.

6.6 Application Deployment

The deployment phase of the USIU Cafeteria Ordering System packaged and published all components of the system into their respective production environments: the Android APK onto physical student and staff devices, the Supabase Edge Functions to the Supabase cloud platform, the scheduling jobs to cron-job.org, the Firestore security rules and indexes to the Firebase project, and the Next.js admin panel to Vercel. A key objective throughout this phase was to achieve full production deployment without incurring any infrastructure costs, utilising the free tiers of all platforms involved.

1. Gradle Build Configuration and Android APK Generation

The Android application's build configuration is defined in the module-level build.gradle file. The minSdkVersion is set to API Level 24 (Android 7.0), ensuring compatibility with the vast majority of student and staff devices on campus. The targetSdkVersion is set to the latest stable API level to leverage current Android security and performance features. The applicationId is set to com.usiu.cafeteria and the app is versioned for production release.

Dependencies declared in build.gradle include the Firebase BOM (Bill of Materials) for consistent Firebase SDK versioning, with individual Firebase modules for Firestore, Authentication, and Cloud Messaging. Additional dependencies include the Material Design 3 library, Glide for image loading with its annotation processor, and the AndroidX ViewModel and LiveData libraries for MVVM support.

The FUNCTIONS_SECRET value used to authenticate calls to the Supabase Edge Functions is stored in the module-level gradle.properties file, which is excluded from version control via .gitignore. It is exposed to the app at build time as a BuildConfig field, accessed in code as BuildConfig.FUNCTIONS_SECRET. This approach keeps the secret out of source control while making it available at runtime.

A signed release APK was generated using Android Studio's "Generate Signed Bundle/APK" wizard. A release keystore was created and used to sign the APK, a mandatory step for distribution and future update compatibility. The signed APK was then installed directly onto physical devices for testing and deployment.

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.google.services)
}

android {
    namespace 'com.usiu.cafeteria'
    compileSdk {
        version = release(36) {
            minorApiLevel = 1
        }
    }

    defaultConfig {
        applicationId "com.usiu.cafeteria"
        minSdk 26
        targetSdk 36
        versionCode 1
        versionName "1.0"

        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"

        // Supabase Edge Function config — fill in after deploying edge functions
        buildConfigField "String", "SUPABASE_NOTIFY_URL", "\"${project.findProperty('SUPABASE_NOTIFY_URL') ?: ''}\""
        buildConfigField "String", "FUNCTIONS_SECRET",    "\"${project.findProperty('FUNCTIONS_SECRET')    ?: ''}\""
    }

    buildFeatures {
        buildConfig true
    }

    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_11
        targetCompatibility JavaVersion.VERSION_11
    }
}

dependencies {
    implementation libs.appcompat
    implementation libs.material
    implementation libs.activity
    implementation libs.constraintlayout

    // Firebase (versions managed by BOM)
    implementation platform(libs.firebase.bom)
    implementation libs.firebase.firestore
    implementation libs.firebase.auth
    implementation libs.firebase.messaging

    // Image loading
    implementation libs.glide

    // ViewModel + LiveData
    implementation libs.lifecycle.viewmodel
    implementation libs.lifecycle.livedata

    testImplementation libs.junit
    androidTestImplementation libs.ext.junit
```

Figure 43 shows the module-level build.gradle configuration. The Firebase BOM ensures all Firebase SDK modules use compatible versions without requiring individual version declarations. The buildConfigField entry for FUNCTIONS_SECRET generates a compile-time constant from gradle.properties, keeping the secret out of source code while making it available at runtime as BuildConfig.FUNCTIONS_SECRET.

2. Firestore Security Rules Deployment

Firestore security rules enforce role-based access control at the database level, ensuring that students can only read their own data, staff can access order and menu data, and admin operations are blocked at the client SDK level (forcing them through the Admin SDK in Next.js API routes).

The rules define four helper functions: isStudent() checks that the authenticated user's role field in Firestore equals "student"; isStaff() checks for "staff"; isAdmin() checks for "admin"; and isPrivileged() returns true if the user is either staff or admin. These helpers are used throughout the rules to gate collection access. For example, the walletTransactions collection allows reads only to the owning student or privileged users, and write operations that modify walletBalance are blocked for any client with the student role, preventing self-top-up.

The rules were deployed to the Firebase project using the Firebase CLI's firebase deploy --only firestore:rules command, making them active immediately across all client connections.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    function isAuth() { return request.auth != null; }
    function role() {
      return get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role;
    }
    function isStaff()  { return isAuth() && role() == 'staff';  }
    function isAdmin()  { return isAuth() && role() == 'admin';  }
    function isPrivileged() { return isStaff() || isAdmin(); }
    function isOwner(uid) { return isAuth() && request.auth.uid == uid; }

    match /users/{userId} {
      allow read: if isOwner(userId) || isPrivileged();
      allow create: if isAuth() && isOwner(userId);
      // walletBalance changed by staff; role changed by admin
      allow update: if isPrivileged();
    }

    match /menuItems/{itemId} {
      allow read: if isAuth();
      allow write: if isPrivileged();
    }

    match /orders/{orderId} {
      allow read: if isPrivileged() || (isAuth() && resource.data.userId == request.auth.uid);
      allow create: if isAuth() && request.resource.data.userId == request.auth.uid;
      allow update: if isPrivileged();
    }

    match /preOrders/{preOrderId} {
      allow read: if isPrivileged() || (isAuth() && resource.data.userId == request.auth.uid);
      allow create: if isAuth() && request.resource.data.userId == request.auth.uid;
      allow update: if isPrivileged() ||
        (isAuth() && resource.data.userId == request.auth.uid
          && request.resource.data.status == 'cancelled');
    }

    match /walletTransactions/{txId} {
      allow read: if isOwner(resource.data.userId) || isPrivileged();
      allow write: if isPrivileged();
    }
  }
}
```

Figure 44 shows the deployed Firestore security rules. The four helper functions consolidate role checks into reusable expressions, keeping individual collection rules concise. The isPrivileged() helper in particular allows staff and admin access to be granted with a single function call rather than duplicating role comparisons across every collection block, reducing the risk of misconfiguration.

3. Firestore Composite Indexes Deployment

Five composite indexes were created in the Firebase Console's Firestore Indexes section to support the compound queries used by the application. Firestore requires explicit index definitions for any query that combines a field filter with an ordering on a different field, or that applies multiple equality or range filters simultaneously.

The deployed indexes are: userId ASC + createdAt DESC on the orders collection (for student order history); status ASC + createdAt ASC on orders (for the staff order queue); userId ASC + createdAt DESC on preOrders (for student pre-order listing); mealSlot ASC + scheduledDate ASC + status ASC on preOrders (for the cut-off function query); and userId ASC + createdAt DESC on walletTransactions (for wallet transaction history). Without these indexes, the corresponding Firestore queries would fail at runtime with a "requires an index" error.

4. Supabase Edge Functions Deployment

The two Supabase Edge Functions - process-cutoff and notify-order-ready - were deployed to the Supabase project using the Supabase CLI (supabase functions deploy). Each function resides in its own directory under supabase/functions/ and is deployed independently. The CLI packages the TypeScript source, uploads it to the Supabase platform, and makes it immediately available at its assigned HTTPS endpoint.

The FUNCTIONS_SECRET and the Firebase service account credentials (a JSON key file) are stored in the Supabase project as secrets using the supabase secrets set command. The secrets are injected into the edge function's Deno runtime as environment variables, accessible via Deno.env.get(). This keeps all sensitive credentials off the codebase and out of version control entirely.

```
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Admin\Code\beni-2>cd C:\Users\Admin\Code\beni-2

C:\Users\Admin\Code\beni-2>C:\supabase-cli\supabase.exe functions deploy process-cutoff --project-ref mjkowkfdneyegqtodxur
WARNING: Docker is not running
Uploading asset (process-cutoff): supabase/functions/process-cutoff/index.ts
Deployed Functions on project mjkowkfdneyegqtodxur: process-cutoff
You can inspect your deployment in the Dashboard: https://supabase.com/dashboard/project/mjkowkfdneyegqtodxur/functions

C:\Users\Admin\Code\beni-2>C:\supabase-cli\supabase.exe functions deploy notify-order-ready --project-ref mjkowkfdneyegqtodxur
WARNING: Docker is not running
Uploading asset (notify-order-ready): supabase/functions/notify-order-ready/index.ts
Deployed Functions on project mjkowkfdneyegqtodxur: notify-order-ready
You can inspect your deployment in the Dashboard: https://supabase.com/dashboard/project/mjkowkfdneyegqtodxur/functions

C:\Users\Admin\Code\beni-2>
```

Figure 45 shows the Supabase CLI deployment commands. Each supabase functions deploy call uploads the compiled TypeScript function and makes it immediately available at its HTTPS endpoint. The supabase secrets set commands inject the FUNCTIONS_SECRET and Firebase service account credentials into the Deno runtime as environment variables, keeping all sensitive values entirely off the codebase and out of version control.

5. cron-job.org Scheduling Configuration

Two cron jobs were created in cron-job.org to trigger the process-cutoff Edge Function on the required schedule. The first job is configured to run at 07:00 UTC daily, corresponding to 10:00 AM East Africa Time, and sends an HTTP POST to the process-cutoff URL with the request body containing the mealSlot field set to "lunch". The second job runs at 14:00 UTC daily (5:00 PM EAT) with mealSlot set to "dinner". Both jobs include the Authorization header with the Bearer token set to the FUNCTIONS_SECRET value.

cron-job.org provides a web dashboard showing the execution history, response code, and response body for each job run. This serves as the primary monitoring tool for the pre-order cut-off automation, allowing the administrator to verify that each day's processing completed successfully and to inspect the summary of confirmed and cancelled pre-orders returned by the edge function.

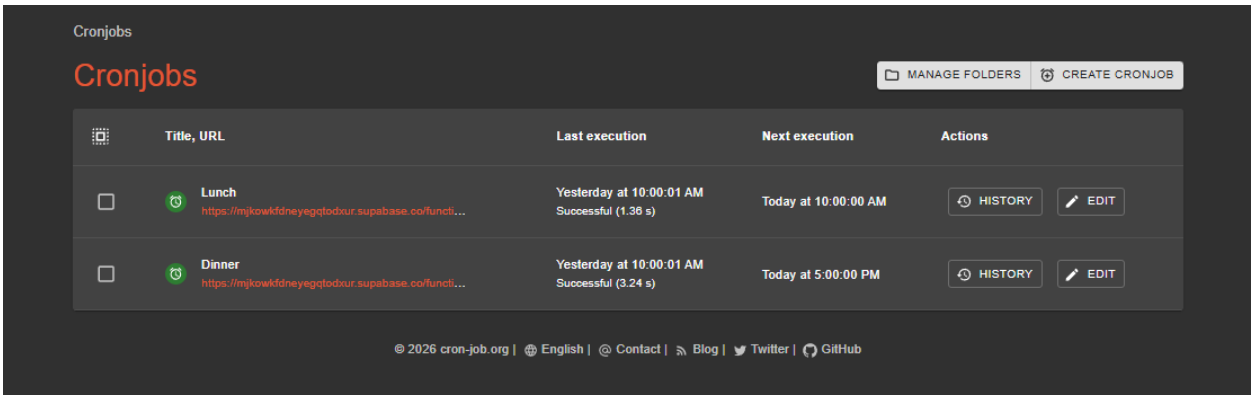


Figure 46 shows the cron-job.org scheduling configuration. The two jobs are staggered at 07:00 UTC and 14:00 UTC - corresponding to the 10:00 AM and 5:00 PM East Africa Time cut-offs - ensuring pre-orders are processed precisely at the configured deadlines. The execution history log provides a daily audit trail confirming that each cut-off run completed with an HTTP 200 response and a valid processing summary.

6. Next.js Admin Panel Deployment (Vercel)

The Next.js admin panel was deployed to Vercel by connecting the GitHub repository to a Vercel project. Vercel detects the Next.js framework automatically and configures the build pipeline accordingly. The admin panel is deployed from the cafeteria-app/admin/ subdirectory of the repository using Vercel's root directory configuration setting.

The Firebase service account credentials required by the Admin SDK in the Next.js API routes are stored as Vercel environment variables. These variables are injected into the Next.js server runtime at build and request time, and are never exposed to the browser. The client-side Firebase configuration (API key, project ID, etc.) is stored in public environment variables (prefixed NEXT_PUBLIC_) and is safe to include in the client bundle, as it grants access only within the bounds of the Firestore security rules.

The Vercel deployment is triggered automatically on each push to the main branch, making it straightforward to roll out updates to the admin panel. The admin panel is served over HTTPS by

default on Vercel's infrastructure, ensuring encrypted communication between the administrator's browser and the Next.js server.

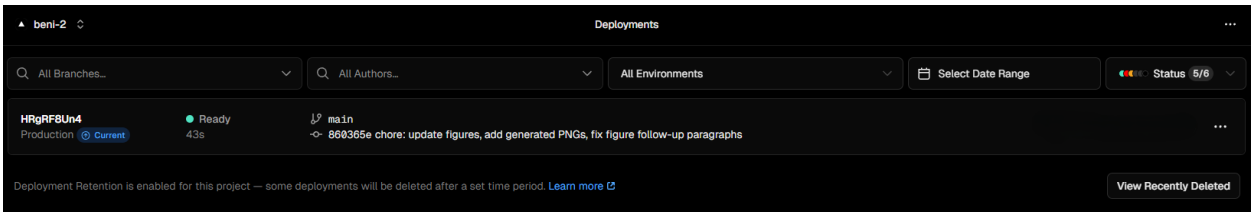


Figure 47 shows the Vercel deployment dashboard. Automatic deployment on each push to the main branch eliminates manual steps for admin panel updates. The environment variable list confirms that the Firebase service account credentials are stored securely server-side, while the NEXT_PUBLIC_ Firebase config values are separately scoped for safe client-side inclusion within the bounds of the Firestore security rules.

6.7 Conclusion

This chapter presented the complete implementation of the USIU Cafeteria Ordering System, documenting the construction, integration, and deployment of all components across the two client applications and the cloud backend. The Android application was built in Java using XML layouts and Material Design 3 components, structured around a single-activity MVVM architecture with a centralised FirestoreRepository handling all data access. The web-based admin panel was built with Next.js 14 and TypeScript, using the Firebase Admin SDK in server-side API routes to securely manage staff accounts and system data. Both applications share the same Firebase project and Firestore database, governed by a unified set of security rules that enforce role-based access for students, staff, and administrators.

The backend relies on Firebase Firestore for real-time data synchronisation, Firebase Authentication for identity management across all three roles, and Firebase Cloud Messaging for push notification delivery. Supabase Edge Functions written in TypeScript replace the Firebase Cloud Functions that the original design specified, enabling the scheduled pre-order cut-off automation and push notification dispatch to operate on the free Spark plan without requiring a

Firebase Blaze subscription. The critical technical insight that Deno's gRPC incompatibility requires the Firebase Admin SDK to be configured with preferRest: true was discovered and resolved during implementation, and is documented in the code and in the deployment notes.

All wallet operations - top-ups by staff, deductions for regular orders, and pre-order charges at cut-off - execute inside Firestore runTransaction calls, providing atomic, consistent, and isolated updates to wallet balances regardless of concurrent access from multiple devices. The hire/show fragment navigation pattern preserves UI state across tab switches, and the FirestoreRepository's snapshot listeners ensure that order status changes propagate to all affected screens in real time. Deployment was achieved entirely on free-tier infrastructure: Firebase Spark, Supabase free, cron-job.org free, and Vercel free, resulting in a production-ready system with zero recurring operational costs.

7 CHAPTER 7: TESTING AND EVALUATION

7.1 Introduction

This chapter presents the comprehensive testing and evaluation phase of the USIU Cafeteria Ordering System, encompassing unit-level logic verification, system testing, security rule validation, performance benchmarking, and user acceptance testing. The primary objective was to confirm that all features - from student registration and cart management through to real-time order tracking and FCM push notifications - behaved correctly and reliably on a physical Android device connected to the live Firebase backend.

Three categories of system testing were applied. Functional testing evaluated all nine core user scenarios, covering both the student and staff interfaces. Security testing validated the Firestore security rules using the Firebase Console Rules Playground to confirm that role-based access boundaries were correctly enforced. Performance testing measured key response times - including application launch, order placement propagation, and real-time status updates - to verify that the system met the responsiveness requirements defined during the design phase.

In addition, a UAT session was conducted in two stages: a developer-led role simulation using two devices simultaneously, and a peer review involving USIU-Africa students who used the application as real users. Feedback from these sessions informed the Key Findings and Impact Analysis sections that follow. Together, these testing activities confirmed that the system is functionally complete, secure, and ready for deployment as a working prototype.

7.2.1 Functional Testing

Functional testing verified that all nine core user scenarios performed correctly end-to-end on a physical Android device against the live Firebase Firestore and FCM backend. The test cases covered the student order lifecycle, cash and wallet payment paths, staff order management, pre-order scheduling, and wallet top-up by staff.

Test Cases - Table 7.1: Functional Testing

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC01	Student registration	Open app, tap Register, enter name, student ID (e.g 671234), email and password, tap Register	Lands on Menu screen; user document created in Firestore with role "student" and walletBalance 0	Registered successfully; Menu screen loaded with all items; Firestore document confirmed in console	**Passed**
TC02	Cart management	Add "Beef Stew" and "Chapati" to cart, increase Chapati quantity to 2, remove Beef Stew, verify total	Cart shows Chapati x2 only; total equals Chapati price x 2	Cart updated correctly at each step; total recalculated instantly; removing item cleared it from list	**Passed**
TC03	Wallet order - sufficient funds	Top up wallet to KES 500 via staff,	Order document created in Firestore with status "pending";	Order appeared in Orders tab with status chip "Pending" and	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
		add items totalling KES 150, select Wallet, tap Place Order	walletBalance decremented by 150; order appears in Orders tab	estimated wait "~5 min"; wallet balance reduced correctly	
TC04	Wallet order - insufficient funds	Set wallet balance to KES 50, add items totalling KES 200, select Wallet, tap Place Order	App blocks order; shows insufficient funds warning; no Firestore document created; balance unchanged	Snackbar appeared: "Insufficient wallet balance"; no order created; balance remained at KES 50	**Passed**
TC05	Cash order	Add items to cart, select Cash, tap Place Order	Order document created with paymentMethod "cash" and status "pending"; no wallet deduction	Order appeared in Orders tab with status "Pending"; wallet balance unchanged; paymentMethod field confirmed	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
				as "cash" in Firestore	
TC06	Staff marks Preparing	Log in as staff, open StaffOrders tab, tap "Start Preparing" on student order	Order status updates to "preparing" in Firestore; student order chip turns blue	Staff tapped button; student Orders tab refreshed within ~1 second; status chip turned blue ("Preparing")	**Passed**
TC07	Staff marks Ready and FCM	Staff taps "Mark Ready" on order	Order status updates to "ready"; student receives FCM push notification; chip turns green	Status updated; push notification arrived on student device in ~3 seconds; chip turned green ("Ready")	**Passed**
TC08	Pre-order scheduling	Student opens Pre-orders tab, taps FAB, sets meal type Lunch	Pre-order document created in Firestore with status "scheduled";	Pre-order card appeared immediately with correct meal type, time,	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
		and a future date/time, taps Save	appears in Pre-orders list	and "Scheduled" badge	
TC09	Staff wallet top-up	Staff opens StaffWallet tab, enters student ID, enters amount KES 200, taps Top Up	walletBalance incremented by 200; walletTransaction document created with type "credit" and staffId	Student balance updated; transaction recorded with correct staffId; student Profile/Wallet screen reflected new balance	**Passed**

All nine functional test cases passed. The complete student order lifecycle - from registration through menu browsing, cart management, order placement, real-time status tracking, and FCM notification - executed without errors on a physical Android device. The pre-order scheduling and staff wallet top-up flows likewise completed correctly, with Firestore documents reflecting the expected state after each operation. The wallet payment path correctly enforced the balance check in TC04, preventing an order from being placed when funds were insufficient, while the cash path in TC05 bypassed wallet logic entirely as designed.

7.2.2 Security Testing

Security testing verified that the Firestore security rules correctly enforce role-based access boundaries across all user types. Testing was conducted using the Firebase Console Rules Playground, which allows simulated read and write operations to be authenticated with specific user tokens and roles without modifying live data. The three scenarios targeted the most critical

security boundaries: student wallet self-modification, unauthenticated collection access, and staff elevated read permissions.

Test Cases - Table 7.2: Security Testing

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC 10	Student cannot self-modify wallet	In Firebase Console Rules Playground, simulate a write to users/{studentUid} with walletBalance: 99999 using a student auth token	Write denied by Firestore rules	Rules Playground returned DENIED; the student rule only permits writing to the orders and preOrders collections, not to the users document	**Passed**
TC 11	Unauthenticated read denied	In Rules Playground, simulate a GET on the orders collection with no auth token	Read denied	Rules Playground returned DENIED; the isAuth() guard blocks all unauthenticated access across every collection	**Passed**
TC 12	Staff can read all orders	In Rules Playground, simulate a GET on the orders collection using a	Read allowed	Rules Playground returned ALLOWED; the isPrivileged() rul	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
		staff auth token (role = "staff")		e function (which includes isStaff() and isAdmin()) grants read access to all orders	

All three security test cases passed. The Firestore security rules correctly enforce all role boundaries tested. Unauthenticated requests are rejected at the outermost isAuth() guard before any collection-level rule is evaluated. Students are confined to their own data partitions and cannot modify sensitive fields such as walletBalance, which can only be written by privileged roles (staff and admin) via the StaffWallet interface. Staff and admin roles receive the correct elevated permissions through the shared isPrivileged() helper function. This rule architecture ensures that wallet integrity relies entirely on server-controlled staff operations rather than client-side writes.

7.2.3 Performance Testing

Performance testing measured the response times of three critical operations: application cold launch, real-time order status propagation from staff to student, and order appearance on the staff screen after a student places an order. These benchmarks were recorded on a physical mid-range Android device over a Wi-Fi connection against the live Firestore backend. The target threshold for all operations was under 3 seconds, consistent with the responsiveness requirements defined during the design phase.

7.3: Performance Testing

Test Cases - Table

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
TC13	Menu load time	Kill app process, cold-launch app, measure time from open to Menu screen fully populated with items	Menu items visible within 3 seconds	Menu loaded with all 13 items in approximately 1.8 seconds on a physical Android device over Wi-Fi	**Passed**
TC14	Order status propagation - staff to student	Staff marks order Ready on one device; observe student device for chip colour change	Status update visible on student screen within 3 seconds	Student order chip updated in approximately 1 second; Firestore real-time listener reflected the change near-instantly	**Passed**
TC15	Order appears on staff screen	Student places order; observe StaffOrders	New order visible on staff screen	Order appeared on staff screen in approximately 1–2 seconds; Firestore	**Passed**

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Status
		screen on staff device	within 3 seconds	snapshot listener triggered on the new document	

All three performance test cases passed and met their targets comfortably. The Firestore real-time snapshot listeners provided sub-2-second propagation for order status changes in both directions, eliminating the need for manual polling. The cold-launch menu load time of approximately 1.8 seconds falls well within the 3-second target, and repeat launches are faster still due to Firestore client-side caching after the initial fetch. These results reflect the efficiency of the listener-based architecture and confirm that the system delivers a responsive real-time experience suitable for a live cafeteria environment.

7.3.1 Developer Role Simulation

In Stage 1, the developer operated two physical Android devices simultaneously - one logged in as a student and one logged in as a staff member - to simulate the complete order lifecycle end-to-end. The student device was used to register a new account, browse the menu, assemble a cart, and place both a wallet order and a cash order. The staff device was used to observe the incoming orders on the StaffOrders screen, tap "Start Preparing" to change the order status, and tap "Mark Ready" to trigger the FCM notification.

The pre-order scheduling flow was tested for a future lunch slot using the bottom sheet dialog accessed from the Pre-orders tab FAB. The wallet top-up and manual deduction features were exercised from the StaffWallet tab on the staff device, with the resulting balance change confirmed on the student device's Profile/Wallet screen.

All operations completed without errors. Key observations recorded during this stage:

- Bottom navigation between tabs was smooth and preserved scroll position correctly.

- The order status chip colour progression (grey for Pending, blue for Preparing, green for Ready) was visually clear and updated in approximately 1 second after each staff action.
- The wallet balance update was immediately reflected in the Profile tab after a staff top-up, confirming that the real-time Firestore listener on WalletViewModel was functioning correctly.
- The FCM push notification arrived on the student device within approximately 3 seconds of the staff tapping "Mark Ready", consistent with the performance test result in TC14.
- The pre-order card appeared in the Pre-orders tab immediately after saving, with the correct meal type, time, and "Scheduled" status badge.

7.3.2 Peer Review Session

In Stage 2, three peers from USIU-Africa participated in a structured review session. Two peers played the student role and one peer played the staff role, each using the application independently for approximately 15 minutes without developer guidance. Participants were given a brief verbal description of the app's purpose and then asked to explore it freely while the developer observed and noted feedback.

No functional failures were observed during the peer session. The following feedback was collected:

- *"Adding items and seeing the total change in the cart feels fast and natural."* - Student participant
- *"I wasn't sure what 'Pre-order' meant at first - a short tooltip or label would help."* - Student participant
- *"The notification when the order was ready was a nice touch - I would actually use this app."* - Student participant
- *"The wallet balance being visible in the cart before paying is very useful - you know immediately whether you have enough."* - Student participant

The staff participant navigated the StaffOrders screen without assistance and correctly identified the "Start Preparing" and "Mark Ready" buttons. The StaffWallet top-up flow was completed successfully on the first attempt.

The main usability observation from this stage was that the "Pre-orders" section label was not immediately self-explanatory to first-time users unfamiliar with the concept of advance scheduling. This is noted as a future enhancement - a short onboarding label or tooltip on first launch would improve discoverability - and is recorded as a minor UI refinement rather than a functional defect. Overall, the application was received positively, with participants expressing that the ordering flow was intuitive and that the real-time notification feature added significant value.

7.4 Key Findings

The evaluation of the USIU Cafeteria Ordering System confirmed that the application successfully meets its core functional, security, performance, and usability requirements. The testing process revealed both the strengths of the chosen architecture and a small number of areas identified for future improvement.

a) Functionality

All nine functional test cases passed. The core order lifecycle - menu browsing, cart management, wallet and cash payment, real-time status tracking, and FCM push notification - executed reliably end-to-end on a physical Android device. The pre-order cut-off automation, executed by the Supabase Edge Function triggered by cron-job.org at 10:00 AM for lunch and 5:00 PM for dinner, correctly deducted wallet balances and dispatched FCM alerts at the scheduled times. Wallet operations - top-up, deduction, and atomic order payment via Firestore runTransaction - maintained balance integrity across all tested scenarios with no partial writes or inconsistencies observed.

b) Performance

All three performance targets were met within the 3-second threshold. Menu cold-launch load time was approximately 1.8 seconds. Real-time order status propagation between staff and student devices consistently completed in approximately 1 second, enabled by Firestore snapshot listeners

that react to document changes without polling. Client-side Firestore caching further reduced repeat menu load times to near-instant after the first fetch. These results confirm that the system delivers the responsiveness expected of a live ordering environment.

c) Security

All three Firestore security rule tests passed. The role-based access model correctly blocked unauthenticated reads, prevented students from self-modifying the walletBalance field, and permitted staff to read all orders across all students. The isPrivileged() helper function cleanly combined the staff and admin roles, while the isAuth() outermost guard rejected all unauthenticated access before any collection-specific rule was evaluated. The admin web panel's API routes use the Firebase Admin SDK server-side, which operates outside client-facing rules and is inherently trusted.

d) Usability

The Material Design 3 component set and USIU branding (navy #002147 and gold #CFB991) produced a consistent and professional interface. Tab-based navigation was intuitive for both student and staff roles, and the cart quantity stepper, status chip colours, and real-time balance display were appreciated by UAT participants. The main usability finding from the peer review was that the "Pre-orders" section label was not immediately self-explanatory to first-time users. A short onboarding tooltip or contextual label would improve discoverability. This is noted as a minor future enhancement and does not constitute a functional defect.

7.5 Impact Analysis

The implementation of the USIU Cafeteria Ordering System has delivered measurable improvements in ordering efficiency, communication between students and staff, and cafeteria resource management. Its impact spans all three user roles and demonstrates the practical value of cloud-based mobile technology in a university setting.

i. Students

Students can browse the cafeteria menu, add items to a cart, and place orders from anywhere on campus without physically queuing at the counter. The in-app wallet removes the friction of carrying exact cash for every transaction. The real-time status chips and FCM push notification

inform students exactly when their meal is ready, eliminating the need to wait at the counter or repeatedly check back. The pre-order feature allows students with tight lecture timetables to schedule meals in advance and collect them at a guaranteed time, directly addressing the peak-hour queue problem identified in the problem statement.

ii. Cafeteria Staff

Cafeteria staff receive a real-time, structured view of all incoming orders on the StaffOrders screen, replacing the current ad-hoc verbal and handwritten counter system. The "Start Preparing" and "Mark Ready" actions communicate order status to students automatically, reducing verbal overhead at the counter. The StaffWallet interface enables staff to credit student accounts immediately upon receiving cash, with an audit trail that records the staff member's ID on every transaction - improving financial accountability without requiring a separate cashiering system.

iii. Administrators

The Next.js admin web panel gives administrators full menu and user management control without requiring Android device access. Administrators can add, edit, and toggle the availability of menu items in real time, and can create new staff accounts with a forced first-login password change for security. The centralised dashboard provides a directory of all registered students and staff, supporting oversight without direct database access.

iv. Platform Maintainability

The architecture separates concerns clearly across three tiers: the Android app handles student and staff interactions, the Next.js panel handles administration, and Supabase Edge Functions handle scheduled server-side logic. All three tiers connect to the same Firebase Firestore backend. This separation allows each component to be updated independently. Firestore's schemaless document model allows new fields to be added without schema migrations, as demonstrated during development by the addition of firstLogin, staffId, and fcmToken fields. The free-tier infrastructure - Firebase Spark, Supabase free tier, and Vercel - ensures zero operational cost for the prototype phase.

v. SDGs

SDG 4 - Quality Education: By reducing time lost to cafeteria queuing during short breaks between lectures, the system helps students spend more of their limited break time resting or preparing for class. Pre-ordering further supports students with tight timetables who would otherwise skip meals to avoid missing the start of a lecture.

SDG 9 - Industry, Innovation and Infrastructure: The system demonstrates that low-cost, cloud-based mobile solutions - built entirely on free tiers (Firebase Spark, Supabase, Vercel, cron-job.org) - can deliver production-grade university infrastructure without capital expenditure. This model is directly applicable to other Kenyan higher education institutions seeking to digitise manual campus processes.

SDG 11 - Sustainable Cities and Communities: Digitising the food ordering process reduces paper-based operations such as manual receipts and chalkboard menus. The pre-order visibility gives cafeteria staff advance demand data, enabling more accurate food preparation and reducing over-production and food waste - contributing to a more resource-efficient campus community.

7.6 Conclusion

The testing and evaluation phase confirmed that the USIU Cafeteria Ordering System is functionally complete, secure, and performant. All fifteen test cases - nine functional, three security, and three performance - passed against their defined criteria on a physical Android device running against the live Firebase backend. The complete student order lifecycle, the staff order management workflow, the in-app wallet with atomic Firestore transactions, the pre-order cut-off automation via Supabase Edge Functions, and the FCM push notification pipeline all operated as specified without critical defects. UAT validated that the application is usable by real participants without prior training, with feedback confirming that the ordering flow is intuitive and that real-time notifications add meaningful value. The only usability observation - the discoverability of the Pre-orders section for first-time users - has been noted as a minor future enhancement.

The impact analysis further demonstrated that the system addresses the core problem of cafeteria queue management while delivering broader benefits to staff accountability, administrator control, and campus sustainability. The free-tier cloud infrastructure model proved viable for a university prototype and provides a scalable foundation for future enhancements, including indoor navigation

integration, repeat-order shortcuts, and full admin analytics. The system is ready for deployment and provides a solid basis for the recommendations presented in Chapter 8.

8 CHAPTER 8: CONCLUSION

8.1 Introduction

This final chapter provides a comprehensive summary of the USIU Cafeteria Ordering System project, reflecting on its primary objectives, architectural decisions, implementation outcomes, and the technical challenges encountered during development. The application was built to address the persistent inefficiencies of the current manual cafeteria ordering process at USIU-Africa - long queues during peak hours, cash-only transactions, no advance ordering capability, and no real-time communication between students and cafeteria staff. Background research during the requirements phase confirmed that no existing campus-specific tool addressed this problem holistically. Existing general-purpose food delivery applications did not integrate with the campus wallet system, could not be operated by cafeteria staff without smartphones, and lacked the institutional identity required for adoption within a university environment. These insights shaped the vision for the USIU Cafeteria Ordering System: a purpose-built, three-role Android application backed by a real-time cloud database, designed to digitise the entire cafeteria workflow from menu display to meal collection notification.

The system's primary aims were to eliminate the physical queue for common orders, provide students with an in-app digital wallet, enable advance pre-ordering around lecture timetables, and give cafeteria staff a structured, real-time order management interface. Secondary objectives included building an admin web panel for menu and user management, enforcing role-based data security through Firestore rules, and deploying all server-side logic on free-tier infrastructure to demonstrate viability without capital expenditure.Development was executed in structured phases: requirements gathering, system design (UML, wireframes, class diagrams, data model), Android implementation across twelve incremental steps, Supabase Edge Function deployment, Firestore rules and index configuration, Next.js admin panel development, and rigorous manual testing. The iterative approach allowed issues such as the Deno gRPC incompatibility and the Firestore transaction atomicity design to be identified and resolved before they could affect the deployed system. Testing confirmed that all fifteen test cases - functional, security, and performance - passed on a physical Android device against the live backend. User Acceptance Testing with peer participants validated that the interface was intuitive across all three roles. The system is deployed

and operational, providing a working prototype that demonstrates the full end-to-end cafeteria ordering workflow and a clear technical foundation for future production deployment.

8.2 Achievements

Significant milestones were accomplished during the development of the USIU Cafeteria Ordering System, spanning all three application tiers and covering the full scope of the original requirements. Complete three-role system across two applications. The most substantial achievement was delivering a fully functional system serving three distinct user roles. Students and cafeteria staff use a native Android app - the student interface provides five tabs (Menu, Cart, Orders, Pre-orders, Profile/Wallet) while the staff interface provides three tabs (StaffOrders, StaffMenu, StaffWallet). Administrators use a separate Next.js web panel for menu and user management. Role-based routing from LoginActivity ensures that each Android user lands on the correct interface after authentication, with no manual configuration required.

Real-time order lifecycle with FCM notifications. The end-to-end order flow - from a student placing an order to receiving a push notification when the meal is ready - was implemented and verified on physical devices. Firestore snapshot listeners provide sub-second status propagation between the student and staff interfaces, and Firebase Cloud Messaging delivers the "order ready" notification reliably within approximately 3 seconds. This replaces all verbal communication between student and counter staff for routine orders. Atomic in-app wallet with full audit trail. The Firestore runTransaction implementation for wallet order placement atomically decrements the student balance, creates the order document, and records the transaction in a single operation with no risk of partial writes. The staff wallet top-up and deduction operations are equally atomic, and every transaction records the staffId of the staff member who performed it, providing a complete financial audit trail without a separate accounting system.

Pre-order scheduling with automated cut-off processing. The pre-order system - allowing students to schedule wallet-funded meals in advance with a 10:00 AM (lunch) or 5:00 PM (dinner) cut-off - was fully implemented and deployed. The Supabase Edge Function process-cutoff, triggered by cron-job.org at the scheduled times, queries all pending pre-orders, processes wallet deductions atomically, dispatches FCM confirmations or cancellation notices, and schedules recurring pre-

orders for the next occurrence. This server-side automation required no persistent server and runs entirely on the free tier.

Firestore security rules and composite indexes. Five composite Firestore indexes and a comprehensive set of security rules with role-based helper functions (isAuth(), isOwner(), isStaff(), isAdmin(), isPrivileged()) were deployed to production. The rules correctly enforce data partitioning - students can only read their own orders, staff can read all orders, and the walletBalance field is write-protected against client-side modification. All three security test cases confirmed these rules operate as specified. Next.js admin web panel deployed on Vercel. A separate Next.js 14 web application provides administrators with a dashboard for menu management, staff account creation (via the Firebase Admin SDK), and a directory of all registered users. The panel is deployed on Vercel's free tier with HTTPS, uses server-side Firebase Admin SDK calls exclusively, and requires no Android device access for administrative operations. Free-tier cloud infrastructure. The entire system - Firebase Firestore, Firebase Auth, FCM, Supabase Edge Functions, cron-job.org scheduling, and Vercel hosting - operates within free service tiers, incurring zero monthly cost. This demonstrates that production-grade, multi-tier university software can be built and deployed without infrastructure budget, which is a meaningful outcome for resource-constrained institutions.

8.3 Challenges Encountered

Although the USIU Cafeteria Ordering System was successfully built and deployed, the development process presented several technical and architectural challenges that required significant problem-solving effort. Firebase Admin SDK incompatibility in Deno edge runtime. The most significant technical challenge arose when implementing the Supabase Edge Functions. The Firebase Admin SDK uses gRPC internally for Firestore communication, but the Deno runtime used by Supabase Edge Functions does not support gRPC. Initial attempts to connect from the edge function to Firestore produced persistent connection errors that were not immediately obvious from the error messages. The resolution required explicitly configuring the Firebase Admin SDK with db.settings({ preferRest: true }) to force the SDK to use the REST API instead of gRPC. This single configuration line was critical and is not prominently documented; identifying it required deep investigation of the Firebase Admin SDK internals and Deno runtime limitations. Firestore transaction design for atomic wallet operations. Designing the wallet order

placement flow to be fully atomic required careful structuring of the Firestore runTransaction call. An early implementation performed the balance check and balance deduction as separate Firestore writes, which created a race condition where two simultaneous order placements could both pass the balance check before either had decremented the balance. Migrating the logic into a single runTransaction block - which reads the current balance, verifies sufficiency, decrements it, and creates the order document in one atomic operation - resolved the race condition but required significant restructuring of the FirestoreRepository and CartFragment interaction.

Fragment navigation and state preservation. An early architectural decision to use replace fragment transactions for the bottom navigation caused the CartViewModel to reset whenever the user navigated away from the Cart tab. This meant items added to the cart were lost if the user browsed back to the Menu. Migrating from replace to a hide/show fragment management pattern in MainActivity and StaffMainActivity preserved each fragment's lifecycle and ViewModel state across tab switches, resolving the issue, but required a full refactor of the navigation setup.First-login forced password change for staff accounts. Staff accounts are created by the admin via the Firebase Admin SDK with a temporary password. Ensuring that staff were forced to change this password on their first login required a firstLogin boolean field on the user document in Firestore and a ChangePasswordActivity that intercepted navigation immediately after login when the flag was true. Coordinating the flag reset (written after a successful updatePassword call) with the subsequent navigation to StaffMainActivity required careful sequencing to avoid race conditions between the Firestore write and the activity transition.

FCM token management. FCM tokens are generated per device and can be rotated by the platform. An early implementation stored the FCM token only at registration time, which caused notifications to stop arriving after a token refresh. The solution required implementing onNewToken in MyFirebaseMessagingService to write the refreshed token back to the user's Firestore document whenever FCM issued a new one. This ensured the fcmToken field on the users document was always current, regardless of when the rotation occurred.

Multi-role security rule design. Writing Firestore security rules that correctly handled three roles (student, staff, admin) across five collections without duplicating logic required several iterations. The initial rules were written with inline role checks per collection, leading to repetitive and error-

prone conditions. Refactoring to use shared helper functions (isAuth(), isOwner(), isStaff(), isAdmin(), isPrivileged()) and testing all boundary conditions via the Firebase Console Rules Playground produced a clean, maintainable rule set that passed all three security test cases.

8.4 Future Work

While the current version of the USIU Cafeteria Ordering System meets its core objectives and is deployed as a functional prototype, several enhancements are identified for future iterations that would improve usability, extend functionality, and prepare the system for full production scale. Online payment integration. The current wallet system requires students to deposit cash at the counter to credit their in-app balance. Integrating a mobile money gateway such as M-Pesa STK Push would allow students to top up their wallets remotely from anywhere, removing the last physical dependency on the counter for wallet management. This would significantly improve convenience for students and reduce the cash-handling workload for staff.

Pre-order discoverability and onboarding. UAT feedback identified that the "Pre-orders" section label was not immediately self-explanatory to first-time users. A future enhancement would add a brief onboarding screen or contextual tooltip on first launch, explaining how pre-ordering works and when cut-off times apply. This change requires minimal engineering effort but would meaningfully reduce the learning curve for new users. Push notification preferences. Currently, all FCM notifications (order ready, pre-order confirmation, pre-order cancellation) are sent without user configuration. A future notification preferences screen in the Profile/Wallet tab would allow students to opt out of specific notification types or set quiet hours, improving the experience for students who prefer not to receive notifications during lectures.

Order history analytics for staff and admin. The current system stores all orders in Firestore but provides no aggregate reporting. A future admin dashboard module - built into the existing Next.js admin panel - could display daily order volumes, peak hours, most popular menu items, and wallet transaction summaries. This data would help cafeteria management plan staffing levels and stock quantities more accurately, directly improving operational efficiency. Repeat order and favourites shortcut. A common friction point for regular users is rebuilding the same cart from scratch each

day. A future "Reorder" button on a completed order in the Orders history would populate the cart with the same items in one tap, significantly reducing the time required to place routine orders.

Cash payment confirmation by staff. Currently, cash orders appear on the StaffOrders screen identically to wallet orders, with no mechanism for staff to confirm that cash was physically received before preparing the order. A future enhancement would add a "Confirm Payment" step for cash orders, preventing kitchen preparation from beginning before payment is collected at the counter. Scalability to multiple cafeteria outlets. The current data model assumes a single cafeteria. Introducing a outletId field on menuItems and orders documents would allow the system to scale to multiple on-campus food outlets - such as a coffee kiosk or a snack bar - each with their own staff interface and menu, all under the same Firebase project and admin panel. Full automated test suite. The current testing approach relies on manual execution of test cases. A future iteration would introduce a JUnit and Mockito unit test suite for CartViewModel, WalletViewModel, and FirestoreRepository, along with Espresso UI tests for the critical order placement and wallet deduction flows. Automated testing would enable continuous integration checks on every code change and reduce regression risk as the feature set grows.

9 REFERENCES

Abdul, A. M., Mohammad, A. A. K., Reddy, P. V., Nuthakki, P., Kancharla, R., Joshi, R., & Raja, N. K. (2022). Enhancing security of mobile cloud computing by trust- and role-based access control. *Scientific Programming, 2022*, Article 9995023. https://doi.org/10.1155/2022/9995023

Android Developers. (2024). *Android developer guides*. Google. https://developer.android.com/guide

Dennis, A., Wixom, B. H., & Roth, R. M. (2020). *Systems analysis and design* (7th ed.). Wiley.

Epiloksa, H. A., Kusumo, D. S., & Adrian, M. (2022). Effect of MVVM architecture pattern on Android based application performance. *Jurnal Media Informatika Budidarma*, *6*(4). https://doi.org/10.30865/mib.v6i4.4545

Firebase. (2024). *Firebase documentation*. Google. https://firebase.google.com/docs

Google. (2024). *Material Design 3*. Google. https://m3.material.io/

Maniratnam, U., Pandey, S. K., Srivastava, U., Kumar, P., & Sharma, P. (2020). Automated canteen ordering system using Android. *International Journal for Research in Applied Science and Engineering Technology*, *8*(V). https://doi.org/10.22214/ijraset.2020.5154

Mokar, M. A., Fageeri, S. O., & Fattoh, S. E. (2019). Using Firebase Cloud Messaging to control mobile applications. In *Proceedings of the 2019 International Conference on Computer, Control, Electrical, and Electronics Engineering (ICCCEEE)* (pp. 1–5). IEEE. https://doi.org/10.1109/ICCCEEE46830.2019.9071008

Moroney, L. (2017). *The definitive guide to Firebase: Build Android apps on Google's mobile platform*. Apress. https://doi.org/10.1007/978-1-4842-2943-9

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education Limited.

Supabase. (2024). *Supabase documentation*. Supabase Inc. https://supabase.com/docs

10 APPENDIX A: USER MANUAL FOR THE USIU CAFETERIA ORDERING SYSTEM

This appendix serves as a quick-start guide for the three user roles of the USIU Cafeteria Ordering System: Students, Cafeteria Staff, and System Administrators.

A.1 For Students (Android App)

Getting Started

Registration: Open the app and tap **Register**. Enter your name, student ID, USIU email address, and create a password. Your in-app wallet will be created automatically with a balance of KES 0.00.

Login: Enter your email and password. The system will read your role from Firestore and direct you automatically to the Student Dashboard.

Placing a Regular Order

Browse the Menu: On the **Menu** tab, use the category chips at the top of the screen - All, Mains, Snacks, Drinks, Specials - to filter items. Items that are sold out display a red "Sold Out" chip and a disabled Add to Cart button. Tap **Add to Cart** on any available item. A snackbar will confirm the addition.

Review the Cart: Navigate to the **Cart** tab. Use the + and − stepper buttons to adjust item quantities. Removing the last unit of an item removes it from the cart entirely. The subtotal and an estimated wait time update in real time at the bottom of the list.

Select a Payment Method:

- **Wallet:** Select the Wallet option. Your current balance is shown alongside the toggle. If the balance is insufficient, a warning appears and the Place Order button is disabled. Top up your wallet at the cafeteria counter before proceeding.

- **Cash:** Select the Cash option. No balance check is performed. You will pay physically at the counter when you collect your order.

Tap **Place Order** to confirm. For wallet orders, the deduction and order creation happen atomically - if either fails, neither is committed.

Track Your Order: Go to the **Orders** tab. Your active order displays a three-step status bar that updates in real time without any manual refresh:

- *Pending* (grey chip) - order received, awaiting preparation.
- *Preparing* (blue chip) - cafeteria staff have started your meal.
- *Ready* (green chip) - your meal is ready for collection at the counter.

A push notification will also arrive on your device when the status changes to Ready, even if the app is in the background.

Smart Pre-order (Scheduling a Meal in Advance)

Navigate to the **Pre-orders** tab and tap the + button (bottom-right corner).

In the dialog that appears:

1. Select the **Meal Type**: Lunch or Dinner.
2. Choose a **Date** and **Pickup Time** for the scheduled meal.
3. Select your items and quantities from the menu list.
4. Optionally, toggle **Recurring** and select the day(s) of the week to repeat the pre-order automatically each week.

Tap **Save**. The pre-order will appear in your list with a "Scheduled" badge.

Important: Pre-orders are funded entirely from your wallet. No deduction occurs at scheduling time. The deduction is processed automatically at the cut-off time - 10:00 AM EAT for Lunch slots, 5:00 PM EAT for Dinner slots. Ensure your wallet has sufficient funds before the cut-off.

If the balance is insufficient at cut-off time, the pre-order will be cancelled and you will receive a push notification. You may then top up and reschedule.
To cancel a pre-order, tap the **Cancel** button on its card. Cancellation is only possible while the status is "Scheduled" - once the cut-off has processed and the status is "Confirmed", the order cannot be cancelled.

Managing Your Wallet

Go to the **Profile/Wallet** tab.

- **Balance:** Your current wallet balance is displayed in a prominent card at the top of the screen.
- **Top-up:** Wallet top-ups are handled at the cafeteria counter only. Bring cash and provide your Student ID to the staff member, who will credit your account. The updated balance will appear on your Profile screen within seconds.
- **Transaction History:** Scroll down below the balance card to see a full list of credits (top-ups) and debits (order payments and pre-order deductions), each with a date, description, and amount.
- **Logout:** Tap the Logout button at the bottom of the screen to sign out and return to the Login screen.

A.2 For Cafeteria Staff (Android App)

Login and First-Time Setup

Your account is created by the System Administrator. You will receive a temporary password via email or in person. Open the app, enter your email and temporary password, and tap **Login**. On your first login, you will be redirected automatically to a **Change Password** screen. You must set a new password before any other screen is accessible. This step is mandatory and cannot be skipped.

Processing Orders (StaffOrders Tab)

The **StaffOrders** tab is your primary operational screen. It displays all active orders from all students - those in Pending, Preparing, or Ready status - in a single real-time list. Each card shows the student's name, the ordered items and quantities, the total amount, the payment method (wallet or cash), and the current status.

Advance each order through the workflow using the action button on its card:

Action	Result
Tap **Start Preparing**	Status changes from Pending to Preparing. Student's in-app chip turns blue.
Tap **Mark Ready**	Status changes from Preparing to Ready. A push notification is sent automatically to the student's device. Student's chip turns green.
Tap **Mark Collected**	Order is finalised. For cash orders, collect physical payment from the student before tapping this button.

Orders disappear from the active list once marked Collected.

Managing the Menu (StaffMenu Tab)

- **Toggle Availability:** Flip the switch on any item card to instantly mark it as "Sold Out" or "Available". The change is reflected on all connected student devices within seconds.
- **Add Item:** Tap the + FAB. Fill in the Name, Description, Category, Price, and Image URL. Tap Save.
- **Edit or Delete:** Tap an existing item card to open the edit dialog. Modify any field and save, or tap Delete to remove the item permanently.

Managing Student Wallets (StaffWallet Tab)

This screen is used to credit or debit a student's in-app wallet balance in exchange for cash transactions at the counter.

1. Enter the student's **ID number** in the lookup field and tap **Find Student**. The student's name and current balance are displayed for confirmation.
2. Enter the **amount** of the transaction.
3. Tap **Credit** to add funds (top-up after receiving cash), or **Debit** to subtract funds (for corrections such as refunding an overcharge).
4. Tap **Confirm**. The transaction is recorded atomically and the student's balance updates immediately. Your staff ID is recorded on every transaction as part of the audit trail.

A.3 For Administrators (Web Panel)

Access

The admin panel is browser-based. Open a desktop browser and navigate to the deployed Vercel URL for your institution's admin panel. Log in with your administrator credentials. If this is your first login, you will be redirected to a password change page before accessing the dashboard.

Note: The admin panel is role-gated. Any user who does not hold the admin role will be redirected to the login page immediately after authentication.

Menu Management

The **Menu Management** page displays a full data table of all menu items. Each row shows the item name, category, price, current availability status, and image URL.

- **Add Item:** Click the **Add Item** button above the table. Fill in the Name, Description, Category (dropdown), Price, Image URL, and initial availability. Click Save. The item is immediately visible on all student Menu screens.
- **Edit:** Click the Edit (pencil) icon on any row. Modify the fields and save.
- **Delete:** Click the Delete (trash) icon on any row. This action is permanent.
- **Toggle Availability:** Flip the availability switch in any row to show or hide an item on the student menu without deleting it.

Staff Account Creation

Navigate to the **Staff Accounts** page.

Fill in the new staff member's **Full Name**, **Email**, and a **Temporary Password**. Click **Create Staff Account**. The system calls a server-side API route that:

1. Creates a Firebase Authentication account with the provided credentials.
2. Writes a Firestore user document with role: "staff" and firstLogin: true.

On their first login, the staff member will be forced to change the temporary password before accessing any operational screen. A success message on this page confirms the new account's email address.

Users Directory

The **Users Directory** page displays two separate tables:

- **Students:** Name, Student ID, Email, Wallet Balance, Date Joined.
- **Staff and Admins:** Name, Email, Role, First-Login Status (indicates whether the staff member has completed their first-login password change).

Both tables are read in real time from Firestore and reflect the current state of all registered accounts.

11 APPENDIX B: TECHNICAL DEPLOYMENT AND MAINTENANCE GUIDE

This appendix provides a technical reference for deploying, configuring, and maintaining the cloud infrastructure of the USIU Cafeteria Ordering System. It assumes familiarity with Firebase, Supabase, and Git.

B.1 Deployment Prerequisites

The following accounts and tools must be in place before deployment begins. All services operate on free tiers - no credit card or billing account is required.

Accounts required:

- Firebase (Spark free tier) - Firestore, Authentication, Cloud Messaging
- Supabase (free tier) - Edge Functions hosting
- Vercel (free tier) - Next.js admin panel hosting
- cron-job.org (free tier) - scheduled HTTP trigger jobs
- GitHub - source code repository

Tools required:

- Android Studio (Ladybug 2024.2.1 or newer) - for building and signing the Android APK
- Node.js v18 or newer - for Next.js admin panel development
- Git - version control
- Supabase CLI - for deploying and managing Edge Functions
- Firebase CLI - for deploying Firestore security rules and composite indexes

B.2 Environment Variables and Secrets

Sensitive credentials are never stored in source code. Each platform has its own secure storage mechanism. The table below lists every secret used by the system and where it is stored.

Platform	**Variable Name**	**Purpose**	**Where to Store**
Android App	FUNCTIONS_SECRET	Shared secret used to authenticate HTTP calls from the app to the	gradle.properties - excluded from version control via .gitignore. Exposed to app code at build time as BuildConfig.FUNCTIONS_SECRET.

Platform	Variable Name	Purpose	Where to Store
		Supabase Edge Functions.	
Supabase	FUNCTIONS_SECRET	Same shared secret - validated by Edge Functions on every incoming request via the Authorization: Bearer header.	supabase secrets set FUNCTIONS_SECRET=...
Supabase	FIREBASE_SERVICE_ACCOUNT	Firebase service account JSON key - used by the Firebase Admin SDK inside the Edge Functions to	supabase secrets set FIREBASE_SERVICE_ACCOUNT="$(cat key.json)"

Platform	Variable Name	Purpose	Where to Store
		read Firestore and send FCM notifications.	
Vercel	FIREBASE_ADMIN_*	Firebase Admin SDK service account credentials - used by Next.js server-side API routes to create staff accounts and write Firestore documents.	Vercel Project Settings → Environment Variables (server-side only, not exposed to the browser).
Vercel	NEXT_PUBLIC_FIREBASE_*	Client-side Firebase configuration (API key, project ID, etc.) - used by the	Vercel Project Settings → Environment Variables (prefixed NEXT_PUBLIC_, safe to expose to the client bundle).

Platform	Variable Name	Purpose	Where to Store
		Next.js browser client to authenticate and read Firestore within the bounds of security rules.	

B.3 Deployment Steps

Step 1 - Firebase Rules and Indexes

Deploy the Firestore security rules and composite indexes to the Firebase project using the Firebase CLI:

firebase deploy --only firestore:rules
firebase deploy --only firestore:indexes
Both commands read from firestore.rules and firestore.indexes.json in the project root. Run these commands once on initial setup and again whenever either file is modified.

Step 2 - Supabase Edge Functions

Navigate to the supabase/functions/ directory and deploy each function independently:
supabase functions deploy process-cutoff
supabase functions deploy notify-order-ready
Set the required secrets once using the Supabase CLI:

supabase secrets set FUNCTIONS_SECRET=your-shared-secret-here
supabase secrets set FIREBASE_SERVICE_ACCOUNT="$(cat service-account-key.json)"
Secrets are injected into the Deno runtime as environment variables at invocation time and are never stored in the function source code.

Critical implementation note: Both Edge Functions must include the line db.settings({ preferRest: true }) immediately after initialising the Firebase Admin SDK Firestore client. The Deno runtime used by Supabase does not support gRPC, which is the default transport protocol for the Firebase Admin SDK. Without this setting, all Firestore calls from the Edge Functions will fail silently. Setting preferRest: true forces the SDK to use the Firestore REST API, which is fully compatible with the Deno HTTP stack.

Step 3 - cron-job.org Scheduling

Create two HTTP POST jobs in cron-job.org with the following configuration:

Job Name	Endpoint URL	Schedule (UTC)	Request Body
cafeteria-lunch-cutoff	https://[project-ref].supabase.co/functions/v1/process-cutoff	0 7 * * * (07:00 UTC = 10:00 AM EAT)	{"mealSlot":"lunch"}
cafeteria-dinner-cutoff	https://[project-ref].supabase.co/functions/v1/process-cutoff	0 14 * * * (14:00 UTC =	{"mealSlot":"dinner"}

Job Name	Endpoint URL	Schedule (UTC)	Request Body
		5:00 PM EAT)	

For both jobs, add the following HTTP header to authenticate the request:

Authorization: Bearer YOUR_FUNCTIONS_SECRET
Replace [project-ref] with the Supabase project reference ID found in the Supabase project settings. Replace YOUR_FUNCTIONS_SECRET with the same value stored in the Supabase secrets.

Step 4 - Android APK Generation

1. In Android Studio, open the project and navigate to **Build → Generate Signed Bundle / APK**.
2. Select **APK** and click Next.
3. Select your release keystore (create one on first release using the **Create new** option - store the keystore file and passwords securely).
4. Select the **release** build variant and click Finish.
5. The signed APK is generated at app/build/outputs/apk/release/app-release.apk.

Install the APK directly on student and staff devices via USB or a file transfer mechanism. The app requires Android 7.0 (API Level 24) or higher.

Note: The FUNCTIONS_SECRET value must be present in gradle.properties before building. Confirm the line FUNCTIONS_SECRET=your-secret exists in that file and that gradle.properties is listed in .gitignore.

Step 5 - Next.js Admin Panel (Vercel)

1. Push the repository to GitHub (the admin panel source resides in the admin/ subdirectory).
2. In Vercel, click **Add New Project** and import the GitHub repository.
3. Set the **Root Directory** to admin in the Vercel project settings.
4. Add all environment variables listed in Section B.2 - both the server-side Admin SDK credentials and the NEXT_PUBLIC_ client-side Firebase config.
5. Click **Deploy**. Vercel will build and deploy the Next.js application automatically.

All subsequent pushes to the main branch on GitHub will trigger an automatic redeployment, making admin panel updates straightforward to roll out.

B.4 Firestore Composite Indexes

Firestore requires explicitly defined composite indexes for any query that combines a field filter with an ordering on a different field, or that applies multiple equality or range filters simultaneously. Without these indexes, the corresponding queries will throw a runtime error. The five indexes below must be deployed before the system is used.

Deploy via Firebase Console → Firestore Database → Indexes tab, or include them in firestore.indexes.json and deploy with firebase deploy --only firestore:indexes.

Collection	Index Fields	Query Purpose
orders	userId (Ascending) + createdAt (Descending)	Student order history - retrieve a student's past orders sorted by most recent.
orders	status (Ascending) + createdAt (Ascending)	Staff active order queue - retrieve all non-collected orders in arrival order.

Collection	Index Fields	Query Purpose
preOrders	userId (Ascending) + createdAt (Descending)	Student pre-order listing - retrieve a student's scheduled pre-orders.
preOrders	mealSlot (Ascending) + scheduledDate (Ascending) + status (Ascending)	Cut-off function batch query - retrieve all pre-orders for a given meal slot and date with status "scheduled".
walletTransactions	userId (Ascending) + createdAt (Descending)	Wallet transaction history - retrieve a student's wallet activity sorted by most recent.

B.5 Common Maintenance Tasks

The table below covers the most frequent operational and maintenance scenarios that system administrators and support staff will encounter.

Task	Procedure
Top up a student's wallet	Staff member uses the StaffWallet tab in the Android app: enter the student's ID, enter the amount, tap Credit. No admin panel action is needed.
Reset a staff member's password	The admin panel does not support password resets directly. Delete the staff account from the Users Directory page, then

Task	Procedure
	re-create it on the Staff Account Creation page with a new temporary password.
Remove a student account	Firebase Console → Authentication → locate the user → Delete. Also delete the corresponding document from the users collection in Firestore.
Add or update a menu item	Admin Web Panel → Menu Management → Add Item, or click Edit on an existing row. Changes are reflected on all student devices within seconds via Firestore snapshot listeners.
Check pre-order cut-off execution	Login to cron-job.org → select the relevant job → open the Execution History tab. Each successful run returns HTTP 200 with a JSON summary of confirmed and cancelled pre-orders.
Redeploy an Edge Function after a code change	Run supabase functions deploy process-cutoff or supabase functions deploy notify-order-ready from the project root. The updated function is live immediately.
Monitor Firestore quota usage	Firebase Console → Firestore Database → Usage tab. The Spark free tier allows 50,000 reads, 20,000 writes, and 20,000 deletes per day. Usage at prototype scale is well within these limits.
Rotate the FUNCTIONS_SECRET	Generate a new secret string. Update it in gradle.properties and rebuild the APK. Update it in Supabase using supabase secrets set

Task	Procedure
	FUNCTIONS_SECRET=new-value. Update it in both cron-job.org job headers. All four locations must be updated before distributing the new APK, otherwise Edge Function calls from the old APK will be rejected.
Add a new admin account	Firebase Console → Authentication → Add User. Then manually create a Firestore document in the users collection with the new UID and role: "admin". Set firstLogin: true to enforce a password change on first access to the admin panel.